



《数据库技术及应用 A》

课程实践项目

项目名称:

《魔法少女的魔女审判》多角色权限管理系统

任课教师: L老师

学 院: 魔法学院

成 员 一 : 暗杀楼

成 员 二 : 寻觅意义的理科生

成 员 三 : mortal

学 期: 2025-2026 学年度第 1 学期

2025 年 12 月



目录

1.项目背景与概述	1
1.1 项目背景与灵感来源	1
1.2 游戏世界观与系统映射	1
1.3 项目价值与意义	3
1.4 开发环境与技术栈	3
1.4.1 开发环境	3
1.4.2 核心技术栈与架构	4
1.4.3 技术选型概要	4
1.5 团队分工与协作	5
2.需求分析	6
2.1 系统功能需求	6
2.1.1 用户管理模块	6
2.1.2 魔女档案管理模块	6
2.1.3 审判系统投票模块	7
2.1.4 处刑平台管理模块	8
2.1.5 图鉴系统模块	9
2.1.6 五子棋对弈系统模块	10
2.1.7 数据可视化模块	11
2.2 权限需求分析	11
2.2.1 四层权限体系设计	11
2.2.2 各角色权限矩阵	13
2.2.3 数据隔离要求	15
2.3 非功能性需求	19
2.3.1 性能需求	19
2.3.2 安全性需求	19
2.3.3 可维护性需求	19
2.3.3 体验与还原需求	19
3.概念结构设计	20
3.1 系统核心实体识别	20
3.1.1 核心基础实体（8 个）	20
3.1.2 审判系统实体（3 个）	21
3.1.3 处刑平台实体（2 个）	22
3.1.5 其他系统实体（3 个）	22
3.2 实体间联系	23
3.2.1 核心基础实体联系	23
3.2.2 审判系统实体联系	24
3.2.3 处刑平台实体联系	25
3.2.4 游戏系统实体联系	25
3.3 E-R 模型设计	26
3.3.1 核心基础实体 E-R 图	26
3.3.2 审判投票子系统 E-R 图	28
3.3.3 处刑平台子系统 E-R 图	29



3.3.4 图鉴系统 E-R 图	29
3.3.5 其它 E-R 图	30
3.4 完整系统 E-R 图	30
4. 逻辑结构设计	34
4.1 关系模式设计 (18 个)	34
4.1.1 核心基础表 (8 个)	34
4.1.2 审判系统表 (3 个)	34
4.1.3 处刑平台表 (2 个)	34
4.1.4 图鉴系统表 (2 个)	34
4.1.5 其他系统表 (3 个)	34
4.2 数据表详细设计 (18 张表)	35
4.2.1 Role 表 (角色表)	35
4.2.2 User 表 (用户表)	35
4.2.3 Island 表 (岛屿表)	35
4.2.4 Batch 表 (批次表)	36
4.2.5 Witch 表 (魔女表) (核心表, 43 字段)	36
4.2.6 UserWitch 表 (用户-魔女关联表)	38
4.2.7 IslandRegulator 表 (岛屿-监管员关联表)	38
4.2.8 IslandWarden 表 (岛屿-典狱长关联表)	39
4.2.9 TrialSession 表 (审判会话表)	39
4.2.10 TrialParticipant 表 (审判参与者表)	40
4.2.11 TrialNotification 表 (审判通知表)	40
4.2.12 ExecutionPlatform 表 (处刑台表)	40
4.2.13 PlatformMovementLog 表 (处刑台移动记录表)	41
4.2.14 Evidence 表 (证物表)	42
4.2.15 Record 表 (记录表)	42
4.2.16 GomokuMatchLog 表 (五子棋对局日志表)	42
4.2.17 OperationLog 表 (操作日志表)	43
4.2.18 WitchDescBackup 表 (魔女描述备份表)	43
4.3 数据完整性约束	43
4.3.1 实体完整性约束	43
4.3.2 参照完整性约束	44
4.3.3 用户自定义完整性约束	44
5. 物理结构设计	46
5.1 存储结构设计	46
5.1.1 数据库文件结构	46
5.1.2 Schema 逻辑组织	46
5.1.3 表空间分配与估算	47
5.1.4 数据类型优化选择	47
5.1.5 存储结构设计小结	48
5.2 数据库文件组织	48
5.2.1 表结构设计规范	48
5.2.2 索引设计策略	50
5.2.3 存储过程设计	51
5.2.4 数据库文件组织小结	52



5.3 性能优化策略	52
5.3.1 数据库连接管理优化	52
5.3.2 索引优化策略	54
5.3.3 存储过程优化	55
5.3.4 事务管理优化	56
5.3.5 视图优化	57
6. 数据库实现	58
6.1 数据库创建与初始化	58
6.1.1 数据库创建脚本	58
6.1.2 Schema 创建	58
6.1.3 数据表创建脚本	59
6.2 存储过程设计	62
6.3 视图设计	62
6.3.1 v_WitchFullProfile (魔女完整档案视图)	62
6.3.2 v_WitchPublic (魔女公开信息视图)	64
6.4 触发器设计	64
7. 系统实现	65
7.1 三层架构实现	65
7.1.1 数据访问层 (DAL) 设计	65
7.1.2 业务逻辑层 (BLL) 设计	71
7.1.3 用户界面层 (UI) 设计	77
7.2 数据库连接管理	86
7.2.1 DBHelper 类设计	86
7.2.2 连接池管理	88
7.3 安全机制实现	88
7.3.1 密码加密 (SHA256 + Salt)	88
7.3.2 SQL 注入防护	89
7.3.3 权限验证机制	90
8. 核心功能演示及对应代码	91
8.1 用户登录与权限验证	91
8.1.1 登录界面与流程	91
8.1.2 密码验证代码	93
8.1.3 角色判断与界面跳转	94
8.2 四层权限主界面	95
8.2.1 国家端 Admin 主界面 (Form1_Admin)	95
8.2.2 岛屿管理者端 Meruru 主界面 (Form1_Regulator)	95
8.2.3 典狱长端 Warden 主界面 (Form1_Warden)	96
8.2.4 普通魔女端 Witch 主界面 (PhoneForm)	96
8.3 魔女档案管理 (43 字段完整档案)	97
8.3.1 魔女档案查询 (WitchDetailForm)	97
8.3.2 魔女档案编辑 (WitchEditForm)	100
8.3.3 教育与工作经历管理 (JSON 存储)	103
8.3.4 添加新魔女 (WitchAddForm)	105
8.3.5 创建新魔女账号	109
8.4 审判投票系统 (7 个状态流转)	111



8.4.1 创建审判会话 (CreateTrialDialog)	112
8.4.2 投票功能实现 (TrialVotingForm)	114
8.4.3 实时通知机制 (NotificationPopupForm)	117
8.4.4 投票结果统计 (VotingResultDialog)	118
8.4.5 处刑确认 (TrialExecutionConfirmForm)	120
8.4.6 单机多账号切换支持	122
8.5 处刑平台管理	123
8.5.1 处刑台信息管理 (ExecutionPlatformManagementForm)	123
8.5.2 处刑台升起/返回 (PlatformMoveDialog)	124
8.5.3 刑具管理 (ToolManagementDialog)	125
8.5.4 移动日志查看 (MovementLogViewForm)	126
8.5.5 匿名记录设计	128
8.6 图鉴系统 (5 个图鉴)	129
8.6.1 人物图鉴 (PokedexForm)	129
8.6.2 证物图鉴 (EvidenceForm)	133
8.6.3 记录图鉴 (RecordsForm)	136
8.6.4 地图图鉴 (MapForm)	138
8.6.5 规定图鉴 (RulesForm, 自定义字体)	140
8.7 五子棋对弈系统	142
8.7.1 五子棋对局 (GomokuBoardForm)	142
8.7.2 积分系统实现	147
8.7.3 对局日志 (GomokuMatchLogForm)	147
8.7.4 排行榜功能	148
8.8 数据可视化大屏 (LiveCharts)	149
8.8.1 核心 KPI 指标区	150
8.8.2 全局状态分布区	150
8.8.3 分岛屿状态分布区	150
8.8.4 批次状态矩阵区	151
8.8.5 岛屿选择功能	151
8.9 智慧大模型	152
8.10 录音机界面	153
8.11 对话界面	155
9 项目总结与展望	157
9.1 项目完成情况	157
9.1.1 功能完成度统计	157
9.1.2 代码规模统计	157
9.1.3 数据库规模统计	158
9.2 技术难点与解决方案	158
9.3 项目创新点	159
9.4 不足与改进方向	160
9.5 个人收获与体会	160
参考文献	161



1.项目背景与概述

1.1 项目背景与灵感来源

本系统依托 C# WinForms 框架与 SQL Server 数据库开发，核心目标为复刻《魔法少女的魔女审判》游戏中“国家 - 梅露露 - 典狱长 - 普通魔女”的多层级管理场景，最终构建起一套集数据存储管理、精细化权限控制、可视化交互操作于一体的全流程数据库管理系统。

项目灵感来源于日本动漫文字推理游戏《魔法少女的魔女审判》（魔法少女ノ魔女裁判）。该作品以独特的世界观和复杂的角色关系为特色，讲述了在一个神秘的魔女审判岛屿上，魔法少女们被囚禁并接受审判的故事。

作为数据库系统课程设计项目，我们选择以这一游戏为背景，不仅是因为其丰富的角色设定和复杂的管理体系，更是因为其中蕴含的多层级权限管理、复杂业务流程和大量数据建模需求，为数据库设计提供了极佳的实践场景。



图 1 《魔法少女的魔女审判》游戏官网截图

1.2 游戏世界观与系统映射

《魔法少女的魔女审判》构建了以“魔女审判岛屿”为核心的封闭管理世界观，其游戏系统与故事背景设定与《弹丸论破》系列相近。游戏围绕 13 名少女的命运展开叙事：她们因体内携带“魔女因子”被认定为潜在魔女，遭囚禁于孤岛牢房中，被迫卷入一场自相残杀的死亡游戏。监狱管理方刻意营造高压环境，刺激少女体内的魔女因子失控生长，致使被关押者异化为异形并引发自相残杀；后续会通过举行“魔女审判”判定杀人凶手，并对其实施处刑。

玩家将以樱羽艾玛、二阶堂希罗等普通魔女的身份参与游戏，核心玩法分为两大环节：一是日常环节，采用文字冒险游戏的形式，玩家可在此阶段展开搜查，收集案件相关证据；二是魔女裁判环节，玩家需操控主角，利用此前搜集的证据指出其他角色发言中的矛盾之处，通过辩论推理最终指认真凶。游戏中的案件设计多融入少女们的专属魔法能力，使其更具独特性；同时剧情大量触及主要角色的背景故事与心灵创伤，并着重刻画凶手的处刑场景，进一步强化了世界观的沉浸感。[1][2]



我们基于《魔法少女的魔女审判》核心情节展开合理延伸，设计出“国家 - 梅露露（岛屿管理者） - 典狱长 - 普通魔女”四层级管理体系，并构建了“多岛屿 - 多批次”的核心运行框架，具体设定如下：

国家层（Admin）承担核心管控职责，负责抓捕世界上持有“魔女因子”的少女，并将其分流至岛屿 1、岛屿 2 两座专属审判岛屿接受后续流程。每座岛屿设置两层管理者：上层为岛屿管理者（梅露露），其核心特征是拥有双重身份与账号——既是统筹岛屿运营的管理者（如岛屿 1 的 meruru_regulator 账号），也是潜藏在普通魔女中的幕后黑手（对应普通魔女身份的 meruru 账号）；下层为典狱长，服从岛屿管理者的指令，主要负责刑具管理与魔女审判的具体组织执行。普通魔女被分配至对应岛屿后，将由岛屿管理者按“13 人 / 批次”进行编组（含 12 位普通魔女与 1 位伪装其中的岛屿管理者），通过多轮审判推进流程，期间不断有魔女因被杀或被判定有罪处刑而淘汰；若岛屿管理者的幕后黑手身份暴露，系统将立即终止该批次，将剩余普通魔女全部处死并开启下一批次。

该“多岛屿分流 + 多批次循环”的设计大幅提升了系统的复杂度与工程量。为进一步丰富内容趣味性，我们还在游戏原有角色基础上，引入了《魔女之旅》、《魔女宅急便》、《憧憬成为魔法少女》、《魔法少女小圆》、《中二病也要谈恋爱》、《青春猪头少年不会梦到兔女郎学姐》、《重返未来：1999》等多部魔法少女系列影视作品及游戏中的经典人物，让整个审判体系更具多元性。

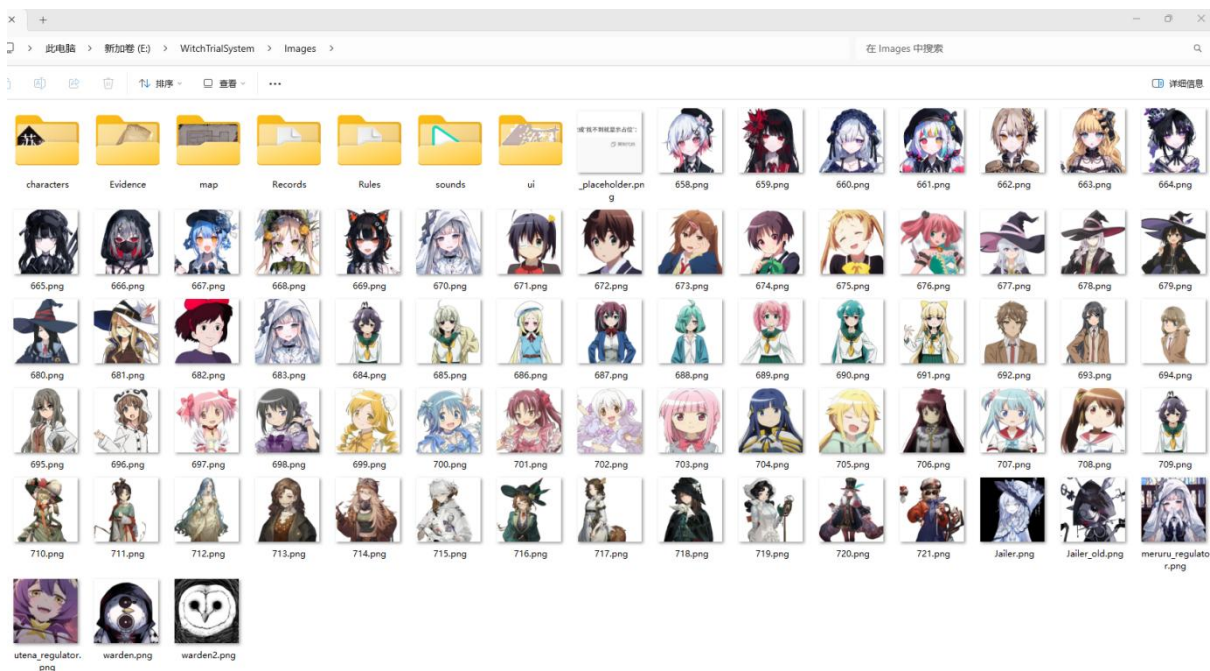


图 2 项目 Images 文件夹截图

本项课设专为《魔法少女的魔女审判》游戏粉丝打造，核心是还原 13 位魔法少女的岛屿生活，复刻“手机交互 + 监管系统”的经典场景，让粉丝沉浸式代入剧情。正如《游戏设计艺术》作者 Jesse Schell 的定义：幽默感源于“毫无关联的事物经形式重构后突然相连”，而惊喜则是大脑本能渴求的愉悦体验[3]——我们正是循着这份设计逻辑，为粉丝藏了诸多巧思彩蛋：

审判投票结果宣布时，会猝不及防响起滑稽音效，用反差感打破紧张氛围；“照相”待开发功能中，还埋了一张充满趣味的合影彩蛋——这张源自网络的照片里，原游戏



中作为幕后管理者的冰上梅露露（右 1），与被其杀害的黑部奈叶香（左 1）同框出镜，用粉丝熟知的剧情梗制造意外笑点。（注：该照片若涉及侵权，请联系删除）



图 3 照相功能背景图片彩蛋

1.3 项目价值与意义

本项目兼具《数据库技术及应用 A》课程设计与《魔法少女的魔女审判》游戏二创的双重属性，其核心价值与意义可从多维度展开：

从课程设计维度来看，我们期望通过本次项目开发充分完成专业能力训练，不仅扎实掌握 SQL、C# WinForms 等课内核心知识，还能完整体验项目全流程开发，并熟练运用 Git 工具实现多人协作，真正将课堂理论转化为实践能力；

从游戏二创维度而言，本项目旨在助力《魔法少女的魔女审判》玩家更深度地体验、还原游戏核心情节，强化沉浸式代入感，为游戏粉丝提供专属的情绪价值；

此外，我们也希望这一创新型课程设计项目，能为后续学习数据库课程的同学带来启发与思考——期望大家能突破课程内容本身的局限，在各类课程设计中挖掘趣味切入点，以兴趣为驱动开展创作，最终在实践中实现对课程知识的掌握与升华。

1.4 开发环境与技术栈

1.4.1 开发环境

本系统基于 Windows 桌面应用开发环境构建，核心配置如下：

表 1 开发环境表

类别	详情
操作系统	Windows 10/11（64 位）
开发工具	Visual Studio 2022（集成开发环境）
辅助工具	Kiro/Cursor（AI 编程辅助）、SQL Server Management Studio (SSMS) 19（数据库管理）
核心依赖	.NET 9.0 SDK、Git 2.40+（版本控制）
代码托管	GitHub（团队协作与代码存储）

项目的 GitHub 网址为：<https://github.com/Yipintianxia-MiddleRingRoad/WitchTrialSystem>

1.4.2 核心技术栈与架构

系统采用经典三层架构设计，技术选型聚焦稳定性、开发效率与功能适配性，核心技术栈如下：

表 2 核心技术与架构表

类别	详情
架构模式	三层架构（表示层→业务逻辑层→数据访问层）
前端框架	C# WinForms（.NET 9.0） - 桌面应用界面开发
编程语言	C# 12.0（面向对象编程）
数据访问	ADO.NET - 原生数据库交互（支持事务、参数化查询）
数据库	SQL Server 2022 - 关系型数据库（含 18 张核心表）
关键组件	LiveCharts.WinForms（数据可视化）、SHA256+Salt（密码加密）
辅助技术	JSON 序列化（复杂数据存储）、Markdown 渲染（文档展示）

1.4.3 技术选型概要

本项目技术选型概要如下：

表 3 技术选型概要表

类别	详情
前端设计	基于 WinForms，搭配自定义字体、背景图片与热键交互，实现沉浸式体验
后端实现	ADO.NET+ 存储过程，高效处理数据访问与复杂业务逻辑，保障数据一致性
数据库设计	依托 SQL Server 的事务、约束与索引优化，支撑多岛屿 - 多批次数据管理
安全防护	参数化查询防 SQL 注入，SHA256 加盐哈希存储密码，保障系统安全

本系统技术栈选型贴合桌面数据库管理系统的核心需求，兼顾开发效率与运行稳定性，三层架构设计使职责边界清晰，为后续功能扩展与维护奠定了基础。

1.5 团队分工与协作

本团队分工见下表：

表 4 团队分工详情表

成员	职务	任务分工
暗杀楼	组长	1. 项目整体构思、需求分析与架构设计 2. 核心功能开发（魔女档案管理、审判投票系统、处刑平台核心逻辑、图鉴系统主体……）； 3. 前后端全栈开发； 4. 数据库设计与实现； 5. 项目统筹与进度管理、Git 版本控制主导、文档编写； 6. 系统测试、Bug 修复与性能优化
寻觅意义的理科生	组员	准备数据集，负责“图鉴·证物”“图鉴·记录”，录音机，对话功能开发、文档编写
mortal	组员	准备数据集，负责“图鉴·地图”“图鉴·说明”开发，后期对界面进行优化调整、文档编写



2.需求分析

2.1 系统功能需求

结合《魔法少女的魔女审判》游戏世界观（即，13 名含“魔女因子”的少女被囚禁孤岛，通过高压环境诱发自相残杀，经审判投票判定凶手并处刑），深度还原游戏剧情。系统围绕“多岛屿 - 多批次”管理体系，实现以下七大核心功能模块：

2.1.1 用户管理模块

• 游戏背景：

还原游戏中“国家 - 梅露露 - 典狱长 - 普通魔女”的层级管理设定，不同角色拥有不同权限边界。在《魔法少女的魔女审判》原剧情中，魔女“梅露露”作为岛屿幕后黑手，潜藏在普通魔女之中，管理着每一批次魔女的详细信息。典狱长作为本岛屿监管者（如：梅露露），作为其使魔，听从安排。而背后国家层为更神秘的幕后组织，抓捕疑似含有魔女因子的少女，将她们分配至魔女岛。

• 核心功能：

- 1.用户登录：输入用户名 / 密码，通过 SHA256+Salt 验证，按 RoleID 跳转对应界面（Admin→Form1_Admin、Witch→PhoneForm 等）。
- 2.权限验证：按四层权限体系（Admin>Meruru>Warden>Witch）控制数据可见范围（Admin 查看所有岛屿，Witch 仅查看自身）。
- 3.账号创建：Admin 可直接创建魔女账号；Meruru 仅可为所属岛屿、状态为“分配至岛屿”且无账号的魔女创建账号（用户名 = 囚犯编号，默认密码 123456）。
- 4.密码安全：密码以加盐哈希形式存储，不存储明文，防止泄露。

2.1.2 魔女档案管理模块

• 游戏背景：

游戏中每个魔女均有详细个人档案，是审判和管理的核心依据，系统还原普通魔女完整档案。



图 4 原游戏魔女图鉴·证物

• 核心功能:

- 1.档案查询: 支持按 ID、囚犯编号、姓名、岛屿 / 批次、状态筛选, 按权限展示 (Admin 查看全部, Witch 仅看自身), A4 版式布局展示 43 字段 (基本信息、教育经历、家庭背景等)。
- 2.档案编辑: Admin 可编辑所有字段, 分标签页设计 (基本信息、教育 / 工作经历等), 支持数据验证 (邮箱格式、出生日期合理性等), 通过存储过程批量更新。
- 3.经历管理: 动态添加、编辑、删除教育 / 工作经历, 以 JSON 格式存储。
- 4.状态与批次管理: Admin 可修改魔女状态 (待抓捕 / 审判中 / 死亡等), 分配批次, 支持状态和批次统计。
- 5.头像管理: 按囚犯编号匹配头像, 无头像时使用默认占位图, 在图鉴、投票等界面展示。

2.1.3 审判系统投票模块

• 游戏背景:

还原游戏核心玩法, 典狱长发起审判, 到场魔女投票指认凶手, 最终通过处刑台执行处刑的完整流程。



图 5 原游戏中玩家亲自点击处刑按钮画面

• 核心功能:

- 1.审判创建:典狱长选择所属岛屿、批次及 2-13 名参与魔女,创建审判会话(状态为 Pending),系统自动发送通知。
- 2.投票管理:典狱长启动投票(状态转为 Voting),魔女登录后接收弹窗通知,进入投票界面选择对象,支持单机多账号切换投票,实时显示投票进度。
- 3.结果处理:典狱长查看投票统计结果,宣布处刑对象(状态转为 Confirmed),魔女确认处刑对象后跳转处刑界面。
- 4.审判完成:魔女点击处刑按钮,典狱长标记审判为 Completed,支持任意阶段取消审判(状态转为 Cancelled)。

2.1.4 处刑平台管理模块

• 游戏背景:

游戏中每个岛屿有若干个固定处刑台,处刑时需升起到审判庭,处刑后返回原位,系统还原该场景的动态管理。



图 6 处刑台即将上升至审判庭中央

• 核心功能:

- 1.处刑台操作：支持处刑台从原位（1-49）升起至审判庭（50 号位）、从审判庭返回原位，验证前置条件（如升起时审判庭无占用）。
- 2.刑具管理：Regulator 可编辑处刑台的刑具名称、类型及描述，Admin 仅查看，Warden 无编辑权限。
- 3.移动日志：记录处刑台移动详情（起始位置、目标位置、移动时间等），支持按日期、编号、移动类型筛选，匿名记录不泄露操作人。
- 4.状态统计：统计空闲 / 使用中处刑台数量及使用率，在可视化大屏展示。

2.1.5 图鉴系统模块

• 游戏背景:

还原游戏中图鉴收集功能，魔女可通过手机界面查看人物、证物、地图等关键信息，辅助参与审判。图鉴内的信息随着游戏的进行不断更新（在游戏脚本中，游戏作者采用了 NaniScript 格式更新具体的信息），这里我们默认是上层的管理人员可以对数据库的字段信息进行修改。



图 7 原游戏图鉴·人物界面

• 核心功能:

- 1.人物图鉴：卡片式展示魔女基本信息（头像、姓名、囚犯编号等），支持分页浏览和搜索，按权限过滤数据。
- 2.证物图鉴：展示证物编号、名称、图片及描述，底部热键或方向键切换，左侧显示基础信息，右侧展示详情。
- 3.记录图鉴：渲染 Markdown 格式历史记录（如调查书、日记），支持长标题完整显示，滚动查看内容。
- 4.地图图鉴：加载 4 张岛屿地图，通过左下角透明按钮或数字键切换，无需数据库支持。
- 5.规定图鉴：展示岛屿管理规定，使用自定义字体渲染，支持 Markdown 语法，左下角热键切换。

2.1.6 五子棋对弈系统模块

• 游戏背景:

模拟游戏中魔女等待审判期间在娱乐室的休闲娱乐场景，增加角色互动性。

• 核心功能:

- 1.对局功能：15×15 标准棋盘，支持单设备双人对弈，随机分配黑白方，自动判定五子连珠胜负。
- 2.特殊功能：支持悔棋（魔法按钮）、和棋（伪证按钮）、认输（疑问按钮），需对方确认后生效。
- 3.积分与日志：胜利 +30 分、失败 -10 分、平局 +5 分，记录对局双方信息、时长、步数等，支持按条件筛选对局记录。



4.排行榜：展示 TOP13 魔女积分排名，前三名标记特殊图标，在手机界面和可视化大屏显示。

2.1.7 数据可视化模块模块

• 游戏背景：

为管理员、监管员、典狱长提供岛屿运营数据统计，辅助决策管理，贴合游戏中“高压监管”设定。

• 核心功能：

- 1.魔女统计：展示魔女总数、岛屿分布、状态分布饼图、批次分布柱状图，按权限过滤数据范围。
- 2.五子棋统计：展示积分排行榜（TOP10）及最近 10 场对局记录。
- 3.审判统计：统计进行中、已完成及总审判数量。
- 4.处刑台统计：通过数字卡片和饼图展示空闲 / 使用中处刑台数量及使用率，30 秒自动刷新数据。

2.2 权限需求分析

权限管理是数据库系统安全性与业务合规性的核心保障。本系统基于《魔法少女的魔女审判》游戏世界观，构建了“国家层-岛屿监管员-典狱长-普通魔女”的四层权限体系，通过“数据库字段约束+应用层逻辑验证”双重机制，实现角色权限的精准管控与数据隔离，确保不同层级用户仅能访问权限范围内的功能与数据。

2.2.1 四层权限体系设计

1 设计背景

本系统需支持多岛屿、多批次魔女的分级管理，核心诉求包括：① 最高权限角色掌控系统全局配置；② 中间层级角色聚焦所属岛屿的监管与运营；③ 底层角色仅拥有个人操作与业务参与权限。基于此，设计严格的自上而下权限体系，既贴合游戏剧情中的管理逻辑，又满足数据库系统“最小权限”的安全原则。

2 权限层级结构

系统权限逐层递减，每个角色对应唯一 RoleID（数据库中通过角色表关联），具体层级与权限范围如下：

权限递减规则：国家层(Admin, RoleID=1) > 岛屿监管员(Meruru, RoleID=2) > 典狱长(Warden, RoleID=3) > 普通魔女(Witch, RoleID=4)



表 5 角色权限分类表

层级	角色名称 (RoleID)	核心权限范围	典型职责
第一层	国家层管理员 (1)	所有岛屿、所有批次、所有功能	系统全局配置、全量数据管理、用户账号创建
第二层	岛屿监管员 (2)	所属岛屿、只读权限、监督功能	监督岛屿运营、查看所属岛屿数据、参与娱乐功能
第三层	典狱长 (3)	所属岛屿、审判管理、处刑平台操作	发起审判、管理处刑台、维护刑具信息
第四层	普通魔女 (4)	仅个人数据、审判参与权限	参与投票、确认处刑结果、使用个人功能 (照相、五子棋)

3 各层级详细说明

①国家层管理员 (Admin, RoleID=1)

示例账号: admin

对应界面: Form1_Admin.cs

核心职责:

- 数据管理: 管理所有岛屿的魔女档案 (43 字段完整权限), 支持新增、编辑、批次分配操作;
- 系统配置: 创建各角色用户账号, 分配岛屿与批次关联关系;
- 业务管控: 查看全量系统操作日志, 管理所有岛屿的审判会话与处刑平台;
- 数据可视化: 访问所有岛屿的可视化大屏数据, 进行全局运营分析。

②岛屿监管员 (Meruru, RoleID=2)

示例账号: meruru_regulator, utena_regulator

对应界面: Form1_Regulator.cs

核心职责:

- 监督权限: 查看所属岛屿的魔女档案 (43 字段, 只读) 与可视化大屏数据;
- 业务参与: 参与五子棋对局, 无业务管理权限。

权限限制: 不可编辑任何数据、不可创建审判会话、不可管理处刑平台、不可跨岛屿访问数据。

③典狱长 (Warden, RoleID=3)

示例账号: warden, warden2



对应界面：Form1_Warden.cs

核心职责：

- 审判管理：创建所属岛屿的审判会话，选择参与魔女，发起投票、确认投票结果、宣布处刑对象；
- 处刑平台：管理 49 个处刑台的升起/返回操作，维护刑具信息，查看处刑台移动日志；
- 数据访问：查看所属岛屿的魔女档案（43 字段，只读）与可视化数据。

权限限制：不可编辑魔女信息、不可参与投票、不可跨岛屿访问数据。

④普通魔女（Witch, RoleID=4）

示例账号：原游戏人物名字英文小写（如 ema），以及编号 671-711

对应界面：PhoneForm.cs（模拟手机界面）

核心职责：

- 个人数据：查看自己的完整档案（43 字段），查看其他魔女的公开描述；
- 审判参与：接收审判通知，参与投票、确认处刑对象、点击处刑按钮；
- 娱乐功能：参与五子棋对局，查看五子棋排行榜，使用照相功能。

权限限制：不可查看其他魔女完整档案、不可编辑任何数据、无管理权限。

2.2.2 各角色权限矩阵

下表以“功能类别-功能项”为维度，清晰呈现四个角色的权限差异，其中 ✓ 表示拥有权限，✗ 表示无权限，括号内为权限范围补充说明。

表 6 角色权限矩阵表



功能类别	功能项	Admin (RoleID=1)	Meruru (RoleID=2)	Warden (RoleID=3)	Witch (RoleID=4)
数据 访问 范围	所有岛屿数据	√	×	×	×
	所属岛屿数据	√	√	√	×
	所属批次数据	√	√	√	√
	仅自己数据	√	√	√	√
魔女 档案 管理	查看魔女列表	√ (所有岛屿)	√ (所属岛屿)	√ (所属岛屿)	×
	查看魔女详情	√ (43 字段)	√ (43 字段只读)	√ (43 字段只读)	√ (仅自己)
	编辑魔女信息	√	×	×	×
	添加新魔女	√	×	×	×
	修改魔女状态	√	×	×	×
	批次分配	√	×	×	×
	创建审判会话	√ (所有岛屿)	×	√ (所属岛屿)	×
审判 投票 系统	开始投票	√	×	√	×
	参与投票	×	×	×	√
	查看投票结果	√	×	√	×
	宣布处刑对象	√	×	√	×
	确认处刑	×	×	×	√
处刑 平台	完成审判	√	×	√	×
	查看处刑台列表	√ (所有岛屿)	×	√ (所属岛屿)	×
	处刑台升起	√	×	√	×



管理	处刑台返回	√	×	√	×
	刑具管理	√	×	√	×
	查看移动日志	√(所有岛屿)	×	√(所属岛屿)	×
系统 管理	查看操作日志	√	×	×	×
	用户管理	√	×	×	×
公共 功能	图鉴系统访问	√	√	√	√
	五子棋功能	√	√	√	√

权限特殊说明

1.审判角色分离：典狱长负责“审判流程管理”，普通魔女负责“投票参与”，权限互斥确保审判公正性，避免管理者干预投票结果；

2.处刑日志匿名性：处刑台移动日志仅记录时间、位置变化，不记录操作人及操作方式，保护典狱长隐私；

3.图鉴开放性：人物、证物等图鉴为公共信息，所有角色均可访问，不涉及数据安全风险。

2.2.3 数据隔离要求

数据隔离是权限体系的核心落地手段，系统采用“数据库层过滤+应用层验证”的双重机制，确保数据访问的精准性与安全性，避免越权访问。

1 数据隔离原则

- 最小权限原则：用户仅能访问完成业务所需的最小数据集；
- 层级隔离原则：按“国家层-岛屿-批次-个人”层级过滤数据，上层可访问下层数据，下层不可跨层级访问；
- 双重验证原则：数据库层通过 SQL 条件过滤数据，应用层通过权限校验方法二次确认，防止绕过前端的越权操作。

2 数据隔离实施方案

2.1 数据库层隔离（核心实现）

在数据访问层（DAL）通过动态拼接 SQL 条件，根据当前用户角色自动添加数据过滤规则，确保查询结果仅包含权限范围内的数据。以下为魔女档案查询的核心实现代码（对应文件：DAL/PermissionDAL.cs）：

```
/// <summary>  
/// 根据用户权限查询魔女档案（43 字段完整查询）  
/// </summary>  
/// <param name="username">当前登录用户名</param>  
/// <param name="nameLike">魔女名称模糊查询条件（可选）</param>  
/// <returns>权限范围内的魔女档案数据集</returns>  
public DataTable GetWitchesByPermission(string username, string? nameLike = null)  
{
```



```
// 1. 查询当前用户的权限信息（角色、所属岛屿、批次）
const string userPermissionSql = @"
SELECT u.UserID, r.Name AS RoleName,
       u.IslandID, u.BatchID
FROM wt.[User] u
INNER JOIN wt.Role r ON u.RoleID = r.RoleID
WHERE u.Username = @username";

var userDt = DBHelper.ExecDataTable(userPermissionSql,
                                    new SqlParameter("@username", username));
var userRow = userDt.Rows[0];
string roleName = userRow["RoleName"].ToString();
int userIslandId = Convert.ToInt32(userRow["IslandID"]);
int userBatchId = Convert.ToInt32(userRow["BatchID"]);

// 2. 构建魔女档案查询基础 SQL（包含 43 个字段）
string baseSql = @"
SELECT WitchID, PrisonerNo, Name, Magic, Status, IslandID, BatchID,
       AvatarPath, DescriptionPublic, Gender, BirthDate, Height, Weight,
       /* 此处省略其他 35 个字段，实际开发需完整列出 */
FROM wt.Witch
WHERE 1=1"; // 占位条件，便于后续拼接

var parameters = new List<SqlParameter>();

// 3. 按角色拼接权限过滤条件（核心数据隔离逻辑）
switch (roleName)
{
    case "Admin":
        // 管理员：无数据过滤，查询所有岛屿所有批次
        break;

    case "Meruru":
        // 监管员：仅查询所属岛屿数据
        baseSql += " AND IslandID = @islandId";
        parameters.Add(new SqlParameter("@islandId", userIslandId));
        break;

    case "Warden":
        // 典狱长：仅查询所属岛屿数据
        baseSql += " AND IslandID = @islandId";
        parameters.Add(new SqlParameter("@islandId", userIslandId));
        break;

    case "Witch":
        // 普通魔女：仅查询所属岛屿+所属批次数据
        baseSql += " AND IslandID = @islandId AND BatchID = @batchId";
        parameters.Add(new SqlParameter("@islandId", userIslandId));
        parameters.Add(new SqlParameter("@batchId", userBatchId));
        break;
}

// 4. 拼接模糊查询条件（可选）
if (!string.IsNullOrEmpty(nameLike))
{
    baseSql += " AND Name LIKE @nameLike";
    parameters.Add(new SqlParameter("@nameLike", $"%{nameLike}%"));
}
```



```
// 5. 排序并执行查询
baseSql += " ORDER BY PrisonerNo";
return DBHelper.ExecDataTable(baseSql, parameters.ToArray());
}
```

不同角色的最终查询 SQL 效果对比：

- Admin: SELECT * FROM wt.Witch ORDER BY PrisonerNo (无过滤) ;
- Meruru: SELECT * FROM wt.Witch WHERE IslandID = 1 ORDER BY PrisonerNo (岛屿过滤) ;
- Witch: SELECT * FROM wt.Witch WHERE IslandID = 1 AND BatchID = 1 ORDER BY PrisonerNo (岛屿+批次过滤) 。

2.2 核心业务模块的隔离实现

针对审判系统、处刑平台等核心模块，需结合业务场景设计专属隔离逻辑，确保数据安全与业务合规。

1. 审判系统数据隔离

典狱长仅能管理所属岛屿的审判会话，普通魔女仅能查看自己参与的审判，核心 SQL 示例：

```
-- 典狱长查询所属岛屿的审判会话
SELECT SessionID, IslandID, BatchID, Status, CreatedAt
FROM wt.TrialSession
WHERE IslandID = @IslandID -- 岛屿隔离条件
ORDER BY CreatedAt DESC;

-- 普通魔女查询自己参与的审判
SELECT ts.SessionID, ts.Status, tp.HasVoted, tp.HasConfirmedExecution
FROM wt.TrialSession ts
INNER JOIN wt.TrialParticipant tp ON ts.SessionID = tp.SessionID
WHERE tp.UserID = @UserID -- 个人参与隔离条件
ORDER BY ts.CreatedAt DESC;
```

2. 处刑平台数据隔离

处刑台与移动日志均按岛屿隔离，典狱长无法跨岛屿操作，核心 SQL 示例：

```
-- 典狱长查询所属岛屿的处刑台
SELECT PlatformID, PlatformNumber, CurrentPosition, ToolName
FROM wt.ExecutionPlatform
WHERE IslandID = @IslandID -- 岛屿隔离条件
ORDER BY PlatformNumber;

-- 典狱长查询所属岛屿的处刑台移动日志
SELECT LogID, PlatformNumber, FromPosition, ToPosition, MovementTime
FROM wt.PlatformMovementLog
WHERE IslandID = @IslandID -- 岛屿隔离条件

ORDER BY MovementTime DESC;
```

2.3 应用层权限验证（二次保障）



为防止通过修改前端参数绕过数据库过滤，需在应用层（BLL）提供权限验证方法，对关键操作进行二次校验。核心代码示例：

```
/// <summary>
/// 验证用户是否有权访问特定魔女的档案
/// </summary>
/// <param name="username">当前登录用户名</param>
/// <param name="witchId">目标魔女 ID</param>
/// <returns>true=有权访问，false=无权访问</returns>
public bool CanAccessWitch(string username, int witchId)
{
    const string verifySql = @"
SELECT CASE
    -- 管理员：全权限
    WHEN r.Name = 'Admin' THEN 1
    -- 监管员：仅访问所属岛屿魔女
    WHEN r.Name = 'Meruru' AND u.IslandID = w.IslandID THEN 1
    -- 典狱长：仅访问所属岛屿魔女
    WHEN r.Name = 'Warden' AND u.IslandID = w.IslandID THEN 1
    -- 普通魔女：仅访问自己
    WHEN r.Name = 'Witch' AND u.UserID = w.BindUserID THEN 1
    -- 其他情况：无权访问
    ELSE 0
END AS CanAccess
FROM wt.Witch w
INNER JOIN wt.[User] u ON w.IslandID = u.IslandID
INNER JOIN wt.Role r ON u.RoleID = r.RoleID
WHERE u.Username = @username AND w.WitchID = @witchId";

    var resultDt = DBHelper.ExecDataTable(verifySql,
        new SqlParameter("@username", username),
        new SqlParameter("@witchId", witchId));

    return resultDt.Rows.Count > 0 &&
        Convert.ToBoolean(resultDt.Rows[0]["CanAccess"]);
}
```

3 数据隔离测试场景

为验证数据隔离效果，需设计针对性测试场景，确保权限控制无漏洞，核心测试场景如下表：



表 7 数据隔离测试表

测试场景	测试用户	操作步骤	预期结果
跨岛屿数据访问	Meruru(RoleID=2, IslandID=1)	尝试查询 IslandID=2 的魔女列表	查询结果为空，系统提示“无权限访问该岛屿数据”
跨批次数据访问	Witch(RoleID=4, BatchID=1)	尝试查看 BatchID=2 的魔女档案	仅能看到本批次（BatchID=1）的魔女公开描述，无法查看完整档案
越权创建审判	Warden(RoleID=3, IslandID=1)	尝试创建 IslandID=2 的审判会话	下拉框仅显示 IslandID=1 的魔女，无法选择其他岛屿参与者
越权编辑数据	Meruru(RoleID=2)	点击魔女档案的“编辑”按钮	按钮灰显，鼠标悬浮提示“无编辑权限”

2.3 非功能性需求

2.3.1 性能需求

登录响应时间 < 1 秒，档案查询 < 500ms，处刑台操作 < 1 秒，图表加载 < 3 秒，支持 49 个处刑台同时管理。

2.3.2 安全性需求

参数化查询防止 SQL 注入，密码加盐哈希存储，按角色隔离数据，关键操作记录审计日志。

2.3.3 可维护性需求

现阶段，国家正大力推动医疗行业智能化升级，出台一系列政策支持基于眼底医学影像的眼科

2.3.3 体验与还原需求

界面贴合游戏哥特 + 日式美少女风格，使用自定义字体，支持热键交互和响应式布局，操作有音效反馈，错误提示清晰。贴合游戏剧情（如审判状态流转、处刑台位置规则），融入多批次循环机制（幕后黑手身份暴露则终止当前批次）。



3.概念结构设计

3.1 系统核心实体识别

3.1.1 核心基础实体（8 个）

• 角色

别名：角色基本信息

描述：这是魔女审判管理系统的核心数据结构，定义了系统四层权限体系的角色类型。

组成：角色 ID，角色名称

• 用户

别名：用户账号信息

描述：这是魔女审判管理系统的主要数据结构，定义了所有用户的账号信息和权限关联。

组成：用户 ID，用户名，密码哈希值，密码盐值，角色 ID，岛屿 ID，批次 ID，五子棋积分，头像路径

• 岛屿

别名：魔女审判岛屿信息

描述：这是魔女审判管理系统的地理数据结构，定义了魔女所在的审判岛屿。

组成：岛屿 ID，岛屿名称

• 批次

别名：魔女批次信息

描述：这是魔女审判管理系统的分组数据结构，定义了魔女的批次分配和编号体系。

组成：批次 ID，岛屿 ID，魔女数量，本岛批次编号

• 魔女档案

别名：魔女完整档案信息

描述：这是魔女审判管理系统的核心数据结构，定义了魔女的完整 43 字段档案信息，覆盖基本信息、个人详情、教育工作、家庭信息、个性特征、时间记录六大维度。

组成：魔女 ID，姓名，魔法能力，囚犯编号，状态，处刑结果，头像路径，岛屿 ID，批次 ID，公开描述，身份证号，曾用名，性别，出生日期，民族国籍，出生地，身高，体重，血型，住址，电话，邮箱，Line 账号，最高学历，教育经历，工作经历，家庭结构，父亲信息，母亲信息，其他家庭成员 1，其他家庭成员 2，其他家庭成员 3，



技能特长，兴趣爱好，理想梦想，厌恶事物，心理创伤，魔女化方法，备注，抓捕时间，离开时间，到达时间，死亡时间

• 用户魔女关联

别名：用户账号与魔女档案关联信息

描述：这是魔女审判管理系统的关联数据结构，建立用户账号与魔女档案的一对一关联关系。

组成：用户 ID，魔女 ID

• 岛屿监管员关联

别名：岛屿与监管员关联信息

描述：这是魔女审判管理系统的权限数据结构，定义了岛屿监管员（Meruru 角色）与岛屿的管理关系。

组成：用户 ID，岛屿 ID，监管员姓名

• 岛屿典狱长关联

别名：岛屿与典狱长关联信息

描述：这是魔女审判管理系统的权限数据结构，定义了典狱长（Warden 角色）与岛屿的管理关系。

组成：用户 ID，岛屿 ID，典狱长姓名

3.1.2 审判系统实体（3 个）

• 审判会话

别名：审判会话生命周期信息

描述：这是魔女审判管理系统的核心业务数据结构，定义了审判会话的完整生命周期，支持 6 种状态流转（Pending → Voting → Confirmed → Executing → Completed/Cancelled）。

组成：会话 ID，岛屿 ID，批次 ID，审判状态，创建人 ID，创建时间，投票开始时间，投票结束时间，处刑对象魔女 ID，处刑确认时间，完成时间

• 审判参与者

别名：审判参与者投票确认信息

描述：这是魔女审判管理系统的投票数据结构，记录每个审判会话的参与者及其投票和确认处刑状态。

组成：参与者 ID，会话 ID，魔女 ID，用户 ID，是否已投票，投票给魔女 ID，投票时间，是否已确认处刑，确认处刑时间

• 审判通知



别名：审判通知消息信息

描述：这是魔女审判管理系统的通知数据结构，记录发送给参与者的审判通知消息和已读状态。

组成：通知 ID，会话 ID，用户 ID，通知消息，是否已读，创建时间

3.1.3 处刑平台实体（2 个）

• 处刑台

别名：处刑台位置刑具信息

描述：这是魔女审判管理系统的处刑设施数据结构，管理每个岛屿的 49 个处刑台位置和刑具信息，支持处刑台升起到审判庭（50 号位）和返回原位的动态管理。

组成：处刑台 ID，岛屿 ID，处刑台编号，原位位置，当前位置，刑具名称，刑具类型，刑具描述，状态，创建时间，更新时间

• 处刑台移动记录

别名：处刑台移动历史信息

描述：这是魔女审判管理系统的日志数据结构，记录处刑台的所有移动历史，支持匿名记录和手动时间输入，用于审计追踪。

组成：日志 ID，岛屿 ID，处刑台 ID，处刑台编号，起始位置，目标位置，刑具名称，移动时间，是否手动输入时间，移动类型

• 3.1.4 图鉴系统实体（2 个）

• 证物

别名：魔女图鉴证物信息

描述：这是魔女审判管理系统的图鉴数据结构，存储魔女图鉴中的证物信息，支持图片展示和描述查看。

组成：证物 ID，证物编号，证物名称，证物描述，证物图片路径

• 记录

别名：魔女图鉴历史记录信息

描述：这是魔女审判管理系统的图鉴数据结构，存储魔女图鉴中的历史记录信息，支持 Markdown 格式文档展示。

组成：记录 ID，记录标题，Markdown 文件路径

3.1.5 其他系统实体（3 个）

• 五子棋对局日志

别名：五子棋对局历史信息



描述：这是魔女审判管理系统的游戏数据结构，记录五子棋对局的完整信息，支持积分系统和排行榜功能。

组成：对局 ID，玩家 1 用户名，玩家 1 姓名，玩家 2 用户名，玩家 2 姓名，开始时间，结束时间，玩家 1 结果，玩家 1 积分变化，玩家 2 结果，玩家 2 积分变化，总步数，对局时长

• 操作日志

别名：系统操作审计日志信息

描述：这是魔女审判管理系统的审计数据结构，记录系统所有关键操作，支持审计追踪和安全监控。

组成：日志 ID，用户 ID，操作用户名，操作类型，操作目标，操作详情，创建时间

• 魔女描述备份

别名：魔女公开描述备份信息

描述：这是魔女审判管理系统的备份数据结构，备份魔女的公开描述信息，支持数据恢复和历史追溯。

组成：魔女 ID，囚犯编号，公开描述备份

3.2 实体间联系

3.2.1 核心基础实体联系

• 角色-用户 (1:N)

一个角色可以对应多个用户，一个用户只能属于一个角色。

联系属性：角色 ID

• 用户-岛屿 (N:1)

多个用户可以属于同一个岛屿，一个用户只能属于一个岛屿(魔女/典狱长/监管员)。对于国家层而言，查看具体数据时需要选择岛屿，或者针对多岛屿信息一起查看

联系属性：岛屿 ID

• 用户-批次 (N:1)

多个用户可以属于同一个批次，一个用户只能属于一个批次(仅魔女用户)。

对于“幕后黑手”而言(如：冰上梅露露，柊舞缇娜)，创建了同姓名的多个账号，每次参与新一轮的审判登录新的账号。

联系属性：批次 ID

• 岛屿-批次 (1:N)

一个岛屿可以包含多个批次，一个批次只能属于一个岛屿。



联系属性：岛屿 ID

- 批次-魔女档案 (1:N)

一个批次可以包含多个魔女，一个魔女只能属于一个批次。

联系属性：批次 ID

- 岛屿-魔女档案 (1:N)

一个岛屿可以包含多个魔女，一个魔女只能属于一个岛屿。

联系属性：岛屿 ID

- 用户-魔女档案 (1:1)

一个魔女用户对应一个魔女档案，一个魔女档案对应一个魔女用户。通过用户魔女关联表建立关联。

联系属性：用户 ID，魔女 ID

- 用户-岛屿监管员关联 (1:1)

一个监管员用户对应一个岛屿，一个岛屿只能有一个监管员。

联系属性：用户 ID，岛屿 ID

- 用户-岛屿典狱长关联 (1:1)

一个典狱长用户对应一个岛屿，一个岛屿只能有一个典狱长。

联系属性：用户 ID，岛屿 ID

3.2.2 审判系统实体联系

- 岛屿-审判会话 (1:N)

一个岛屿可以有多个审判会话，一个审判会话只能属于一个岛屿。

联系属性：岛屿 ID

- 批次-审判会话 (1:N)

一个批次可以有多个审判会话，一个审判会话只能针对一个批次。

联系属性：批次 ID

- 用户-审判会话 (1:N)

一个用户（典狱长）可以创建多个审判会话，一个审判会话只能由一个用户创建。

联系属性：创建人 ID

- 魔女档案-审判会话 (1:N)

一个魔女可以成为多个审判会话的处刑对象，一个审判会话只能有一个处刑对象。

联系属性：处刑对象魔女 ID

- 审判会话-审判参与者 (1:N)



一个审判会话可以有多个参与者，一个参与者记录只能属于一个审判会话。

联系属性：会话 ID

- 魔女档案-审判参与者 (1:N)

一个魔女可以参与多个审判会话，一个参与者记录对应一个魔女。

联系属性：魔女 ID

- 用户-审判参与者 (1:N)

一个用户可以参与多个审判会话，一个参与者记录对应一个用户。

联系属性：用户 ID

- 审判会话-审判通知 (1:N)

一个审判会话可以产生多条通知，一条通知只能属于一个审判会话。

联系属性：会话 ID

- 用户-审判通知 (1:N)

一个用户可以收到多条通知，一条通知只能发送给用户。

联系属性：用户 ID

3.2.3 处刑平台实体联系

- 岛屿-处刑台 (1:N)

一个岛屿可以有多个处刑台（49 个），一个处刑台只能属于一个岛屿。

联系属性：岛屿 ID

- 处刑台-处刑台移动记录 (1:N)

一个处刑台可以有多条移动记录，一条移动记录只能属于一个处刑台。

联系属性：处刑台 ID

- 岛屿-处刑台移动记录 (1:N)

一个岛屿可以有多条处刑台移动记录，一条移动记录只能属于一个岛屿。

联系属性：岛屿 ID

3.2.4 游戏系统实体联系

- 用户-五子棋对局日志 (1:N)

一个用户可以参与多场五子棋对局，一场对局涉及两个用户（玩家 1 和玩家 2）。



联系属性：玩家 1 用户名，玩家 2 用户名

- 用户-操作日志 (1:N)

一个用户可以产生多条操作日志，一条操作日志只能属于一个用户。

联系属性：用户 ID

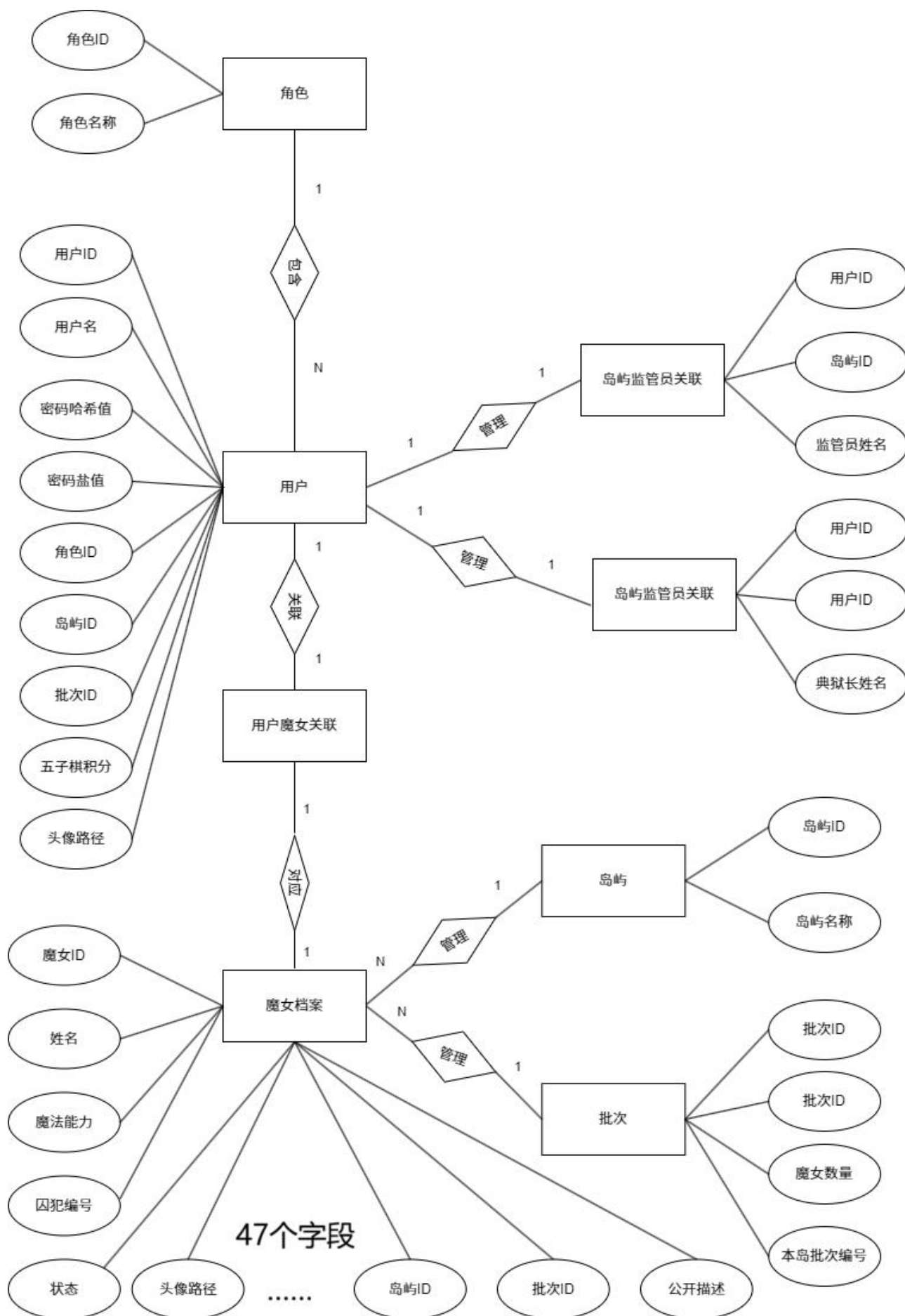
- 魔女档案-魔女描述备份 (1:1)

一个魔女档案对应一条描述备份记录，一条备份记录只能属于一个魔女档案。

联系属性：魔女 ID

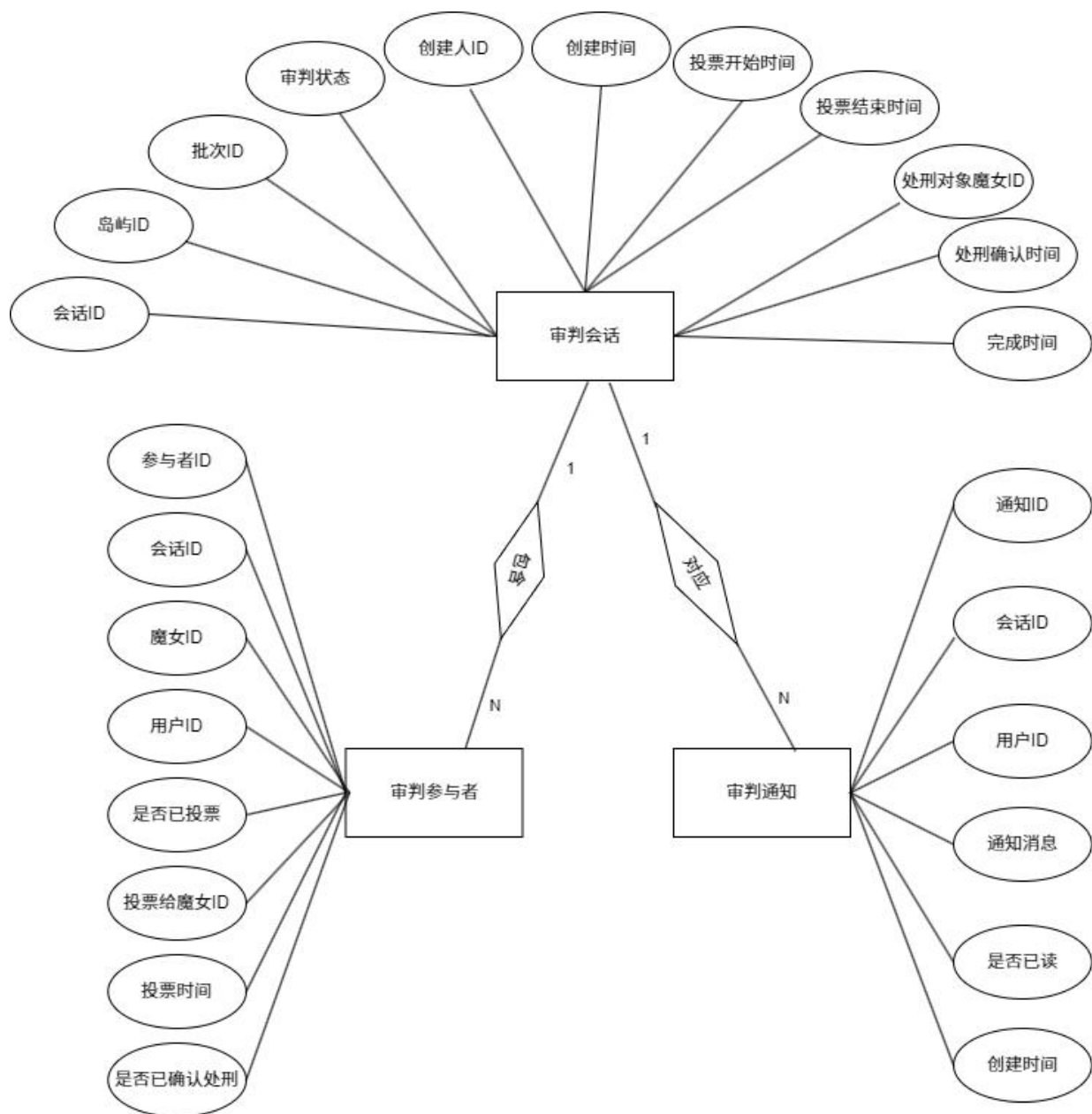
3.3 E-R 模型设计

3.3.1 核心基础实体 E-R 图



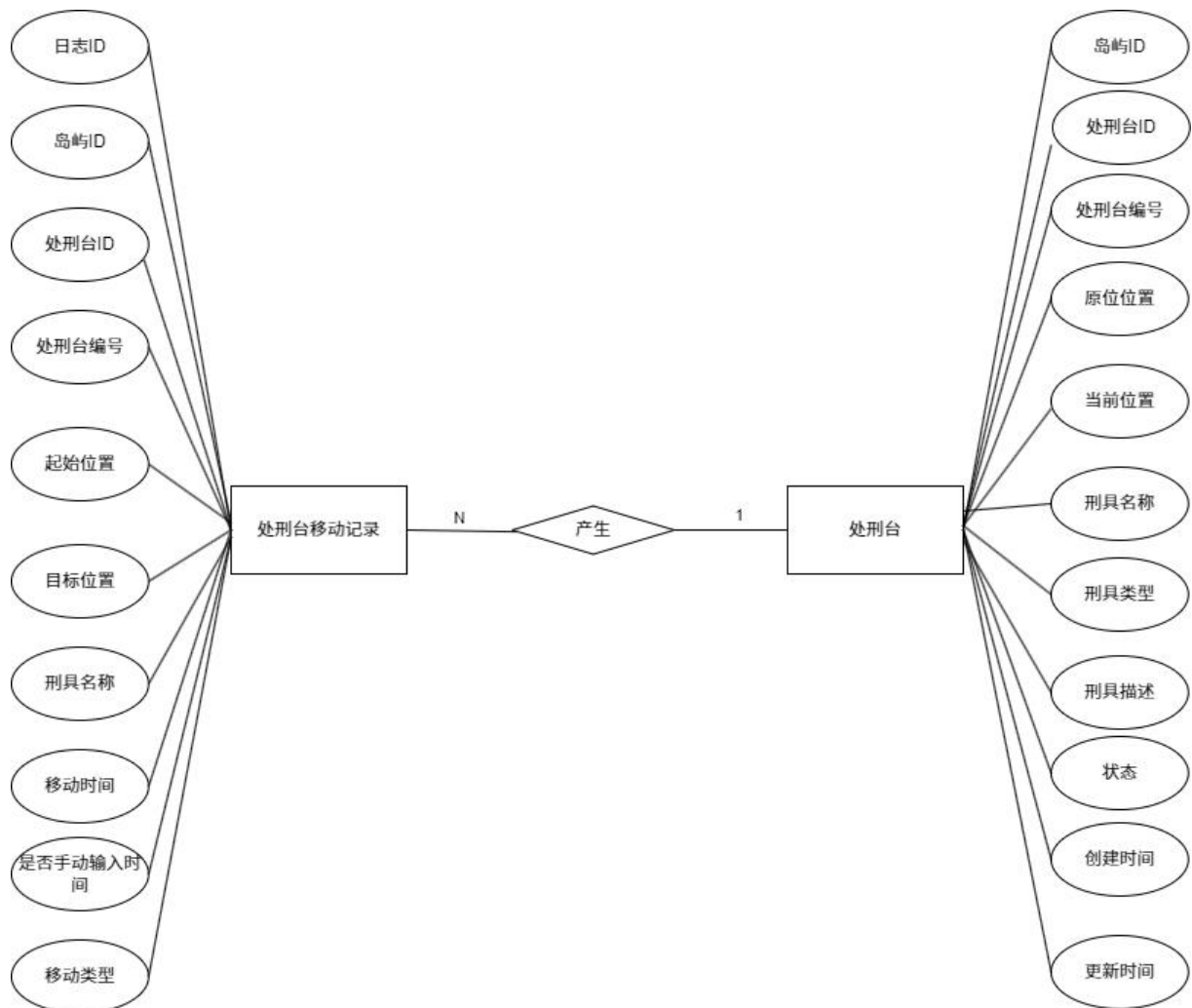


3.3.2 审判投票子系统 E-R 图

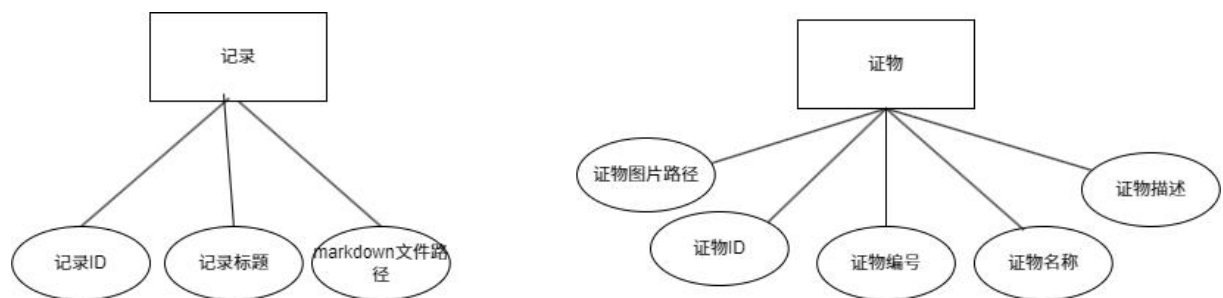




3.3.3 处刑平台子系统 E-R 图

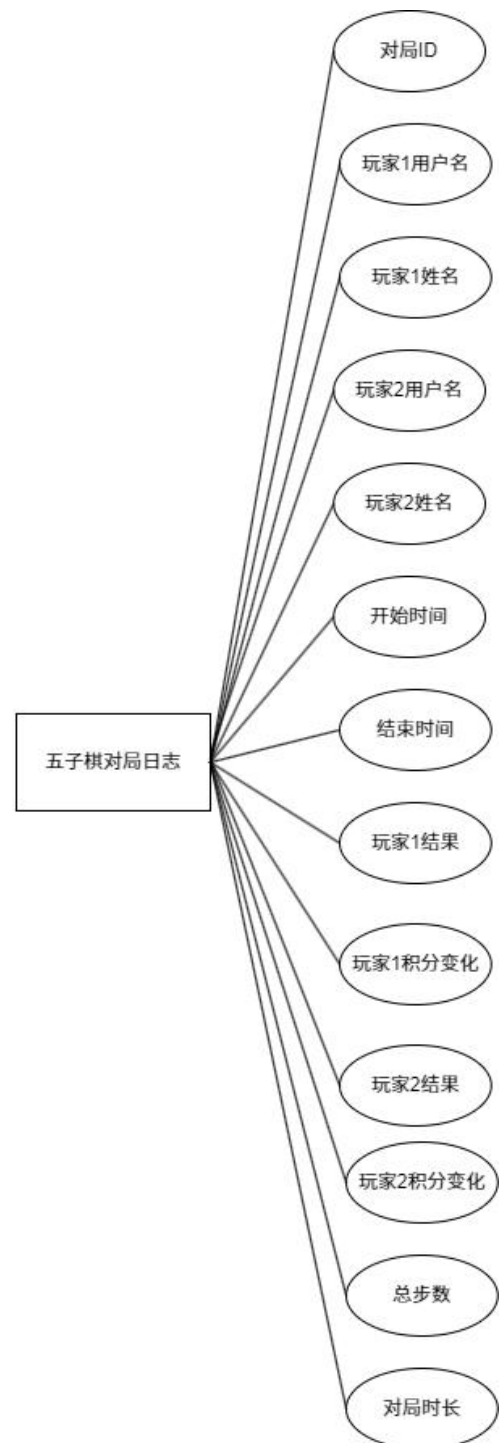
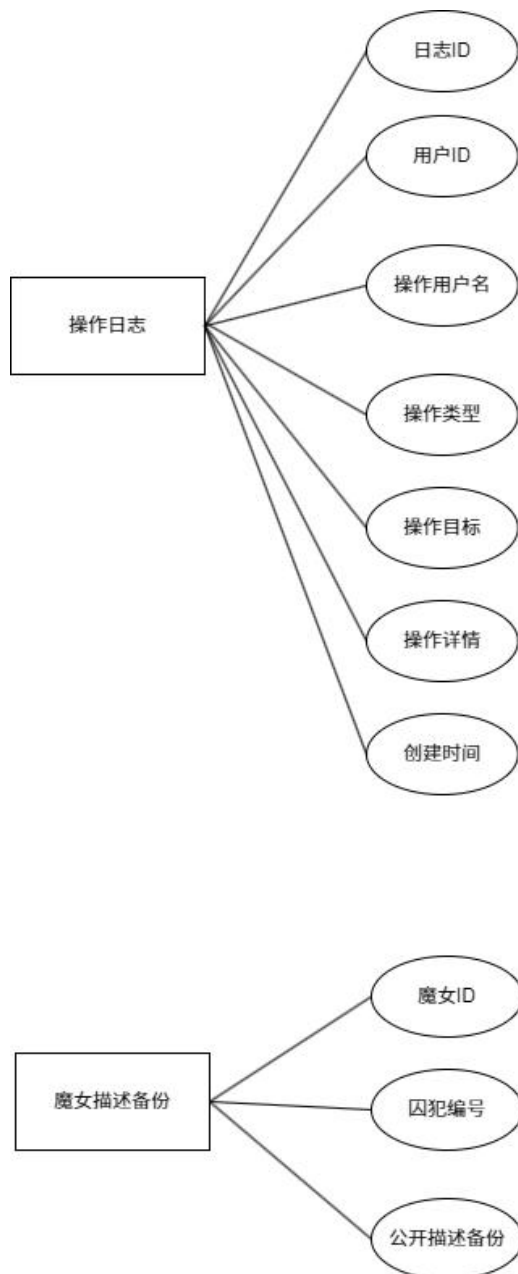


3.3.4 图鉴系统 E-R 图





3.3.5 其它 E-R 图

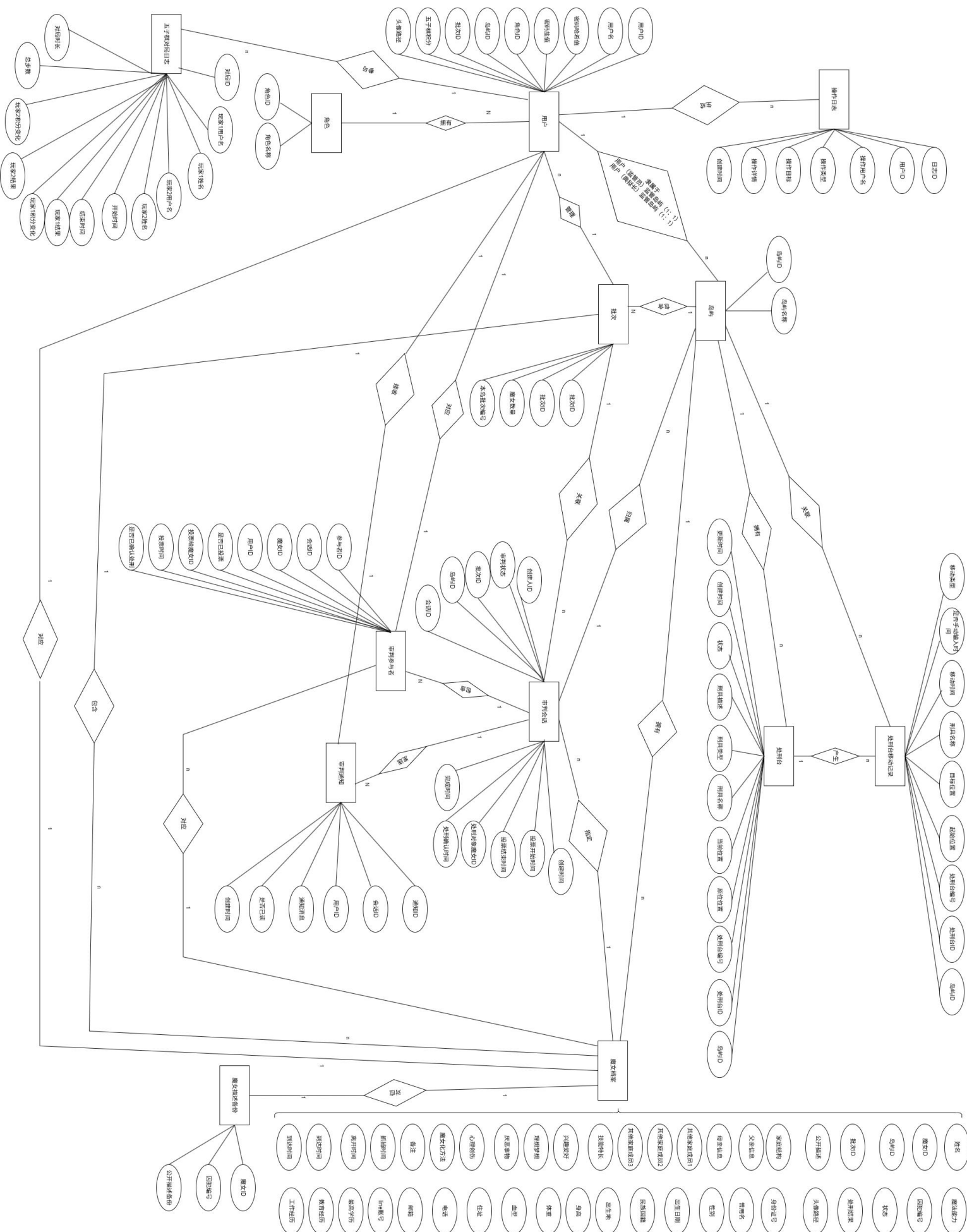


3.4 完整系统 E-R 图

完整系统 E-R 图见下页

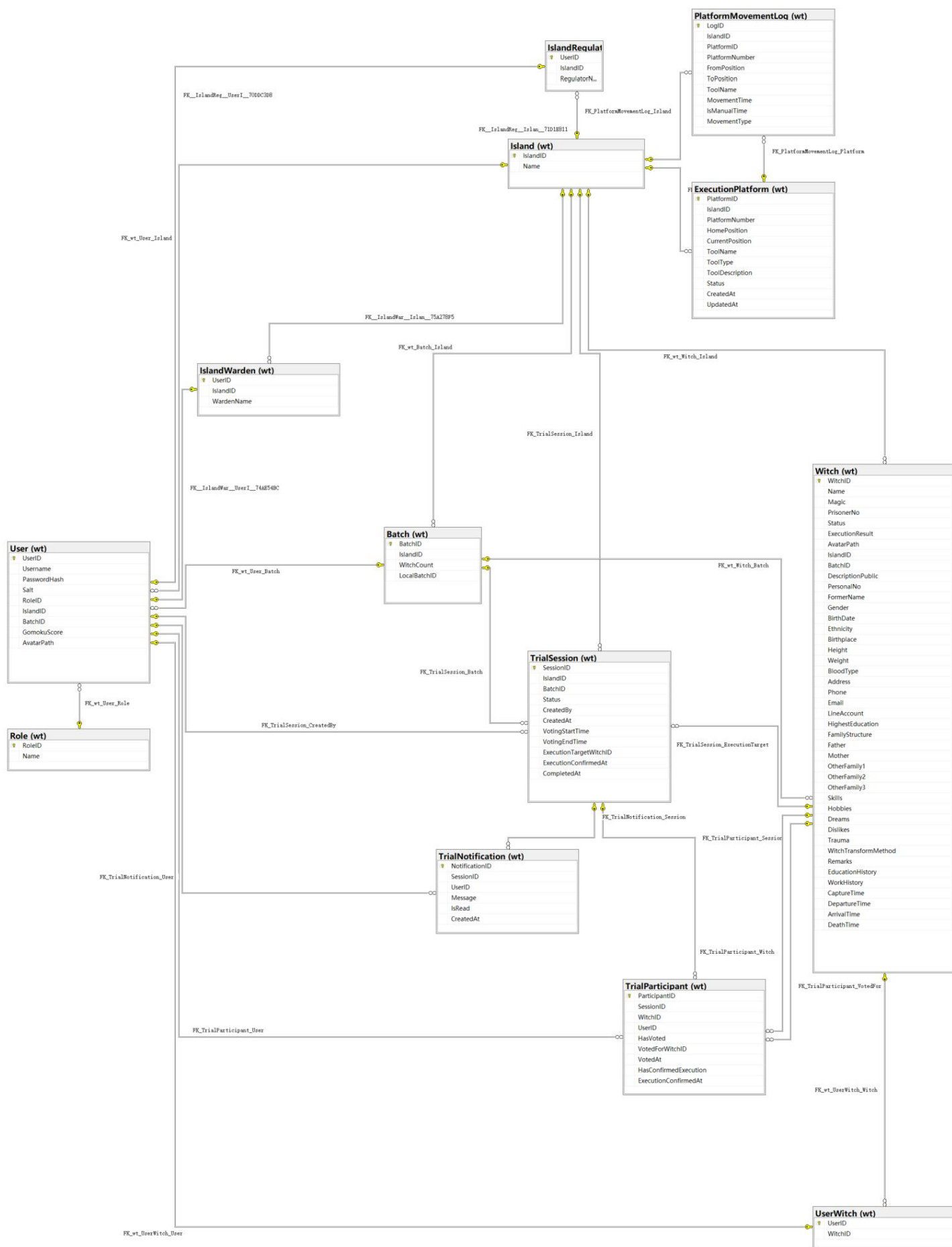


《魔法少女的魔女审判》 多角色权限管理系统





另附上从 SQL Server 2022 直接导出的完整 ER 图 (含外键):





接上图，有五张表独立于主体：

WitchDescBackup (wt)	
WitchID	
PrisonerNo	
DescriptionPublic	

Evidence (wt)	
EvidenceID	
EvidenceNo	
Name	
Description	
ImagePath	

GomokuMatchLog (wt)	
MatchID	
Player1Username	
Player1Name	
Player2Username	
Player2Name	
StartTime	
EndTime	
Player1Result	
Player1ScoreChange	
Player2Result	
Player2ScoreChange	
TotalMoves	
Duration	

OperationLog (wt)	
LogID	
UserID	
Username	
Action	
Target	
Detail	
CreatedAt	

Record (wt)	
RecordID	
Title	
[Content]	



4. 逻辑结构设计

4.1 关系模式设计（18 个）

4.1.1 核心基础表（8 个）

- 角色（角色ID，角色名称）
- 用户（用户ID，用户名，密码哈希值，密码盐值，角色ID，岛屿ID，批次ID，五子棋积分，头像路径）
- 岛屿（岛屿ID，岛屿名称）
- 批次（批次ID，岛屿ID，魔女数量，本岛批次编号）
- 魔女档案（魔女ID，姓名，魔法能力，囚犯编号，状态，处刑结果，头像路径，岛屿ID，批次ID，公开描述，身份证号，曾用名，性别，出生日期，民族国籍，出生地，身高，体重，血型，住址，电话，邮箱，Line账号，最高学历，教育经历，工作经历，家庭结构，父亲信息，母亲信息，其他家庭成员1，其他家庭成员2，其他家庭成员3，技能特长，兴趣爱好，理想梦想，厌恶事物，心理创伤，魔女化方法，备注，抓捕时间，离开时间，到达时间，死亡时间）
- 用户魔女关联（用户ID，魔女ID）
- 岛屿监管员关联（用户ID，岛屿ID，监管员姓名）
- 岛屿典狱长关联（用户ID，岛屿ID，典狱长姓名）

4.1.2 审判系统表（3 个）

- 审判会话（会话ID，岛屿ID，批次ID，审判状态，创建人ID，创建时间，投票开始时间，投票结束时间，处刑对象魔女ID，处刑确认时间，完成时间）
- 审判参与者（参与者ID，会话ID，魔女ID，用户ID，是否已投票，投票给魔女ID，投票时间，是否已确认处刑，确认处刑时间）
- 审判通知（通知ID，会话ID，用户ID，通知消息，是否已读，创建时间）

4.1.3 处刑平台表（2 个）

- 处刑台（处刑台 ID，岛屿 ID，处刑台编号，原位位置，当前位置，刑具名称，刑具类型，刑具描述，状态，创建时间，更新时间）
- 处刑台移动记录（日志 ID，岛屿 ID，处刑台 ID，处刑台编号，起始位置，目标位置，刑具名称，移动时间，是否手动输入时间，移动类型）

4.1.4 图鉴系统表（2 个）

- 证物（证物 ID，证物编号，证物名称，证物描述，证物图片路径）
- 记录（记录 ID，记录标题，Markdown 文件路径）

4.1.5 其他系统表（3 个）

- 五子棋对局日志（对局 ID，玩家 1 用户名，玩家 1 姓名，玩家 2 用户名，玩家 2 姓名，开始时间，结束时间，玩家 1 结果，玩家 1 积分变化，玩家 2 结果，玩家 2 积分变化，总步数，对局时长）
- 操作日志（日志 ID，用户 ID，操作用户名，操作类型，操作目标，操作详情，创建时间）
- 魔女描述备份（魔女 ID，囚犯编号，公开描述备份）



4.2 数据表详细设计（18 张表）

4.2.1 Role 表（角色表）

用途：定义系统中用户角色类型。

字段名	数据类型	可空	说明
RoleID	int	NO	主键，角色 ID
Name	nvarchar(20)	NO	角色名称

角色类型（四层权限体系）：

RoleID	角色名称	角色中文名称	权限	示例账号
1	Admin	国家层管理员	最高权限	admin
2	Meruru	岛屿监管员	监管权限	岛屿 1: meruru_regulator, 岛屿 2: utena_regulator
3	Warden	典狱长	管理权限	岛屿 1: warden 岛屿 2: warden2
4	Witch	普通魔女	个人权限	原游戏中的角色英文简称： 如 ema.hiro,sherry,hanna 以及囚犯编号 671-711

权限层级: Admin(1) > Meruru(2) > Warden(3) > Witch(4)

4.2.2 User 表（用户表）

用途：存储所有用户账号信息。

字段名	数据类型	可空	说明
UserID	int	NO	主键，用户 ID
Username	nvarchar(50)	NO	用户名（唯一）
PasswordHash	nvarchar(64)	NO	密码哈希值
Salt	nvarchar(64)	NO	密码盐值
RoleID	int	NO	外键，关联 Role 表
IslandID	int	YES	外键，关联 Island 表（魔女/典狱长/监管员所属岛屿）
BatchID	int	YES	外键，关联 Batch 表（魔女所属批次）
GomokuScore	int	NO	五子棋积分（默认 0）
AvatarPath	nvarchar(255)	YES	头像路径

备注：后续变更了魔女头像以及姓名缩略图的加载方式，普通魔女不再使用头像路径加载，而是直接在项目文件夹中加载以对应囚犯编号命名的图像。但国家层管理员、岛屿监管员、典狱长仍然使用头像路径。

4.2.3 Island 表（岛屿表）

用途：定义魔女审判岛屿。



字段名	数据类型	可空	说明
IslandID	int	NO	主键，岛屿 ID
Name	nvarchar(50)	NO	岛屿名称

当前岛屿:

- IslandID=1: 魔女岛・壹
- IslandID=2: 魔女岛・贰

4.2.4 Batch 表（批次表）

用途: 定义魔女批次信息现阶段。

字段名	数据类型	可空	说明
BatchID	int	NO	主键，批次 ID（全局唯一）
IslandID	int	NO	外键，关联 Island 表
WitchCount	int	NO	该批次魔女数量（默认 0）
LocalBatchID	int	NO	本岛屿批次编号（岛屿内唯一）

关系:

- 一个批次属于一个岛屿
- 一个批次包含多个魔女

批次编号说明:

BatchID: 全局批次 ID，跨岛屿唯一，自增主键

LocalBatchID: 本岛屿批次编号，每个岛屿内独立编号（1, 2, 3...）

例如:

- 岛屿 1 的第 1 批: BatchID=1, IslandID=1, LocalBatchID=1
- 岛屿 1 的第 2 批: BatchID=2, IslandID=1, LocalBatchID=2
- 岛屿 2 的第 1 批: BatchID=4, IslandID=2, LocalBatchID=1
- 岛屿 2 的第 2 批: BatchID=5, IslandID=2, LocalBatchID=2

约束:

- UQ_Batch_Island_LocalBatch: (IslandID, LocalBatchID) 组合唯一现阶段，国家正大力推动医疗行业智能化升级，出台一系列政策支持基于眼底医学影像的眼科

4.2.5 Witch 表（魔女表）（核心表，43 字段）

用途: 存储魔女的完整档案信息

① 基本信息字段

字段名	数据类型	可空	说明
WitchID	int	NO	主键，魔女 ID
Name	nvarchar(50)	NO	姓名
Magic	nvarchar(100)	YES	魔法能力
PrisonerNo	nvarchar(20)	YES	囚犯编号
Status	nvarchar(20)	NO	状态
ExecutionResult	nvarchar(50)	YES	处刑结果



AvatarPath	nvarchar(255)	YES	头像路径
IslandID	int	YES	外键，关联 Island 表
BatchID	int	YES	外键，关联 Batch 表
DescriptionPublic	nvarchar(MAX)	YES	公开描述

魔女状态列表:

- 待抓捕
- 分配至岛屿
- 审判中
- 死亡(正常)
- 死亡(魔女化)
- 其它

②个人详细信息字段

字段名	数据类型	可空	说明
PersonalNo	nvarchar(20)	YES	身份证号
FormerName	nvarchar(100)	YES	曾用名
Gender	nvarchar(10)	YES	性别
BirthDate	date	YES	出生日期
Ethnicity	nvarchar(50)	YES	民族/国籍
Birthplace	nvarchar(100)	YES	出生地
Height	decimal	YES	身高 (cm)
Weight	decimal	YES	体重 (kg)
BloodType	nvarchar(10)	YES	血型
Address	nvarchar(500)	YES	住址
Phone	nvarchar(50)	YES	电话
Email	nvarchar(100)	YES	邮箱
LineAccount	nvarchar(100)	YES	Line 账号

③教育与工作信息字段

字段名	数据类型	可空	说明
HighestEducation	nvarchar(100)	YES	最高学历
EducationHistory	nvarchar(MAX)	YES	教育经历 (JSON 格式)
WorkHistory	nvarchar(MAX)	YES	工作经历 (JSON 格式)

④家庭信息字段

字段名	数据类型	可空	说明
FamilyStructure	nvarchar(200)	YES	家庭结构
Father	nvarchar(200)	YES	父亲信息
Mother	nvarchar(200)	YES	母亲信息
OtherFamily1	nvarchar(200)	YES	其他家庭成员 1
OtherFamily2	nvarchar(200)	YES	其他家庭成员 2
OtherFamily3	nvarchar(200)	YES	其他家庭成员 3



④个性与特征字段

字段名	数据类型	可空	说明
Skills	nvarchar(500)	YES	技能特长
Hobbies	nvarchar(500)	YES	兴趣爱好
Dreams	nvarchar(500)	YES	理想梦想
Dislikes	nvarchar(500)	YES	厌恶事物
Trauma	nvarchar(MAX)	YES	心理创伤
WitchTransformMethod	nvarchar(500)	YES	魔女化方法

⑤时间记录字段

字段名	数据类型	可空	说明
CaptureTime	datetime2	YES	抓捕时间
DepartureTime	datetime2	YES	离开时间
ArrivalTime	datetime2	YES	到达时间
DeathTime	datetime2	YES	死亡时间

字段统计:

- 基本信息: 10 个字段
- 个人详细信息: 13 个字段
- 教育与工作: 3 个字段
- 家庭信息: 6 个字段
- 个性与特征: 7 个字段
- 时间记录: 4 个字段

总计: 43 个字段

4.2.6 UserWitch 表 (用户-魔女关联表)

用途: 关联用户账号与魔女档案

字段名	数据类型	可空	说明
UserID	int	NO	主键, 外键, 关联 User 表
WitchID	int	YES	外键, 关联 Witch 表

关系:

- 一个魔女用户账号对应一个魔女档案
- 一对一关系

重要说明:

- UserWitch 表中没有 IslandID 字段
- 要获取魔女的 IslandID, 应该从 User 表或 Witch 表中获取

4.2.7 IslandRegulator 表 (岛屿-监管员关联表)

用途: 关联岛屿与监管员



字段名	数据类型	可空	说明
UserID	int	NO	主键，外键，关联 User 表
IslandID	int	NO	外键，关联 Island 表
RegulatorName	nvarchar(50)	NO	监管员姓名

关系:

- 一个监管员管理一个岛屿
- 一个岛屿只能有一个监管员

4.2.8 IslandWarden 表（岛屿-典狱长关联表）

用途: 关联岛屿与典狱长

字段名	数据类型	可空	说明
UserID	int	NO	主键，外键，关联 User 表
IslandID	int	NO	外键，关联 Island 表
WardenName	nvarchar(50)	NO	典狱长姓名

关系:

- 一个典狱长管理一个岛屿
- 一个岛屿只能有一个典狱长

4.2.9 TrialSession 表（审判会话表）

用途: 记录审判会话的完整生命周期现阶段,

字段名	数据类型	可空	说明
SessionID	int	NO	主键，审判会话 ID
IslandID	int	NO	外键，关联 Island 表
BatchID	int	NO	外键，关联 Batch 表
Status	nvarchar(20)	NO	审判状态（默认 Pending）
CreatedBy	int	NO	外键，创建人 UserID
CreatedAt	datetime2	NO	创建时间
VotingStartTime	datetime2	YES	投票开始时间
VotingEndTime	datetime2	YES	投票结束时间
ExecutionTargetWitchID	int	YES	外键，处刑对象 WitchID
ExecutionConfirmedAt	datetime2	YES	处刑确认时间
CompletedAt	datetime2	YES	完成时间

审判状态流转:

1. Pending - 待开始（典狱长已创建，未开始投票）
2. Voting - 投票进行中
3. Confirmed - 已确认处刑对象（典狱长已选定）
4. Executing - 处刑执行中（等待魔女确认）
5. Completed - 已完成
6. Cancelled - 已取消

索引:



- IX_TrialSession_Island (IslandID)
- IX_TrialSession_Status (IslandID, Status)
- IX_TrialSession_CreatedAt (CreatedAt DESC)

4.2.10 TrialParticipant 表（审判参与者表）

用途: 记录每个审判会话的参与者及其投票/确认状态

字段名	数据类型	可空	说明
ParticipantID	int	NO	主键, 参与者 ID
SessionID	int	NO	外键, 关联 TrialSession 表
WitchID	int	NO	外键, 参与的魔女 ID
UserID	int	NO	外键, 参与的用户 ID
HasVoted	bit	NO	是否已投票 (默认 0)
VotedForWitchID	int	YES	外键, 投票给哪个魔女
VotedAt	datetime2	YES	投票时间
HasConfirmedExecution	bit	NO	是否已确认处刑 (默认 0)
ExecutionConfirmedAt	datetime2	YES	确认处刑时间

约束:

- UQ_TrialParticipant_Session_Witch: 每个会话中每个魔女只能参与一次
- ON DELETE CASCADE: 删除会话时自动删除参与者记录

索引:

- IX_TrialParticipant_Session (SessionID)
- IX_TrialParticipant_User (UserID)
- IX_TrialParticipant_Witch (WitchID)
- IX_TrialParticipant_HasVoted (SessionID, HasVoted)
- IX_TrialParticipant_HasConfirmed (SessionID, HasConfirmedExecution)

4.2.11 TrialNotification 表（审判通知表）

用途: 记录发送给参与者的审判通知

字段名	数据类型	可空	说明
NotificationID	int	NO	主键, 通知 ID
SessionID	int	NO	外键, 关联 TrialSession 表
UserID	int	NO	外键, 接收通知的用户 ID
Message	nvarchar(500)	NO	通知消息内容
IsRead	bit	NO	是否已读 (默认 0)
CreatedAt	datetime2	NO	创建时间

约束:

- ON DELETE CASCADE: 删除会话时自动删除通知记录

索引:

- IX_TrialNotification_User (UserID, IsRead)
- IX_TrialNotification_Session (SessionID)
- IX_TrialNotification_CreatedAt (CreatedAt DESC)

4.2.12 ExecutionPlatform 表（处刑台表）



用途: 管理每个岛屿的处刑台位置和刑具信息

字段名	数据类型	可空	说明
PlatformID	int	NO	主键, 处刑台 ID
IslandID	int	NO	外键, 关联 Island 表
PlatformNumber	int	NO	处刑台编号 (1-49)
HomePosition	int	NO	原位位置 (1-49)
CurrentPosition	int	NO	当前位置 (1-50, 50 表示审判庭)
ToolName	nvarchar(100)	YES	刑具名称
ToolType	nvarchar(50)	YES	刑具类型
ToolDescription	nvarchar(500)	YES	刑具描述
Status	nvarchar(20)	NO	状态 (空闲/使用中, 默认空闲)
CreatedAt	datetime2	NO	创建时间
UpdatedAt	datetime2	NO	更新时间

约束:

- UQ_ExecutionPlatform_Island_Number: 每个岛屿的处刑台编号唯一
- CK_ExecutionPlatform_Number: 处刑台编号必须在 1-49 之间
- CK_ExecutionPlatform_HomePosition: 原位位置必须在 1-49 之间
- CK_ExecutionPlatform_CurrentPosition: 当前位置必须在 1-50 之间
- CK_ExecutionPlatform_Status: 状态只能是"空闲"或"使用中"

索引:

- IX_ExecutionPlatform_Island (IslandID)
- IX_ExecutionPlatform_CurrentPosition (IslandID, CurrentPosition)

位置说明:

- 位置 1-49: 处刑台的固定位置
- 位置 50: 审判庭 (处刑台升起后的位置)

4.2.13 PlatformMovementLog 表 (处刑台移动记录表)

用途: 记录处刑台的所有移动历史

字段名	数据类型	可空	说明
LogID	int	NO	主键, 日志 ID
IslandID	int	NO	外键, 关联 Island 表
PlatformID	int	NO	外键, 关联 ExecutionPlatform 表
PlatformNumber	int	NO	处刑台编号 (冗余字段)
FromPosition	int	NO	起始位置 (1-50)
ToPosition	int	NO	目标位置 (1-50)
ToolName	nvarchar(100)	YES	移动时的刑具名称 (冗余字段)
MovementTime	datetime2	NO	移动时间 (北京时间)
IsManualTime	bit	NO	是否手动输入时间 (默认 0)
MovementType	nvarchar(20)	NO	移动类型 (升起/返回)

约束:

- CK_PlatformMovementLog_Position: 位置必须在 1-50 之间
- CK_PlatformMovementLog_Type: 移动类型只能是"升起"或"返回"



索引:

- IX_PlatformMovementLog_Island (IslandID)
- IX_PlatformMovementLog_Platform (PlatformID)
- IX_PlatformMovementLog_Time (MovementTime DESC)

设计说明:

- 不记录操作人信息 (匿名记录) (对应原游戏一周目第四案)
- 支持手动输入精确时间 (用于补录历史记录)
- 冗余字段便于快速查询

4.2.14 Evidence 表 (证物表)

用途: 存储魔女图鉴中的证物信息

字段名	数据类型	可空	说明
EvidenceID	int	NO	主键, 证物 ID (自增)
EvidenceNo	nvarchar(20)	NO	证物编号 (唯一)
Name	nvarchar(100)	NO	证物名称
Description	nvarchar(MAX)	YES	证物描述
ImagePath	nvarchar(255)	YES	证物图片路径

约束:

- UQ_Evidence_No: 证物编号唯一

索引:

- IX_Evidence_No (EvidenceNo)

设计说明:

- 证物编号格式: E001, E002, E003...
- 图片路径相对于 Images/Evidence/文件夹
- 用于魔女图鉴的证物界面展示

4.2.15 Record 表 (记录表)

用途: 存储魔女图鉴中的记录信息

字段名	数据类型	可空	说明
RecordID	int	NO	主键, 记录 ID (自增)
Title	nvarchar(100)	NO	记录标题
Content	nvarchar(255)	YES	Markdown 文件路径

索引:

- IX_Record_Title (Title)

设计说明:

- Content 字段存储 Markdown 文件的相对路径
- Markdown 文件位于 Images/Records/文件夹
- 用于魔女图鉴的记录界面展示
- 记录标题示例: "研究人员的调查书", "魔女的日记"等

4.2.16 GomokuMatchLog 表 (五子棋对局日志表)

用途: 记录五子棋对局信息



字段名	数据类型	可空	说明
MatchID	int	NO	主键，对局 ID
Player1Username	nvarchar(50)	NO	玩家 1 用户名
Player1Name	nvarchar(50)	NO	玩家 1 姓名
Player2Username	nvarchar(50)	NO	玩家 2 用户名
Player2Name	nvarchar(50)	NO	玩家 2 姓名
StartTime	datetime2	NO	开始时间
EndTime	datetime2	NO	结束时间
Player1Result	nvarchar(20)	NO	玩家 1 结果（胜/负/平）
Player1ScoreChange	int	NO	玩家 1 积分变化
Player2Result	nvarchar(20)	NO	玩家 2 结果（胜/负/平）
Player2ScoreChange	int	NO	玩家 2 积分变化
TotalMoves	int	NO	总步数
Duration	int	NO	对局时长（秒）

4.2.17 OperationLog 表（操作日志表）

用途：记录系统操作日志

字段名	数据类型	可空	说明
LogID	int	NO	主键，日志 ID
UserID	int	YES	外键，关联 User 表
Username	nvarchar(50)	NO	操作用户名
Action	nvarchar(50)	NO	操作类型
Target	nvarchar(100)	YES	操作目标
Detail	nvarchar(MAX)	YES	操作详情
CreatedAt	datetime2	NO	创建时间

此表暂时用于国家层管理员删除魔女。

4.2.18 WitchDescBackup 表（魔女描述备份表）

用途：备份魔女的公开描述信息

字段名	数据类型	可空	说明
WitchID	int	NO	主键，外键，关联 Witch 表
PrisonerNo	nvarchar(20)	YES	囚犯编号
DescriptionPublic	nvarchar(MAX)	YES	公开描述备份

备注：此表为废弃表，原因是之前尝试将整个项目改为 SQLITE，不慎导致所有数据库内容作废，后幸运在本地备份和 GitHub 中找回。

4.3 数据完整性约束

4.3.1 实体完整性约束

主键约束

所有 18 个表均设置主键，确保每条记录唯一可识别。主键采用 IDENTITY 自增或直接指定，保



证实体完整性。

4.3.2 参照完整性约束

外键约束（共 25 个）

表名	外键字段	参考表	级联删除
用户	角色 ID	角色	×
用户	岛屿 ID	岛屿	×
用户	批次 ID	批次	×
批次	岛屿 ID	岛屿	×
魔女档案	岛屿 ID	岛屿	×
魔女档案	批次 ID	批次	×
用户魔女关联	用户 ID	用户	✓
用户魔女关联	魔女 ID	魔女档案	×
岛屿监管员关联	用户 ID	用户	✓
岛屿监管员关联	岛屿 ID	岛屿	×
岛屿典狱长关联	用户 ID	用户	✓
岛屿典狱长关联	岛屿 ID	岛屿	×
审判会话	岛屿 ID	岛屿	×
审判会话	批次 ID	批次	×
审判会话	创建人 ID	用户	×
审判会话	处刑对象魔女 ID	魔女档案	×
审判参与者	会话 ID	审判会话	✓
审判参与者	魔女 ID	魔女档案	×
审判参与者	用户 ID	用户	×
审判参与者	投票给魔女 ID	魔女档案	×
审判通知	会话 ID	审判会话	✓
审判通知	用户 ID	用户	×
处刑台	岛屿 ID	岛屿	×
处刑台移动记录	岛屿 ID	岛屿	×
处刑台移动记录	处刑台 ID	处刑台	×

级联删除说明：删除用户时自动删除用户魔女关联、岛屿监管员关联、岛屿典狱长关联；删除审判会话时自动删除审判参与者、审判通知。

4.3.3 用户自定义完整性约束

唯一约束（共 5 个）

表名	约束字段	约束名称	说明
角色	角色名称	-	角色名称全局唯一
用户	用户名	-	用户名全局唯一
审判参与者	(会话 ID, 魔女 ID)	UQ_TrialParticipant_Session_Witch	每个会话中每个魔女只能参



处刑台	(岛屿 ID, 处刑台编号)	UQ_ExecutionPlatform_Island_Number	与一次 每个岛屿的处刑台编号唯一
证物	证物编号	UQ_Evidence_No	证物编号唯一

检查约束 (共 7 个)

表名	约束名称	约束条件	说明
审判会话	CK_TrialSession_Status	Status IN (Pending, Voting, Confirmed, Executing, Completed, Cancelled)	审判状态限制 6 种
处刑台	CK_ExecutionPlatform_Number	PlatformNumber BETWEEN 1 AND 49	处刑台编号范围
处刑台	CK_ExecutionPlatform_HomePosition	HomePosition BETWEEN 1 AND 49	原位位置范围
处刑台	CK_ExecutionPlatform_CurrentPosition	CurrentPosition BETWEEN 1 AND 50	当前位置范围 (50 为审判庭)
处刑台	CK_ExecutionPlatform_Status	Status IN (空闲, 使用中)	处刑台状态限制
处刑台移动记录	CK_PlatformMovementLog_Position	FromPosition BETWEEN 1 AND 50 AND ToPosition BETWEEN 1 AND 50	移动位置范围
处刑台移动记录	CK_PlatformMovementLog_Type	MovementType IN (升起, 返回)	移动类型限制

默认值约束

表名	字段	默认值
用户	五子棋积分	0
审判会话	审判状态	Pending
审判参与者	是否已投票	0
审判参与者	是否已确认处刑	0
审判通知	是否已读	0
处刑台	状态	空闲
处刑台	创建/更新时间	GETDATE()



5.物理结构设计

5.1 存储结构设计

存储结构设计聚焦“数据如何安全存储、高效占用空间”，核心包含数据库文件划分、Schema 逻辑组织、表空间估算与数据类型匹配四大维度，为系统稳定运行奠定基础。

5.1.1 数据库文件结构

采用“主数据文件+日志文件”分离存储模式，明确文件属性与增长策略，避免单一文件损坏导致的数据丢失，同时适配系统数据量增长需求。

表 8 数据库文件结构表

文件类型	逻辑名称	物理文件名	初始大小	增长方式	核心作用
主数据文件 (.mdf)	WitchTrialWT	WitchTrialWT.mdf	50 MB	自动增长 10%	存储所有用户表、系统表及 Schema 对象
事务日志文件 (.ldf)	WitchTrialWT_log	WitchTrialWT_log.ldf	10 MB	自动增长 10%，最大 2GB	记录所有事务操作，支持数据恢复与事务回滚

数据库创建脚本：

```
CREATE DATABASE WitchTrialWT
ON PRIMARY -- 主文件组
(
    NAME = N'WitchTrialWT',           -- 逻辑名称
    FILENAME = N'C:\SQLData\WitchTrialWT.mdf', -- 物理路径（需提前创建 SQLData 文件夹）
    SIZE = 50MB,                       -- 初始大小
    MAXSIZE = UNLIMITED,               -- 无最大限制
    FILEGROWTH = 10%                   -- 增长比例
)
LOG ON -- 日志文件配置
(
    NAME = N'WitchTrialWT_log',
    FILENAME = N'C:\SQLData\WitchTrialWT_log.ldf',
    SIZE = 10MB,
    MAXSIZE = 2GB,                     -- 日志文件限制最大 2GB，避免占用过多磁盘
    FILEGROWTH = 10%
);
GO
```

5.1.2 Schema 逻辑组织

为避免表名冲突、简化权限管理，系统所有业务表统一归属“wt”（WitchTrial 缩写）Schema，与 SQL Server 系统表彻底隔离，形成清晰的逻辑分组。



Schema 创建脚本：

```
-- 创建 wt 架构，归属 dbo 用户  
CREATE SCHEMA wt AUTHORIZATION dbo;  
GO
```

Schema 核心优势：

- 权限聚合：可对“wt.*”批量分配权限（如“授予 Meruru 角色 wt 架构的只读权限”），减少重复配置；
- 命名隔离：避免与系统表（如 sys.User）或第三方表重名，表名统一为“wt.表名”（如 wt.Witch、wt.TrialSession）；
- 维护便捷：通过“筛选架构=wt”可快速定位所有业务表，简化备份与迁移操作。

5.1.3 表空间分配与估算

系统共设计 18 张业务表，按“核心基础-审判业务-处刑平台-辅助功能”分类，结合数据量预测完成表空间分配，为存储资源规划提供依据。

表 9 表空间分配与估算表

模块分类	包含表	当前数据量	年增长量/存储说明
核心基础表（8 张）	wt.Role、wt.User、wt.Island、wt.Batch、wt.Witch 等	约 140 行， ≈70 KB	低增长，wt.Witch（43 字段）为核心大表，占比 70%
审判系统表（3 张）	wt.TrialSession、wt.TrialParticipant、wt.TrialNotification	约 260 行， ≈100 KB	中增长，预计年增 1300 行/50 KB，与审判频次强相关
处刑平台表（2 张）	wt.ExecutionPlatform、wt.PlatformMovementLog	约 100 行， ≈70 KB	高增长，处刑台移动日志预计年增 1000 行/50 KB
辅助功能表（5 张）	wt.Evidence、wt.GomokuMatchLog、wt.OperationLog 等	约 550 行， ≈270 KB	中高增长，操作日志年增 5000 行/200 KB，为主要增长源

存储总量估算（不含图片）：

- 当前总数据量：≈510 KB；
- 年增长量：≈400 KB；
- 3 年预估总量：≈1.7 MB，存储压力极小。

图片资源特殊处理：魔女头像、证物图片等二进制资源（约 17.6 MB）存储于文件系统（Images/目录），数据库仅记录相对路径（如“Images/characters/001.jpg”），避免数据库体积膨胀。

5.1.4 数据类型优化选择

遵循“精准匹配业务需求、最小化存储空间”原则选择数据类型，兼顾查询性能与数据完整性，核心类型选择标准如下表：



表 10 数据优化类型选择表

数据类别	推荐类型	适用场景(系统字段示例)	选择理由
字符串型	NVARCHAR(20-500)、NVARCHAR(MAX)	姓名(Name)、魔法描述(Magic)、JSON配置	支持 Unicode 中文, 变长存储节省空间; MAX 类型适配长文本
数值型	INT、DECIMAL(5,2)、BIT	主键 ID (WitchID)、身高(Height)、投票状态(HasVoted)	INT (4 字节) 满足 ID 需求; DECIMAL 保证精度; BIT 仅占 1 位
日期时间型	DATETIME2、DATE	操作时间(CreatedAt)、出生日期(BirthDate)	DATETIME2 精度(100 纳秒) 高于 DATETIME, 范围覆盖 0001-9999 年

关键避坑点:

- 避免使用 VARCHAR: 系统含大量中文, VARCHAR 可能导致乱码, 统一用 NVARCHAR;
- 拒绝 TEXT/NTEXT: 过时类型, 功能受限, 长文本优先用 NVARCHAR(MAX);
- 不用 BIGINT 存 ID: 系统数据量小, INT (最大 21 亿) 完全满足, 比 BIGINT 节省一半空间。

5.1.5 存储结构设计小结

本小节通过“文件分离存储- Schema 逻辑分组-表空间精准估算-数据类型优化”的层级设计, 实现“小体积、高可靠、易维护”的存储目标, 既适配当前课设数据量小的特点, 又为后续功能扩展预留了存储弹性。

5.2 数据库文件组织

数据库文件组织聚焦“表结构规范、索引高效、操作封装”, 通过约束设计保证数据完整性, 通过索引优化查询性能, 通过存储过程封装复杂操作, 形成“安全-高效-易用”的数据库访问层。

5.2.1 表结构设计规范

所有表遵循统一设计规范, 通过主键、外键、约束等确保数据合法性, 核心规范及示例如下:

1. 主键设计: 自增 INT 为主, 保证唯一与性能



所有表采用“表名 ID”命名的自增 INT 主键, 默认创建聚集索引, 兼顾唯一性与插入性能。

```
-- 示例: 魔女表主键设计
CREATE TABLE wt.Witch (
    WitchID INT PRIMARY KEY IDENTITY(1,1), -- 自增主键, 步长 1
    PrisonerNo NVARCHAR(20) NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    -- 其他 40 个字段...
);
GO
```

2. 外键约束: 强引用关联, 保证数据一致性

所有关联字段创建外键, 明确命名规范 (FK_当前表_引用表/字段), 根据业务需求配置级联操作。

```
-- 示例: 审判会话表外键设计
CREATE TABLE wt.TrialSession (
    SessionID INT PRIMARY KEY IDENTITY(1,1),
    IslandID INT NOT NULL, -- 关联岛屿表
    BatchID INT NOT NULL, -- 关联批次表
    CreatedBy INT NOT NULL, -- 关联用户表 (创建人)

    -- 外键约束定义
    CONSTRAINT FK_TrialSession_Island
        FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID),
    CONSTRAINT FK_TrialSession_Batch
        FOREIGN KEY (BatchID) REFERENCES wt.Batch(BatchID),
    CONSTRAINT FK_TrialSession_CreatedBy
        FOREIGN KEY (CreatedBy) REFERENCES wt.[User](UserID)
);
GO
```

级联操作规则: 审判系统表 (如 TrialParticipant) 用 ON DELETE CASCADE (删除会话时自动删参与者); 核心表 (如 Witch、User) 禁用级联删除, 防止误操作。

3. 完整性约束: CHECK/UNIQUE/DEFAULT, 限制非法数据

- **UNIQUE 约束:** 保证组合字段唯一 (如“岛屿 ID+处刑台编号”);
- **CHECK 约束:** 限制字段值范围 (如处刑台编号 1-49、审判状态枚举值);
- **DEFAULT 约束:** 为常用字段设默认值 (如投票状态默认 0=未投票)。

```
-- 示例: 处刑台表多约束设计
CREATE TABLE wt.ExecutionPlatform (
    PlatformID INT PRIMARY KEY IDENTITY(1,1),
    IslandID INT NOT NULL,
    PlatformNumber INT NOT NULL,
    CurrentPosition INT NOT NULL DEFAULT 1,
    Status NVARCHAR(20) NOT NULL DEFAULT N'正常',

    -- 唯一约束: 同一岛屿处刑台编号唯一
    CONSTRAINT UQ_Platform_Island_Number
        UNIQUE (IslandID, PlatformNumber),
    -- CHECK 约束: 处刑台编号 1-49, 位置 1-50

```



```
CONSTRAINT CK_Platform_Number
    CHECK (PlatformNumber BETWEEN 1 AND 49),
CONSTRAINT CK_Platform_Position
    CHECK (CurrentPosition BETWEEN 1 AND 50),
-- 外键关联岛屿
CONSTRAINT FK_Platform_Island
    FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID)
);
GO
```

5.2.2 索引设计策略

索引是提升查询性能的核心，系统遵循“高频查询优先、避免过度索引”原则，设计“聚集索引+非聚集索引”的组合方案，覆盖所有核心查询场景。

1. 索引类型与作用

表 11 索引类型及示例表

索引类型	创建规则	系统应用示例
聚集索引	主键默认创建，数据按索引物理排序	wt.Witch(WitchID)、wt.TrialSession(SessionID)，加速主键查询
单列非聚集索引	WHERE/JOIN 中高频出现的单个字段	wt.TrialParticipant(UserID)：快速查询用户参与的审判
复合非聚集索引	高频查询的字段组合，选择性高的字段放前面	wt.TrialSession(IslandID, Status)：按岛屿+状态筛选审判会话
降序索引	ORDER BY DESC 的高频字段（如时间）	wt.PlatformMovementLog(MovementTime DESC)：最新移动日志优先展示

2. 核心索引创建脚本

```
-- 示例：处刑台表多约束设计
CREATE TABLE wt.ExecutionPlatform (
    PlatformID INT PRIMARY KEY IDENTITY(1,1),
    IslandID INT NOT NULL,
    PlatformNumber INT NOT NULL,
    CurrentPosition INT NOT NULL DEFAULT 1,
    Status NVARCHAR(20) NOT NULL DEFAULT N'正常',

    -- 唯一约束：同一岛屿处刑台编号唯一
    CONSTRAINT UQ_Platform_Island_Number
        UNIQUE (IslandID, PlatformNumber),
    -- CHECK 约束：处刑台编号 1-49，位置 1-50
    CONSTRAINT CK_Platform_Number
        CHECK (PlatformNumber BETWEEN 1 AND 49),
    CONSTRAINT CK_Platform_Position
        CHECK (CurrentPosition BETWEEN 1 AND 50),
    -- 外键关联岛屿
    CONSTRAINT FK_Platform_Island
        FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID)
);
```



GO

3. 索引设计核心原则

- 高频优先：只为 WHERE、JOIN、ORDER BY 中的高频字段建索引；
- 覆盖查询：复合索引包含查询所需所有字段（如 IX_TrialSession_IslandStatus 包含 CreatedAt），避免“回表”；
- 控制数量：每张表索引≤5 个，索引过多会降低 INSERT/UPDATE 性能；
- 定期维护：碎片率>30%重建索引，10%-30%重组索引。

5.2.3 存储过程设计

用存储过程封装复杂业务操作（如魔女档案全字段更新、批量导入），实现“预编译优化+事务保障+安全隔离”的多重目标。

1. 核心存储过程示例：魔女档案全字段更新

-- 示例：处刑台表多约束设计

```
CREATE TABLE wt.ExecutionPlatform (  
    PlatformID INT PRIMARY KEY IDENTITY(1,1),  
    IslandID INT NOT NULL,  
    PlatformNumber INT NOT NULL,  
    CurrentPosition INT NOT NULL DEFAULT 1,  
    Status NVARCHAR(20) NOT NULL DEFAULT N'正常',  
  
    -- 唯一约束：同一岛屿处刑台编号唯一  
    CONSTRAINT UQ_Platform_Island_Number  
        UNIQUE (IslandID, PlatformNumber),  
    -- CHECK 约束：处刑台编号 1-49，位置 1-50  
    CONSTRAINT CK_Platform_Number  
        CHECK (PlatformNumber BETWEEN 1 AND 49),  
    CONSTRAINT CK_Platform_Position  
        CHECK (CurrentPosition BETWEEN 1 AND 50),  
    -- 外键关联岛屿  
    CONSTRAINT FK_Platform_Island  
        FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID)  
);  
GO
```

2. 存储过程调用示例（C#）

```
/// <summary>  
/// 调用存储过程更新魔女档案  
/// </summary>  
public bool UpdateWitch(WitchModel witch)  
{
```



```
using var conn = DBHelper.GetConn();
using var cmd = new SqlCommand("wt.sp_UpdateWitchComplete", conn);
cmd.CommandType = CommandType.StoredProcedure;

// 添加参数（与存储过程参数一一对应）
cmd.Parameters.AddWithValue("@WitchID", witch.WitchID);
cmd.Parameters.AddWithValue("@Name", witch.Name);
cmd.Parameters.AddWithValue("@Magic", witch.Magic);
// 省略其他参数...

try
{
    conn.Open();
    cmd.ExecuteNonQuery();
    return true;
}
catch (SqlException ex)
{
    MessageBox.Show($"更新失败: {ex.Message}");
    return false;
}
}
```

3. 存储过程核心优势

- 性能优化：SQL Server 预编译执行计划，重复调用无需重新解析；
- 事务安全：封装事务逻辑，避免应用层遗漏回滚操作；
- 安全隔离：隐藏表结构，防止 SQL 注入，仅暴露存储过程接口；
- 代码复用：多端（WinForm、后续 Web 端）可共用同一存储过程。

5.2.4 数据库文件组织小结

本小节通过“规范表结构（约束保障）- 高效索引（查询加速）- 存储过程（操作封装）”的三层设计，构建了“数据可靠、查询快速、开发便捷”的数据库核心层，既符合数据库设计规范，又贴合课设中 WinForm 开发的实际需求。

5.3 性能优化策略

性能优化通过索引优化、存储过程、事务管理、连接管理四个维度实现。

5.3.1 数据库连接管理优化

连接管理的核心是“复用连接、减少开销”，通过连接池与 using 语句实现连接的高效管理。



1. 连接字符串配置 (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=.;Database=WitchTrialWT;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

关键参数说明：Max Pool Size=50（连接池最大连接数，适配 50+并发用户）。

2. DBHelper连接管理

```
public static class DBHelper
{
    private static string? _connStr;

    private static string GetConnectionString()
    {
        if (_connStr != null) return _connStr; // 缓存连接字符串

        string json = File.ReadAllText("appsettings.json");
        using var doc = JsonDocument.Parse(json);
        _connStr = doc.RootElement.GetProperty("ConnectionStrings")
            .GetProperty("DefaultConnection").GetString();

        return _connStr;
    }

    public static SqlConnection GetConn()
    {
        var conn = new SqlConnection(GetConnectionString());
        conn.Open(); // 从 ADO.NET 默认连接池获取
        return conn;
    }

    public static DataTable ExecDataTable(string sql, params SqlParameter[] ps)
    {
        using var conn = GetConn();
        using var cmd = new SqlCommand(sql, conn);
        if (ps is { Length: > 0 }) cmd.Parameters.AddRange(ps);
        using var da = new SqlDataAdapter(cmd);
        var dt = new DataTable();
        da.Fill(dt);
        return dt;
        // using 结束：连接释放回连接池（非销毁）
    }
}
```




3. 连接池优势

避免频繁创建/销毁连接（耗时操作），复用已建立的连接，连接池默认参数：最小连接数 0、最大 100、超时 15 秒，完全满足课设并发需求。

- SqlConnection 默认启用连接池（最小 0、最大 100、超时 15 秒）
- `using` 语句自动释放连接回池，避免频繁创建/销毁
- 全项目 18 个 DAL 类统一通过 DBHelper 复用连接池

5.3.2 索引优化策略

1. 索引统计

总计：21 个索引，覆盖 8 个核心表

表 12 索引统计表

分类	表名	索引数	典型索引
审判系统	TrialSession	3	IX_TrialSession_Status (IslandID, Status)
审判系统	TrialParticipant	5	IX_TrialParticipant_HasVoted (SessionID, HasVoted)
审判系统	TrialNotification	3	IX_TrialNotification_User (UserID, IsRead)
处刑平台	ExecutionPlatform	2	IX_ExecutionPlatform_CurrentPosition (IslandID, CurrentPosition)
处刑平台	PlatformMovementLog	3	IX_PlatformMovementLog_Time (MovementTime DESC)
图鉴系统	Evidence	1	IX_Evidence_No (EvidenceNo)
图鉴系统	Record	1	IX_Record_Title (Title)
游戏系统	GomokuMatchLog	3	IX_GomokuMatchLog_StartTime (StartTime DESC)

2. 复合索引优化

案例 1：审判会话状态查询

```
CREATE INDEX IX_TrialSession_Status ON wt.TrialSession(IslandID, Status);
```

查询"岛屿 1 的 Voting 状态会话"时直接通过复合索引定位，避免全表扫描。

案例 2：用户未读通知查询

```
CREATE INDEX IX_TrialNotification_User ON wt.TrialNotification(UserID, IsRead);
```

查询"用户未读通知"时直接通过复合索引过滤，支持通知中心实时刷新。



5.3.3 存储过程优化

1. 核心存储过程

sp_UpdateWitchComplete: 魔女档案完整更新 (42 个参数)

```
CREATE PROCEDURE wt.sp_UpdateWitchComplete
    @WitchID INT,
    @Name NVARCHAR(100),
    @Magic NVARCHAR(500),
    -- ... 共 42 个参数 ...
AS
BEGIN
    BEGIN TRANSACTION;
    UPDATE wt.Witch SET Name = @Name, Magic = @Magic, ... WHERE WitchID = @WitchID;
    COMMIT TRANSACTION;
END;
```

优化效果: 单次调用更新 43 个字段, 避免多次网络往返, 事务保证原子性。

sp_AddWitchComplete: 魔女档案完整新增

```
CREATE PROCEDURE wt.sp_UpdateWitchComplete
    @WitchID INT,
    @Name NVARCHAR(100),
    @Magic NVARCHAR(500),
    -- ... 共 42 个参数 ...
AS
BEGIN
    BEGIN TRANSACTION;
    UPDATE wt.Witch SET Name = @Name, Magic = @Magic, ... WHERE WitchID = @WitchID;
    COMMIT TRANSACTION;
END;
```

优化效果: 封装插入逻辑, 事务保证原子性。

2. 调用示例

```
CREATE PROCEDURE wt.sp_UpdateWitchComplete
    @WitchID INT,
    @Name NVARCHAR(100),
    @Magic NVARCHAR(500),
    -- ... 共 42 个参数 ...
AS
BEGIN
    BEGIN TRANSACTION;
    UPDATE wt.Witch SET Name = @Name, Magic = @Magic, ... WHERE WitchID = @WitchID;
```



```
COMMIT TRANSACTION;
```

```
END;
```

5.3.4 事务管理优化

1. 用户注册事务 (UserDAL.cs)

```
public void RegisterWitchUser(...)
{
    using var conn = new SqlConnection(DBHelper.GetConn().ConnectionString);
    conn.Open();
    using var transaction = conn.BeginTransaction();

    try
    {
        // 1. 插入用户账号
        // 2. 获取新用户 ID
        // 3. 插入用户-魔女关联
        transaction.Commit();
    }
    catch
    {
        transaction.Rollback();
        throw;
    }
}
```

优化效果：保证用户账号与魔女关联的原子性，失败自动回滚。

2. 批次人数控制事务 (sp_AddWitch)

```
CREATE PROCEDURE wt.sp_AddWitch
AS
BEGIN
    BEGIN TRAN;
    DECLARE @cnt INT;
    SELECT @cnt = WitchCount FROM wt.Batch WITH(UPDLOCK, HOLDLOCK) WHERE BatchID = @BatchID;
    IF @cnt >= 13 THROW 50010, N'批次人数≥13', 1;
    INSERT wt.Witch (...) VALUES (...);
    COMMIT;
END;
```

优化效果：`UPDLOCK, HOLDLOCK` 锁定批次记录，防止并发插入导致人数超限。



5.3.5 视图优化

1. 权限视图设计

v_WitchPublic: 魔女公开信息视图（9 个字段）

```
CREATE VIEW wt.v_WitchPublic AS
SELECT WitchID, Name, Magic, PrisonerNo, Status,
       IslandID, BatchID, AvatarPath, DescriptionPublic
FROM wt.Witch;
```

v_WitchFullProfile: 魔女完整档案视图（43 个字段）

```
CREATE VIEW wt.v_WitchFullProfile AS
SELECT * FROM wt.Witch;
```

优化效果：公开视图隐藏 34 个敏感字段（身份证、电话、家庭信息等），普通魔女用户只能查询公开信息。

2. 权限控制应用

```
public DataTable GetWitchByPermission(string username)
{
    var user = GetUserPermission(username);

    if (user.RoleID == 4) // 普通魔女
        return DBHelper.ExecDataTable("SELECT * FROM wt.v_WitchPublic WHERE IslandID = @id", ...);
    else // Admin/Meruru/Warden
        return DBHelper.ExecDataTable("SELECT * FROM wt.v_WitchFullProfile WHERE IslandID = @id", ...);
}
```

本章从连接管理、索引优化、存储过程、事务管理、视图优化五个维度实现性能优化。通过 ADO.NET 默认连接池、21 个索引、2 个存储过程、事务管理、权限视图，确保系统高效稳定运行。



6.数据库实现

6.1 数据库创建与初始化

6.1.1 数据库创建脚本

```
-- 创建数据库
CREATE DATABASE WitchTrialWT
ON PRIMARY
(
    NAME = N'WitchTrialWT_Data',
    FILENAME = N'C:\SQLData\WitchTrialWT_Data.mdf',
    SIZE = 100MB,
    MAXSIZE = UNLIMITED,
    FILEGROWTH = 10MB
)
LOG ON
(
    NAME = N'WitchTrialWT_Log',
    FILENAME = N'C:\SQLData\WitchTrialWT_Log.ldf',
    SIZE = 50MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 10MB
);
GO

USE WitchTrialWT;
GO

-- 设置中文排序规则
ALTER DATABASE WitchTrialWT COLLATE Chinese_PRC_CI_AS;
GO
```

6.1.2 Schema 创建

```
USE WitchTrialWT;
GO

-- 创建 wt Schema (统一管理业务表)
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = N'wt')
BEGIN
    EXEC('CREATE SCHEMA wt');
    PRINT N'✓ Schema wt 创建成功';
END
```



```
ELSE
BEGIN
    PRINT N'△ Schema wt 已存在';
END
GO
```

6.1.3 数据表创建脚本

核心基础表

1.Role 表（角色表）

```
CREATE TABLE wt.Role (
    RoleID INT PRIMARY KEY,
    Name NVARCHAR(20) NOT NULL UNIQUE
);

-- 初始化权限角色
INSERT INTO wt.Role (RoleID, Name) VALUES
(1, N'Admin'), -- 国家层管理员
(2, N'Meruru'), -- 岛屿监管员
(3, N'Warden'), -- 典狱长
(4, N'Witch'); -- 普通魔女
```

说明：定义系统四层权限角色，控制数据访问范围。

2.Island 表（岛屿表）

```
CREATE TABLE wt.Island (
    IslandID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(50) NOT NULL UNIQUE
);

-- 初始化岛屿
INSERT INTO wt.Island (Name) VALUES
(N'魔女岛·壹'),
(N'魔女岛·贰');
```

说明：存储岛屿基础信息，作为核心数据的地理维度。

3.Batch 表（批次表）

```
CREATE TABLE wt.Batch (
    BatchID INT PRIMARY KEY IDENTITY(1,1),
    IslandID INT NOT NULL,
    WitchCount INT NOT NULL DEFAULT 0, -- 批次内魔女数量
    LocalBatchID INT NOT NULL, -- 岛屿内独立批次编号

    CONSTRAINT FK_Batch_Island FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID),
    CONSTRAINT UQ_Batch_Island_LocalBatch UNIQUE (IslandID, LocalBatchID)
```




```
);
```

```
CREATE INDEX IX_Batch_Island ON wt.Batch(IslandID);
```

设计亮点：通过LocalBatchID实现岛屿内批次独立编号，组合唯一约束确保编号唯一性。

4. User 表（用户表）

```
CREATE TABLE wt.[User] (  
    UserID INT PRIMARY KEY IDENTITY(1,1),  
    Username NVARCHAR(50) NOT NULL UNIQUE,  
    PasswordHash NVARCHAR(64) NOT NULL, -- 密码哈希  
    Salt NVARCHAR(64) NOT NULL, -- 盐值  
    RoleID INT NOT NULL,  
    IslandID INT NULL,  
    BatchID INT NULL,  
  
    CONSTRAINT FK_User_Role FOREIGN KEY (RoleID) REFERENCES wt.Role(RoleID),  
    CONSTRAINT FK_User_Island FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID),  
    CONSTRAINT FK_User_Batch FOREIGN KEY (BatchID) REFERENCES wt.Batch(BatchID)  
);
```

```
CREATE INDEX IX_User_Username ON wt.[User](Username);
```

安全设计：密码采用 SHA256 哈希 + 盐值存储，防止明文泄露和彩虹表攻击。

5. Witch 表（魔女档案表）★ 核心表

```
CREATE TABLE wt.Witch (  
    -- 基本信息  
    WitchID INT PRIMARY KEY IDENTITY(1,1),  
    Name NVARCHAR(50) NOT NULL,  
    Magic NVARCHAR(100) NULL, -- 魔法能力  
    PrisonerNo NVARCHAR(20) NULL UNIQUE, -- 囚犯编号  
    Status NVARCHAR(20) NOT NULL DEFAULT N'待抓捕', -- 状态  
    IslandID INT NULL,  
    BatchID INT NULL,  
    -- 个人详细信息（示例字段）  
    Gender NVARCHAR(10) NULL,  
    BirthDate DATE NULL,  
    Height DECIMAL(5,2) NULL,  
    -- 时间记录  
    CaptureTime DATETIME2 NULL, -- 抓捕时间  
    ArrivalTime DATETIME2 NULL, -- 到达岛屿时间  
  
    CONSTRAINT FK_Witch_Island FOREIGN KEY (IslandID) REFERENCES wt.Island(IslandID),  
    CONSTRAINT FK_Witch_Batch FOREIGN KEY (BatchID) REFERENCES wt.Batch(BatchID),  
    CONSTRAINT CK_Witch_Status CHECK (Status IN (  
        N'待抓捕', N'分配至岛屿', N'审判中', N'死亡(正常)', N'死亡(魔女化)', N'其它'  
    ))
```



```
);
```

```
CREATE INDEX IX_Witch_PrisonerNo ON wt.Witch(PrisonerNo);
```

```
CREATE INDEX IX_Witch_Status ON wt.Witch(Status);
```

设计亮点：涵盖魔女全生命周期信息，通过 CHECK 约束限制状态取值，确保数据一致性。

审判系统核心表

TrialSession 表（审判会话表）

```
CREATE TABLE wt.TrialSession (  
    SessionID INT IDENTITY(1,1) PRIMARY KEY,  
    IslandID INT NOT NULL,  
    BatchID INT NOT NULL,  
    Status NVARCHAR(20) NOT NULL DEFAULT N'Pending', -- 会话状态  
    CreatedBy INT NOT NULL, -- 创建人  
    CreatedAt DATETIME2 NOT NULL DEFAULT GETDATE(),  
    VotingStartTime DATETIME2 NULL, -- 投票开始时间  
    ExecutionTargetWitchID INT NULL, -- 处刑目标魔女  
  
    CONSTRAINT FK_TrialSession_Island FOREIGN KEY (IslandID) REFERENCES  
wt.Island(IslandID),  
    CONSTRAINT FK_TrialSession_Batch FOREIGN KEY (BatchID) REFERENCES  
wt.Batch(BatchID),  
    CONSTRAINT CK_TrialSession_Status CHECK (Status IN (  
        N'Pending', N'Voting', N'Confirmed', N'Executing', N'Completed', N'Cancelled'  
    ))  
);  
  
CREATE INDEX IX_TrialSession_Island_Status ON wt.TrialSession(IslandID, Status);
```

状态流转：覆盖审判全流程（Pending→Voting→Confirmed→Executing→Completed），支持流程化管理。

处刑平台核心表

ExecutionPlatform 表（处刑台表）

```
CREATE TABLE wt.ExecutionPlatform (  
    PlatformID INT PRIMARY KEY IDENTITY(1,1),  
    IslandID INT NOT NULL,  
    PlatformNumber INT NOT NULL, -- 处刑台编号（1-49）  
    HomePosition INT NOT NULL, -- 初始位置  
    CurrentPosition INT NOT NULL, -- 当前位置（1-50，50为审判庭）  
    Status NVARCHAR(20) NOT NULL DEFAULT N'空闲', -- 状态  
  
    CONSTRAINT FK_ExecutionPlatform_Island FOREIGN KEY (IslandID) REFERENCES  
wt.Island(IslandID),  
    CONSTRAINT UQ_ExecutionPlatform_Island_Number UNIQUE (IslandID, PlatformNumber)
```



```
);
```

位置设计：通过CurrentPosition区分处刑台状态（原位 / 审判庭），支持处刑流程管理。

6.2 存储过程设计

sp_UpdateWitchComplete（更新魔女完整档案）

```
CREATE PROCEDURE wt.sp_UpdateWitchComplete
    @WitchID INT,
    @Name NVARCHAR(100),
    @Magic NVARCHAR(500),
    @Status NVARCHAR(50),
    -- 其他核心字段参数...
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 更新魔女信息
        UPDATE wt.Witch
        SET Name = @Name, Magic = @Magic, Status = @Status
        -- 其他字段更新...
        WHERE WitchID = @WitchID;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
```

6.3 视图设计

6.3.1 v_WitchFullProfile（魔女完整档案视图）

```
CREATE VIEW wt.v_WitchFullProfile
AS
SELECT
    w.WitchID,
    w.Name,
    w.Magic,
    w.PrisonerNo,
    w.Status,
```



```
w.ExecutionResult,
w.AvatarPath,
w.IslandID,
i.Name AS IslandName,
w.BatchID,
b.LocalBatchID,
w.DescriptionPublic,
-- 个人详细信息
w.PersonalNo,
w.FormerName,
w.Gender,
w.BirthDate,
DATEDIFF(YEAR, w.BirthDate, GETDATE()) AS Age,
w.Ethnicity,
w.Birthplace,
w.Height,
w.Weight,
w.BloodType,
w.Address,
w.Phone,
w.Email,
w.LineAccount,
-- 教育与工作
w.HighestEducation,
w.EducationHistory,
w.WorkHistory,
-- 家庭信息
w.FamilyStructure,
w.Father,
w.Mother,
w.OtherFamily1,
w.OtherFamily2,
w.OtherFamily3,
-- 个性与特征
w.Skills,
w.Hobbies,
w.Dreams,
w.Dislikes,
w.Trauma,
w.WitchTransformMethod,
w.Remarks,
-- 时间记录
w.CaptureTime,
w.DepartureTime,
w.ArrivalTime,
w.DeathTime,
-- 计算字段
DATEDIFF(DAY, w.ArrivalTime, GETDATE()) AS DaysOnIsland
FROM wt.Witch w
LEFT JOIN wt.Island i ON i.IslandID = w.IslandID
LEFT JOIN wt.Batch b ON b.BatchID = w.BatchID;
```

设计亮点:

1. 关联查询: 自动关联Island和Batch表, 获取岛屿名称和本地批次编号
2. 计算字段:
 - 'Age': 根据出生日期自动计算年龄
 - 'DaysOnIsland': 计算在岛屿上的天数
3. 简化查询: 应用层只需`SELECT \* FROM wt.v\_WitchFullProfile WHERE WitchID = @id`



6.3.2 v_WitchPublic（魔女公开信息视图）

```
CREATE VIEW wt.v_WitchFullProfile
AS
SELECT
    w.WitchID, w.Name, w.Magic, w.PrisonerNo,
    i.Name AS IslandName, b.LocalBatchID AS BatchNumber,
    DATEDIFF(YEAR, w.BirthDate, GETDATE()) AS Age -- 计算年龄
FROM wt.Witch w
LEFT JOIN wt.Island i ON i.IslandID = w.IslandID
LEFT JOIN wt.Batch b ON b.BatchID = w.BatchID;
```

作用：关联魔女、岛屿、批次表，简化完整档案查询，自动计算衍生字段（如年龄）。

6.4 触发器设计

```
CREATE TRIGGER wt.trg_Witch_BatchCount
ON wt.Witch
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- 新增/更新魔女时，同步更新批次数量
    IF EXISTS (SELECT * FROM inserted)
    BEGIN
        UPDATE b
        SET WitchCount = (SELECT COUNT(*) FROM wt.Witch WHERE BatchID = b.BatchID)
        FROM wt.Batch b
        WHERE b.BatchID IN (SELECT DISTINCT BatchID FROM inserted);
    END

    -- 删除魔女时，同步减少批次数量
    IF EXISTS (SELECT * FROM deleted)
    BEGIN
        UPDATE b
        SET WitchCount = (SELECT COUNT(*) FROM wt.Witch WHERE BatchID = b.BatchID)
        FROM wt.Batch b
        WHERE b.BatchID IN (SELECT DISTINCT BatchID FROM deleted);
    END
END
```

作用：自动维护 Batch 表的 WitchCount 字段，确保批次内魔女数量实时准确。



7. 系统实现

7.1 三层架构实现

WitchTrialSystem（魔女审判资料管理系统）采用经典的三层架构设计模式，将应用程序分为三个独立的层次：数据访问层（DAL）、业务逻辑层（BLL）和用户界面层（UI）。这种架构设计实现了关注点分离，提高了代码的可维护性、可扩展性和可测试性。

● 架构优势

1. 关注点分离：每一层只负责自己的职责，降低耦合度
2. 易于维护：修改某一层不影响其他层
3. 代码复用：业务逻辑可以在不同 UI 中复用
4. 便于测试：各层可以独立进行单元测试
5. 团队协作：不同开发者可以并行开发不同层次

7.1.1 数据访问层（DAL）设计

A. 设计原则

数据访问层（Data Access Layer，DAL）是系统与数据库交互的唯一入口，负责所有数据库操作。DAL 层遵循以下设计原则：

1. 单一职责：每个 DAL 类只负责一个实体或功能模块的数据访问
2. 参数化查询：所有 SQL 语句使用参数化查询，防止 SQL 注入攻击
3. 统一连接管理：通过`DBHelper`统一管理数据库连接
4. 异常处理：捕获并封装数据库异常，向上层抛出友好的错误信息
5. 无业务逻辑：DAL 层不包含任何业务逻辑，只负责数据的增删改查

B. 核心组件

● DBHelper.cs - 数据库连接帮助类

`DBHelper`是 DAL 层的核心基础设施，提供统一的数据库连接和操作接口。

主要功能：

```
CREATE TRIGGER wt.trg_Witch_BatchCount
ON wt.Witch
namespace WitchTrialSystem.DAL
{
    public static class DBHelper
    {
        // 1. 获取数据库连接字符串（从 appsettings.json 读取）
        private static string GetConnectionString()
```




```
// 2. 获取数据库连接对象
public static SqlConnection GetConn()

// 3. 执行查询, 返回单个值 (标量查询)
public static object? ExecScalar(string sql, params SqlParameter[] ps)

// 4. 执行增删改操作, 返回受影响的行数
public static int ExecNonQuery(string sql, params SqlParameter[] ps)

// 5. 执行查询, 返回 DataTable (数据集查询)
public static DataTable ExecDataTable(string sql, params SqlParameter[] ps)
}
}
```

设计特点:

静态类设计: 无需实例化, 直接调用方法

连接字符串管理: 从`appsettings.json`读取配置, 支持缓存

自动资源管理: 使用`using`语句确保连接自动关闭

异常封装: 将 SQL 异常转换为友好的错误信息

配置示例 (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=WitchTrialWT;Trusted_Connection=True;"
  }
}
```

● 典型 DAL 类示例

UserDAL.cs - 用户数据访问类

```
namespace WitchTrialSystem.DAL
{
    public class UserDAL
    {
        // 根据用户名查询用户信息 (含密码哈希和盐值)
        public (int UserID, string Username, int RoleID, string Salt, string Hash)?
            GetByUsername(string username)
        {
            const string sql = @"
                SELECT TOP 1 UserID, Username, RoleID, Salt, PasswordHash
                FROM wt.[User]
                WHERE Username = @u";
            var dt = DBHelper.ExecDataTable(sql, new SqlParameter("@u", username));
            if (dt.Rows.Count == 0) return null;
            var r = dt.Rows[0];
            return (
                Convert.ToInt32(r["UserID"]),

```



```
Convert.ToString(r["Username"]) ?? "",
Convert.ToInt32(r["RoleID"]),
Convert.ToString(r["Salt"]) ?? "",
Convert.ToString(r["PasswordHash"]) ?? ""
);
}

// 插入新用户
public int Insert(string username, int roleId, string salt, string hash)
{
    const string sql = @"
        INSERT INTO wt. [User] (Username, RoleID, Salt, PasswordHash)
        VALUES (@u, @r, @s, @h)";
    return DBHelper.ExecNonQuery(sql,
        new SqlParameter("@u", username),
        new SqlParameter("@r", roleId),
        new SqlParameter("@s", salt),
        new SqlParameter("@h", hash));
}

// 更新密码
public int UpdatePassword(string username, string newSalt, string newHash)
{
    const string sql = @"
        UPDATE wt. [User]
        SET Salt=@s, PasswordHash=@h
        WHERE Username=@u";
    return DBHelper.ExecNonQuery(sql,
        new SqlParameter("@s", newSalt),
        new SqlParameter("@h", newHash),
        new SqlParameter("@u", username));
}

// 检查用户名是否存在
public bool UserExists(string username)
{
    const string sql = @"
        SELECT COUNT(1)
        FROM wt. [User]
        WHERE Username = @u";
    var result = DBHelper.ExecScalar(sql, new SqlParameter("@u", username));
    return result != null && Convert.ToInt32(result) > 0;
}
}
```



设计要点:

返回值设计: 使用元组或可空类型, 明确表示查询可能无结果

参数化查询: 所有 SQL 参数使用`@参数名`格式, 通过`SqlParameter`传递

命名空间规范: 使用`wt.[表名]`格式, 避免与 SQL Server 系统表冲突

类型转换: 统一使用`Convert`类进行类型转换, 处理 NULL 值

WitchDAL.cs - 魔女数据访问类

```
namespace WitchTrialSystem.DAL
{
    public class WitchDAL
    {
        // 获取所有岛屿
        public DataTable GetIslands()
        => DBHelper.ExecDataTable(
            "SELECT IslandID, Name FROM wt.Island ORDER BY IslandID");

        // 根据岛屿 ID 获取批次列表
        public DataTable GetBatches(int islandId)
        => DBHelper.ExecDataTable(
            "SELECT BatchID, LocalBatchID FROM wt.Batch WHERE IslandID=@i ORDER BY LocalBatchID",
            new SqlParameter("@i", islandId));

        // 条件查询魔女列表 (支持多条件组合)
        public DataTable GetWitches(int? islandId=null, int? batchId=null, string? nameLike=null)
        {
            var sql = @"SELECT
                w.WitchID, w.PrisonerNo, w.Name, w.Magic, w.[Status],
                i.Name AS IslandName, b.LocalBatchID
            FROM wt.Witch w
            LEFT JOIN wt.Island i ON w.IslandID = i.IslandID
            LEFT JOIN wt.Batch b ON w.BatchID = b.BatchID
            WHERE 1=1";

            var ps = new List<SqlParameter>();

            // 动态构建 WHERE 条件
            if (islandId.HasValue)
            {
                sql += " AND w.IslandID=@is";
                ps.Add(new SqlParameter("@is", islandId.Value));
            }

            if (batchId.HasValue)
            {
                sql += " AND w.BatchID=@ba";
                ps.Add(new SqlParameter("@ba", batchId.Value));
            }
        }
    }
}
```



```
if (!string.IsNullOrEmpty(nameLike))
{
    sql += " AND w.Name LIKE @nm";
    ps.Add(new SqlParameter("@nm", "%" + nameLike.Trim() + "%"));
}
sql += " ORDER BY w.PrisonerNo";
return DBHelper.ExecDataTable(sql, ps.ToArray());
}
}
```

设计要点:

动态 SQL 构建: 根据条件动态构建 WHERE 子句, 提高查询灵活性

JOIN 查询: 使用 LEFT JOIN 关联多表, 获取完整信息

可空参数: 使用可空类型支持可选查询条件

LIKE 查询: 使用参数化 LIKE 查询, 支持模糊搜索

C. DAL层文件结构

├── DBHelper.cs	# 数据库连接帮助类 (核心基础设施)
├── UserDAL.cs	# 用户数据访问
├── UserProfileDAL.cs	# 用户档案数据访问
├── WitchDAL.cs	# 魔女数据访问
├── TrialSessionDAL.cs	# 审判会话数据访问
├── TrialParticipantDAL.cs	# 审判参与者数据访问
├── TrialNotificationDAL.cs	# 审判通知数据访问
├── ExecutionPlatformDAL.cs	# 处刑平台数据访问
├── MovementLogDAL.cs	# 移动日志数据访问
├── DashboardDAL.cs	# 大屏数据访问
├── GomokuMatchLogDAL.cs	# 五子棋对局日志数据访问
├── AuditDAL.cs	# 审计日志数据访问
├── EvidenceDAL.cs	# 证物数据访问
├── RecordDAL.cs	# 记录数据访问
├── PermissionDAL.cs	# 权限数据访问
├── Models/	
└── UserProfile.cs	# 数据模型 (部分模型放在 DAL 层)

D. 事务处理

对于需要保证原子性的操作, DAL 层支持事务处理:

```
public int CreateWitchAccountWithAssociation(
    string username, int roleId, int islandId, int batchId,
    string salt, string hash, int witchId)
{
    using var conn = new SqlConnection(DBHelper.GetConn().ConnectionString);
    conn.Open();
    using var transaction = conn.BeginTransaction();
```



```
try
{
    // 1. 创建 User 记录
    const string insertUserSql = @"
        INSERT INTO wt.[User] (Username, RoleId, IslandID, BatchID, Salt, PasswordHash)
        VALUES (@username, @roleId, @islandId, @batchId, @salt, @hash);
        SELECT CAST(SCOPE_IDENTITY() AS INT);";

    int newUserId;
    using (var cmd = new SqlCommand(insertUserSql, conn, transaction))
    {
        // ... 设置参数
        newUserId = Convert.ToInt32(cmd.ExecuteScalar());
    }

    // 2. 创建 UserWitch 关联记录
    const string insertUserWitchSql = @"
        INSERT INTO wt.UserWitch (UserID, WitchID)
        VALUES (@userId, @witchId);";

    using (var cmd = new SqlCommand(insertUserWitchSql, conn, transaction))
    {
        // ... 设置参数并执行
    }

    // 3. 提交事务
    transaction.Commit();
    return newUserId;
}
catch
{
    // 回滚事务
    transaction.Rollback();
    throw;
}
}
```

事务处理要点：

手动事务管理：使用`BeginTransaction()`显式管理事务

异常回滚：捕获异常时回滚事务，保证数据一致性

资源释放：使用`using`语句确保连接和事务正确释放

E. 存储过程调用

DAL 层支持调用数据库存储过程：

```
public void AddWitch(string name, string? magic, string? prisonerNo, int islandId, int batchId)
{
    }
```



```
const string sql = @"EXEC wt.sp_AddWitch
    @Name=@n, @Magic=@m, @PrisonerNo=@p, @IslandID=@i, @BatchID=@b";
DBHelper.ExecNonQuery(sql,
    new SqlParameter("@n", name),
    new SqlParameter("@m", magic ?? (object)DBNull.Value),
    new SqlParameter("@p", prisonerNo ?? (object)DBNull.Value),
    new SqlParameter("@i", islandId),
    new SqlParameter("@b", batchId));
}
```

7.1.2 业务逻辑层（BLL）设计

A. 设计原则

业务逻辑层（Business Logic Layer, BLL）是系统的核心，负责处理业务规则、数据验证、权限控制和业务流程编排。BLL 层遵循以下设计原则：

1. 业务规则封装：将复杂的业务逻辑封装在 BLL 层
2. 数据验证：在 BLL 层进行数据有效性验证
3. 权限控制：根据用户角色和权限控制数据访问范围
4. 异常处理：处理业务异常，返回友好的错误信息
5. 不直接访问数据库：BLL 层通过 DAL 层访问数据库，不直接操作数据库

B. 核心组件

UserBLL.cs - 用户业务逻辑类

```
namespace WitchTrialSystem.BLL
{
    public class UserBLL
    {
        private readonly UserDAL _dal = new();

        /// <summary>
        /// 用户登录验证
        /// </summary>
        /// <param name="username">用户名</param>
        /// <param name="password">密码</param>
        /// <returns>登录成功返回用户信息，失败返回 null</returns>
        public (int UserID, string Username, int RoleID)? Login(string username, string password)
        {
            // 1. 从 DAL 层获取用户信息
            var u = _dal.GetByUsername(username);
            if (u == null) return null;

            // 2. 检查账号是否已初始化
            if (string.Equals(u.Value.Salt, "PENDING", StringComparison.OrdinalIgnoreCase) ||
                string.Equals(u.Value.Hash, "PENDING", StringComparison.OrdinalIgnoreCase))
                return null;
        }
    }
}
```




```
// 3. 验证密码（调用 Security 类进行密码验证）
var ok = Security.Verify(password, u.Value.Salt, u.Value.Hash);
if (!ok) return null;

// 4. 返回用户信息
return (u.Value.UserID, u.Value.Username, u.Value.RoleID);
}

/// <summary>
/// 修改密码
/// </summary>
public bool ChangePassword(string username, string oldPassword, string newPassword)
{
    // 1. 获取用户信息
    var u = _dal.GetByUsername(username);
    if (u == null) return false;

    // 2. 验证账号是否已初始化
    if (string.Equals(u.Value.Salt, "PENDING", StringComparison.OrdinalIgnoreCase) ||
        string.Equals(u.Value.Hash, "PENDING", StringComparison.OrdinalIgnoreCase))
        return false;

    // 3. 校验旧密码
    if (!Security.Verify(oldPassword, u.Value.Salt, u.Value.Hash))
        return false;

    // 4. 生成新密码哈希并更新
    var (salt, hash) = Security.HashPassword(newPassword);
    var rows = _dal.UpdatePassword(username, salt, hash);
    return rows > 0;
}

/// <summary>
/// 为魔女创建账号（包含业务规则验证）
/// </summary>
public (bool Success, string Message) CreateWitchAccount(
    string prisonerNo, int islandId, int batchId, int witchId, int regulatorIslandId)
{
    try
    {
        // 1. 权限验证：魔女必须属于监管员的岛屿
        if (islandId != regulatorIslandId)
        {
            return (false, "您只能为本岛屿的魔女创建账号");
        }
    }
}
```



```
}

// 2. 业务规则验证：检查账号是否已存在
if (_dal.UserExists(prisonerNo))
{
    return (false, $"账号创建失败：用户名已存在（{prisonerNo}）");
}

// 3. 获取 Witch 角色 ID
const string getRoleSql = "SELECT RoleId FROM wt.Role WHERE Name = N'Witch'";
var roleResult = DBHelper.ExecScalar(getRoleSql);
if (roleResult == null)
{
    return (false, "系统错误：无法找到 Witch 角色");
}
int witchRoleId = Convert.ToInt32(roleResult);

// 4. 生成默认密码哈希
const string fixedSalt = "Yipintianxia_MiddleRingRoad_2025";
const string fixedHash =
"0A98E098B42638B461C3C4E820D1D325F896928BB5DB655DA3BDDDD97F1DC976";

// 5. 调用 DAL 层创建账号
int newUserId = _dal.CreateWitchAccountWithAssociation(
    prisonerNo, witchRoleId, islandId, batchId,
    fixedSalt, fixedHash, witchId);

return (true, $"账号创建成功！\n用户名：{prisonerNo}\n默认密码：123456");
}
catch (SqlException ex) when (ex.Number == 2627 || ex.Number == 2601)
{
    // 唯一约束冲突
    return (false, $"账号创建失败：用户名已存在（{prisonerNo}）");
}
catch (Exception ex)
{
    return (false, $"账号创建失败：{ex.Message}");
}
}
}
```

设计要点：

依赖注入模式：在类内部创建 DAL 实例（`private readonly UserDAL \_dal = new()`）

业务规则验证：在 BLL 层进行权限、数据有效性等业务规则验证



异常处理：捕获特定异常（如唯一约束冲突），返回友好的错误信息

-返回值设计：使用元组返回操作结果和消息，便于 UI 层处理

DashboardService.cs - 大屏业务逻辑类

```
namespace WitchTrialSystem.BLL
{
    /// <summary>
    /// 智慧可视化大屏业务逻辑层
    /// 处理数据转换、权限控制和业务逻辑
    /// </summary>
    public class DashboardService
    {
        private readonly DashboardDAL _dal = new();

        /// <summary>
        /// 获取全局统计数据（根据权限过滤）
        /// </summary>
        public GlobalStats GetGlobalStats(string username, string role, int? userIslandId = null)
        {
            var dt = _dal.GetGlobalStatistics();
            if (dt.Rows.Count == 0)
            {
                return new GlobalStats();
            }

            var row = dt.Rows[0];
            return new GlobalStats
            {
                TotalWitches = Convert.ToInt32(row["TotalWitches"]),
                TotalIslands = Convert.ToInt32(row["TotalIslands"]),
                ActiveIslands = Convert.ToInt32(row["TotalIslands"]),
                TotalBatches = Convert.ToInt32(row["TotalBatches"]),
                ActiveBatches = Convert.ToInt32(row["ActiveBatches"])
            };
        }

        /// <summary>
        /// 获取状态分布（根据权限过滤）
        /// </summary>
        public List<StatusCount> GetStatusDistribution(
            string username, string role, int? userIslandId = null)
        {
            DataTable dt;

            // 权限控制：Admin 查看全部，Meruru 查看所属岛屿
            if (role == "Admin")
            {
                dt = _dal.GetStatusCounts();
            }
            else if (role == "Meruru" && userIslandId.HasValue)
            {
                dt = _dal.GetIslandStatusCounts(userIslandId.Value);
            }
            else
            {
                return new List<StatusCount>();
            }
        }
    }
}
```



```
        return ConvertToStatusCounts(dt);
    }

    /// <summary>
    /// 数据转换: DataTable → List<StatusCount>
    /// </summary>
    private List<StatusCount> ConvertToStatusCounts(DataTable dt)
    {
        var result = new List<StatusCount>();
        foreach (DataRow row in dt.Rows)
        {
            result.Add(new StatusCount
            {
                Status = Convert.ToString(row["Status"]) ?? "",
                Count = Convert.ToInt32(row["Count"])
            });
        }
        return result;
    }
}
```

设计要点:

权限控制: 根据用户角色和权限过滤数据

数据转换: 将 DAL 层返回的 DataTable 转换为强类型的 Model 对象

业务逻辑封装: 将复杂的数据处理逻辑封装在 BLL 层

Security.cs - 安全服务类

```
namespace WitchTrialSystem.BLL
{
    /// <summary>
    /// 密码加密和验证服务
    /// </summary>
    public static class Security
    {
        /// <summary>
        /// 生成密码哈希 (SHA256 + Salt)
        /// </summary>
        public static (string Salt, string Hash) HashPassword(string password)
        {
            // 生成随机盐值
            var saltBytes = new byte[32];
            using (var rng = System.Security.Cryptography.RandomNumberGenerator.Create())
            {
                rng.GetBytes(saltBytes);
            }
            string salt = Convert.ToHexString(saltBytes);

            // 计算密码哈希
        }
    }
}
```



```
using var sha256 = System.Security.Cryptography.SHA256.Create();
var passwordBytes = System.Text.Encoding.UTF8.GetBytes(password + salt);
var hashBytes = sha256.ComputeHash(passwordBytes);
string hash = Convert.ToHexString(hashBytes);

return (salt, hash);
}

/// <summary>
/// 验证密码
/// </summary>
public static bool Verify(string password, string salt, string hash)
{
    using var sha256 = System.Security.Cryptography.SHA256.Create();
    var passwordBytes = System.Text.Encoding.UTF8.GetBytes(password + salt);
    var hashBytes = sha256.ComputeHash(passwordBytes);
    string computedHash = Convert.ToHexString(hashBytes);

    return computedHash.Equals(hash, StringComparison.OrdinalIgnoreCase);
}
}
```

设计要点:

静态类设计: 安全服务无需状态, 使用静态类

密码安全: 使用 SHA256 + Salt 加密, 防止彩虹表攻击

可复用性: 密码加密和验证逻辑可在多个地方复用

C. BLL层文件结构

—— UserBLL.cs	# 用户业务逻辑
—— Security.cs	# 密码加密验证服务
—— DashboardService.cs	# 大屏业务逻辑
—— ExecutionPlatformService.cs	# 处刑平台业务逻辑
—— MovementLogService.cs	# 移动日志业务逻辑
—— TrialSessionService.cs	# 审判会话业务逻辑
—— TrialVotingService.cs	# 审判投票业务逻辑
—— TrialNotificationService.cs	# 审判通知业务逻辑
—— RecordingService.cs	# 录音业务逻辑
—— AIService.cs	# AI 业务逻辑
—— IconHelper.cs	# 图标处理辅助类
...	

D. 业务逻辑层职责总结

1. 数据验证: 验证输入数据的有效性和完整性
2. 权限控制: 根据用户角色控制数据访问范围
3. 业务规则: 实现复杂的业务规则和流程控制



4. 数据转换：将 DAL 层返回的数据转换为 UI 层需要的格式
5. 异常处理：捕获和处理业务异常，返回友好的错误信息
6. 事务协调：协调多个 DAL 操作，保证业务事务的完整性

7.1.3 用户界面层（UI）设计

A. 设计原则

用户界面层（User Interface Layer，UI）负责与用户交互，展示数据和接收用户输入。UI 层遵循以下设计原则：

1. 职责单一：UI 层只负责界面展示和用户交互，不包含业务逻辑
2. 调用 BLL 层：所有业务操作通过 BLL 层完成，不直接调用 DAL 层
3. 用户体验优先：注重界面美观、操作便捷、反馈及时
4. 异常处理：捕获异常并友好提示用户
5. 数据绑定：使用数据绑定简化界面更新

B. 核心组件

LoginForm.cs - 登录界面

```
namespace WitchTrialSystem.UI
{
    /// <summary>
    /// 登录界面
    /// 功能：用户登录、密码验证、角色路由
    /// </summary>
    public class LoginForm : Form
    {
        #region 控件定义

        // 背景容器
        private readonly Panel _bg = new()
        {
            Dock = DockStyle.Fill,
            BackgroundImageLayout = ImageLayout.Stretch
        };

        // 输入框
        private readonly TextBox _txtUser = new()
        {
            Width = 303,
            Height = 33,
            Font = new Font("Segoe UI", 14),
            BorderStyle = BorderStyle.None,
            BackColor = Color.FromArgb(27, 16, 13),
            ForeColor = Color.White,
            ImeMode = ImeMode.Disable // 禁用输入法
        };

        private readonly TextBox _txtPass = new()
        {
            Width = 303,
            Height = 33,
            Font = new Font("Segoe UI", 14),
            BorderStyle = BorderStyle.None,
            BackColor = Color.FromArgb(27, 16, 13),
            ForeColor = Color.White,
            UseSystemPasswordChar = true,
        }
    }
}
```




```
ImeMode = ImeMode.Disable
};

// 热键按钮区域 (透明 Panel)
private readonly Panel _btnLogin = new()
{
    Size = new Size(175, 88),
    BackColor = Color.Transparent,
    Cursor = Cursors.Hand
};

#endregion

#region 构造函数和初始化

public LoginForm()
{
    Text = "魔女审判 · 登录";
    Width = 1536;
    Height = 864;
    StartPosition = FormStartPosition.CenterScreen;
    FormBorderStyle = FormBorderStyle.FixedSingle;
    MaximizeBox = false;
    DoubleBuffered = true;
    KeyPreview = true;

    // 设置应用程序图标
    BLL.IconHelper.SetFormIcon(this);

    // 加载背景图
    LoadBackground();
    Controls.Add(_bg);

    // 添加控件
    SetupControls();

    // 绑定事件
    BindEvents();
}

#endregion

#region 事件处理

/// <summary>
/// 登录按钮点击事件
/// </summary>
private void OnLoginClick(object? sender, EventArgs e)
{
    var username = _txtUser.Text.Trim();
    var password = _txtPass.Text;

    // 1. 输入验证
    if (string.IsNullOrEmpty(username))
    {
        ShowMessage("请输入用户名");
        return;
    }
}
```



```
if (string.IsNullOrEmpty(password))
{
    ShowMessage("请输入密码");
    return;
}

try
{
    // 2. 调用 BLL 层进行登录验证
    var bll = new BLL.UserBLL();
    var result = bll.Login(username, password);

    if (result == null)
    {
        ShowMessage("用户名或密码错误");
        return;
    }

    // 3. 根据角色跳转到对应界面
    var (userId, userName, roleId) = result.Value;
    NavigateToRoleForm(userId, userName, roleId);
}
catch (Exception ex)
{
    ShowMessage($"登录失败: {ex.Message}");
}
}

/// <summary>
/// 根据角色跳转到对应界面
/// </summary>
private void NavigateToRoleForm(int userId, string username, int roleId)
{
    Form? nextForm = roleId switch
    {
        1 => new Form1_Admin(), // Admin
        2 => new Form1_Regulator(), // Meruru
        3 => new Form1_Warden(), // Warden
        4 => new UI.PhoneForm(userId), // Witch
        _ => null
    };

    if (nextForm != null)
    {
        nextForm.Show();
        Hide();
    }
    else
    {
        ShowMessage("未知的角色类型");
    }
}

#endregion
}
```



设计要点:

界面布局: 使用 Panel 作为容器, 支持背景图片和透明按钮

输入验证: 在 UI 层进行基本的输入验证 (非空、格式等)

调用 BLL 层: 通过 `new BLL.UserBLL()` 创建 BLL 实例并调用业务方法

异常处理: 使用 try-catch 捕获异常, 友好提示用户

角色路由: 根据用户角色跳转到不同的主界面

PhoneForm.cs - 魔女手机主界面

```
namespace WitchTrialSystem.UI
{
    /// <summary>
    /// 魔女手机主界面
    /// 功能: 模拟手机操作, 提供图鉴、五子棋、审判等功能入口
    /// </summary>
    public class PhoneForm : Form
    {
        private readonly int _userId;
        private readonly BLL.TrialNotificationService _notificationService;

        public PhoneForm(int userId)
        {
            _userId = userId;
            _notificationService = new BLL.TrialNotificationService();

            InitializeComponent();
            LoadUserData();
            CheckNotifications();
        }

        /// <summary>
        /// 加载用户数据
        /// </summary>
        private void LoadUserData()
        {
            try
            {
                // 调用 BLL 层获取用户信息
                var userDAL = new DAL.UserDAL();
                // ... 加载用户相关数据
            }
            catch (Exception ex)
            {
                MessageBox.Show($"加载数据失败: {ex.Message}", "错误",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        /// <summary>
        /// 检查审判通知
        /// </summary>
        private void CheckNotifications()
        {
            try
            {
                // 调用 BLL 层检查通知
            }
        }
    }
}
```



```
var notifications = _notificationService.GetPendingNotifications(_userId);
if (notifications.Count > 0)
{
    // 显示通知弹窗
    var popup = new NotificationPopupForm(notifications);
    popup.ShowDialog();
}
}
catch (Exception ex)
{
    // 静默处理, 不影响主界面显示
    System.Diagnostics.Debug.WriteLine($"检查通知失败: {ex.Message}");
}
}

/// <summary>
/// 打开图鉴界面
/// </summary>
private void OpenPokedex(object? sender, EventArgs e)
{
    try
    {
        var pokedexForm = new PokedexForm(_userId);
        pokedexForm.Show();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"打开图鉴失败: {ex.Message}", "错误",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
}
```

设计要点:

依赖注入: 在构造函数中接收 `userId`, 创建 BLL 服务实例

异步加载: 在 Load 事件中加载数据, 避免阻塞界面

异常处理: 每个操作都进行异常处理, 保证界面稳定性

用户体验: 提供友好的错误提示和加载反馈

Form1_Admin.cs - 管理员主界面

```
namespace WitchTrialSystem
{
    /// <summary>
    /// 管理员主界面
    /// 功能: 魔女档案管理、用户管理、数据统计等
    /// </summary>
    public partial class Form1_Admin : Form
    {
        private readonly BLL.DashboardService _dashboardService;
        private readonly DAL.WitchDAL _witchDAL;

        public Form1_Admin()
        {
            InitializeComponent();
        }
    }
}
```



```
_dashboardService = new BLL.DashboardService();
_witchDAL = new DAL.WitchDAL();

LoadData();
}

/// <summary>
/// 加载魔女列表
/// </summary>
private void LoadData()
{
    try
    {
        // 调用 DAL 层获取数据 (Admin 权限, 无需过滤)
        var dt = _witchDAL.GetWitches();

        // 绑定到 DataGridView
        dataGridViewWitches.DataSource = dt;
        dataGridViewWitches.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.AllCells;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"加载数据失败: {ex.Message}", "错误",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

/// <summary>
/// 添加魔女按钮点击
/// </summary>
private void BtnAddWitch_Click(object? sender, EventArgs e)
{
    try
    {
        var addForm = new UI.WitchAddForm();
        if (addForm.ShowDialog() == DialogResult.OK)
        {
            // 刷新列表
            LoadData();
            MessageBox.Show("添加成功", "提示",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"添加失败: {ex.Message}", "错误",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

/// <summary>
/// 打开可视化大屏
/// </summary>
private void BtnDashboard_Click(object? sender, EventArgs e)
{
    try
    {
```



```
// 调用 BLL 层获取统计数据
var stats = _dashboardService.GetGlobalStats("admin", "Admin", null);

var dashboardForm = new UI.DashboardForm(stats);
dashboardForm.Show();
}
catch (Exception ex)
{
    MessageBox.Show($"打开大屏失败: {ex.Message}", "错误",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
}
```

设计要点:

数据绑定: 使用 DataGridView 的数据绑定功能, 简化数据显示

对话框模式: 使用 ShowDialog() 显示模态对话框, 等待用户操作

数据刷新: 操作完成后刷新数据列表, 保持数据同步

权限控制: Admin 界面可以访问所有数据, 无需过滤

C. UI层文件结构

└── LoginForm.cs	# 登录界面
└── PhoneForm.cs	# 魔女手机主界面
└── BasePokedexForm.cs	# 图鉴基类
└── PokedexForm.cs	# 图鉴·人物
└── EvidenceForm.cs	# 图鉴·证物
└── MapForm.cs	# 图鉴·地图
└── RulesForm.cs	# 图鉴·规定
└── RecordsForm.cs	# 图鉴·记录
└── GomokuModeForm.cs	# 五子棋模式选择
└── GomokuBoardForm.cs	# 五子棋棋盘对弈
└── GomokuMatchLogForm.cs	# 对局历史记录
└── WitchDetailForm.cs	# 魔女详情查看
└── WitchEditForm.cs	# 魔女信息编辑
└── WitchAddForm.cs	# 添加魔女
└── DashboardForm.cs	# 可视化大屏
└── ExecutionForm.cs	# 处刑界面
└── ExecutionPlatformManagementForm.cs	# 处刑平台管理
└── TrialManagementForm.cs	# 审判管理界面
└── TrialVotingForm.cs	# 投票界面
└── CameraForm.cs	# 照相功能
└── ... (其他界面文件)	

根目录/

└── Form1_Admin.cs	# 管理员主界面
└── Form1_Regulator.cs	# 监管员主界面
└── Form1_Warden.cs	# 典狱长主界面
└── Program.cs	# 程序入口点



...

D. UI层设计模式

1. 基类模式

使用基类封装通用功能，减少代码重复：

```
namespace WitchTrialSystem.UI
{
    /// <summary>
    /// 图鉴基类
    /// 提供通用的图鉴浏览功能
    /// </summary>
    public abstract class BasePokedexForm : Form
    {
        protected int _currentIndex = 0;
        protected List<object> _items = new();

        /// <summary>
        /// 加载数据（子类实现）
        /// </summary>
        protected abstract void LoadData();

        /// <summary>
        /// 显示当前项（子类实现）
        /// </summary>
        protected abstract void DisplayCurrentItem();

        /// <summary>
        /// 上一项
        /// </summary>
        protected void PreviousItem()
        {
            if (_currentIndex > 0)
            {
                _currentIndex--;
                DisplayCurrentItem();
            }
        }

        /// <summary>
        /// 下一项
        /// </summary>
        protected void NextItem()
        {
            if (_currentIndex < _items.Count - 1)
            {
                _currentIndex++;
                DisplayCurrentItem();
            }
        }
    }
}
```

2. 对话框模式

使用对话框进行数据输入和确认：

```
public partial class WitchAddForm : Form
```



```
{  
    public WitchAddForm()  
    {  
        InitializeComponent();  
    }  
  
    private void BtnOK_Click(object? sender, EventArgs e)  
    {  
        // 1. 输入验证  
        if (string.IsNullOrEmpty(txtName.Text))  
        {  
            MessageBox.Show("请输入姓名", "提示",  
                MessageBoxButtons.OK, MessageBoxIcon.Warning);  
            return;  
        }  
  
        try  
        {  
            // 2. 调用 BLL 层添加魔女  
            var bll = new BLL.WitchBLL();  
            var result = bll.AddWitch(/* 参数 */);  
  
            if (result.Success)  
            {  
                DialogResult = DialogResult.OK;  
                Close();  
            }  
            else  
            {  
                MessageBox.Show(result.Message, "错误",  
                    MessageBoxButtons.OK, MessageBoxIcon.Error);  
            }  
        }  
        catch (Exception ex)  
        {  
            MessageBox.Show($"添加失败: {ex.Message}", "错误",  
                MessageBoxButtons.OK, MessageBoxIcon.Error);  
        }  
    }  
}
```

3. 数据绑定模式

使用数据绑定简化数据显示:

```
// 绑定 DataTable 到 DataGridView  
dataGridViewWitches.DataSource = _witchDAL.GetWitches();
```



```
// 绑定 List 到 ComboBox
comboBoxIslands.DataSource = _islandDAL.GetIslands();
comboBoxIslands.DisplayMember = "Name";
comboBoxIslands.ValueMember = "IslandID";
```

E. UI层职责总结

1. 界面展示：展示数据、菜单、按钮等 UI 元素
2. 用户交互：接收用户输入、点击、键盘操作等
3. 输入验证：进行基本的输入验证（非空、格式、范围等）
4. 调用 BLL 层：通过 BLL 层完成业务操作
5. 异常处理：捕获异常并友好提示用户
6. 界面跳转：根据业务逻辑跳转到不同界面
7. 数据绑定：使用数据绑定简化数据显示和更新

7.2 数据库连接管理

7.2.1 DBHelper 类设计

DBHelper 作为数据访问层的基础设施类，主要目标是统一管理数据库连接与常用数据库操作，屏蔽底层实现细节，对上层提供简洁易用的 API。

A. 职责定位

连接管理：集中管理连接字符串读取与连接创建逻辑。

命令执行封装：对查询、增删改、标量查询等操作提供统一封装。

资源释放：保证连接、命令、DataReader 等对象在任何情况下都能正确释放。

异常包装：捕获底层 SQLException，包装为更友好的业务异常或日志内容。

B. 类结构示意图

```
namespace WitchTrialSystem.DAL
{
    /// <summary>
    /// 数据库操作帮助类
    /// 负责统一的连接获取与 SQL 执行封装
    /// </summary>
    public static class DBHelper
    {
        /// 从配置中读取并缓存连接字符串
        private static readonly string _connStr = GetConnectionString();

        /// <summary>
        /// 读取连接字符串（可从 appsettings.json / 配置中心 等）
        /// </summary>
        private static string GetConnectionString()
        {
            /// 伪代码：具体实现根据项目配置方式决定
            /// return ConfigurationManager.ConnectionStrings["DefaultConnection"].ConnectionString;
            throw new NotImplementedException();
        }

        /// <summary>
        /// 获取数据库连接对象（注意：SqlConnection 内部支持连接池）
        /// </summary>
    }
}
```



```
public static SqlConnection GetConn()
{
    return new SqlConnection(_connStr);
}

/// <summary>
/// 执行查询 (返回单值)
/// </summary>
public static object? ExecScalar(string sql, params SqlParameter[] ps)
{
    using var conn = GetConn();
    using var cmd = new SqlCommand(sql, conn);
    if (ps is { Length: > 0 }) cmd.Parameters.AddRange(ps);
    conn.Open();
    return cmd.ExecuteScalar();
}

/// <summary>
/// 执行增删改 (返回受影响行数)
/// </summary>
public static int ExecNonQuery(string sql, params SqlParameter[] ps)
{
    using var conn = GetConn();
    using var cmd = new SqlCommand(sql, conn);
    if (ps is { Length: > 0 }) cmd.Parameters.AddRange(ps);
    conn.Open();
    return cmd.ExecuteNonQuery();
}

/// <summary>
/// 执行查询 (返回 DataTable)
/// </summary>
public static DataTable ExecDataTable(string sql, params SqlParameter[] ps)
{
    using var conn = GetConn();
    using var cmd = new SqlCommand(sql, conn);
    if (ps is { Length: > 0 }) cmd.Parameters.AddRange(ps);
    using var adapter = new SqlDataAdapter(cmd);
    var dt = new DataTable();
    adapter.Fill(dt);
    return dt;
}
}
```

C. 设计要点总结

静态类 + 只读字段：保证全局统一入口，连接字符串只读取一次并缓存。

短连接模式：每次执行操作内部创建 `SqlConnection`，用 `using` 自动释放，让 ADO.NET 连接池接管性能问题。

参数化查询：所有方法强制使用 `SqlParameter`，避免字符串拼接导致 SQL 注入。

可扩展性：后续如需支持事务、批量操作、异步方法，只需在 `DBHelper` 内新增重载即可，不影响上层调用代码。



7.2.2 连接池管理

在 .NET 的 `SqlConnection` 中，默认就启用了连接池（`Connection Pooling`）。因此 `WitchTrialSystem` 的连接池管理策略，重点不是“自己手写池”，而是正确地使用和配置 ADO.NET 内置连接池。

A. 连接池工作原理简述

当第一次使用某个连接字符串创建 `SqlConnection` 并 `Open()` 时，ADO.NET 会为该连接字符串创建一个连接池。

`SqlConnection.Close()` / `Dispose()` 并不会真正关闭物理连接，而是将其归还到池中，供下一次 `Open()` 复用。

只要连接字符串完全一致，后续使用 `new SqlConnection(connStr)` 并 `Open()` 都会从池中取出已有连接，大幅降低创建连接的开销。

B. 在 DBHelper 中的最佳实践

始终使用 `using` 包裹 `SqlConnection`，保证无论是否发生异常，都能及时归还到连接池。

一次业务操作只持有短时间连接：在 `ExecNonQuery`、`ExecDataTable` 等内部，先构造 SQL 和参数，再 `Open()`，执行完立即关闭。

不要在全局保存长生命周期的 `SqlConnection` 实例，避免连接泄漏或被占用导致池内连接耗尽。

如果有事务需求，在 `DBHelper` 内提供包装方法：

使用 `SqlConnection + SqlTransaction`，在一个短生命周期内完成多个 SQL 的提交或回滚。

C. 连接池配置

示例在连接字符串中可以通过参数进行简单的池配置，例如

```
Server=(localdb)\mssqllocaldb;  
Database=WitchTrialWT;  
Trusted_Connection=True;  
Pooling=true;  
Min Pool Size=5;  
Max Pool Size=100;  
Connect Timeout=15;
```

Pooling: 是否启用连接池（默认 `true`）。

Min Pool Size: 空闲时保持的最小连接数，适合并发较高、启动瞬间压力较大的场景。

Max Pool Size: 连接池允许的最大连接数，防止无限制创建连接压垮数据库。

Connect Timeout: 连接超时时间，避免长时间卡死在连接阶段。

7.3 安全机制实现

`WitchTrialSystem` 在三层架构之上，构建了一套完整的安全机制，涵盖密码存储、安全访问数据库、权限校验以及大模型 API Key 的安全保存等方面。本节分别从四个子模块进行说明。

7.3.1 密码加密（SHA256 + Salt）

系统中所有用户密码绝不以明文形式存储，而是使用 `SHA256 + Salt` 的方式进行单向加密。核心目标为：

抗暴力破解：密码经过哈希后，即使数据库泄露，攻击者也难以反推出原始密码。

防彩虹表攻击：为每个用户生成独立的 `Salt`，避免使用通用彩虹表进行反查。

可重复验证：登录时使用同一 `Salt` 对输入密码再哈希，对比存库哈希值。

密码加密逻辑由 ``BLL.Security`` 静态类实现，典型实现如下：



```
namespace WitchTrialSystem.BLL
{
    /// <summary>
    /// 安全相关工具类：负责密码加密与验证
    /// </summary>
    public static class Security
    {
        /// <summary>
        /// 生成盐值 + 哈希
        /// </summary>
        public static (string Salt, string Hash) HashPassword(string password)
        {
            // 1. 生成随机盐 (16 字节)
            var saltBytes = RandomNumberGenerator.GetBytes(16);
            string salt = Convert.ToHexString(saltBytes);

            // 2. 计算 SHA256(password + salt)
            using var sha256 = SHA256.Create();
            var passwordBytes = Encoding.UTF8.GetBytes(password + salt);
            var hashBytes = sha256.ComputeHash(passwordBytes);
            string hash = Convert.ToHexString(hashBytes);

            return (salt, hash);
        }

        /// <summary>
        /// 验证密码是否正确
        /// </summary>
        public static bool Verify(string password, string salt, string hash)
        {
            using var sha256 = SHA256.Create();
            var passwordBytes = Encoding.UTF8.GetBytes(password + salt);
            var hashBytes = sha256.ComputeHash(passwordBytes);
            string computedHash = Convert.ToHexString(hashBytes);

            return computedHash.Equals(hash, StringComparison.OrdinalIgnoreCase);
        }
    }
}
```

存储规范：

- `Salt` 和 `PasswordHash` 分别存储在 `wt.[User]` 表的对应字段中。
- 业务层永远不保存明文密码，UI 层在使用完输入的密码后也不再缓存。
- 如需重置密码，由 BLL 重新生成新的 `(Salt, Hash)` 并更新数据库。

7.3.2 SQL 注入防护

系统严格采用参数化查询以及统一的 `DBHelper` 封装，最大限度降低 SQL 注入风险。

A. 参数化查询统一规范

- 所有 DAL 类在构造 SQL 时，禁止使用字符串拼接用户输入。
- 一律通过 `SqlParameter` 传递参数，由数据库引擎负责参数解析与类型检查。

示例（以 `UserDAL.GetByUsername` 为例）：

```
const string sql = @"
    SELECT TOP 1 UserID, Username, RoleID, Salt, PasswordHash
    FROM wt.[User]
```




```
WHERE Username = @u";
```

```
var dt = DBHelper.ExecDataTable(sql, new SqlParameter("@u", username));
```

B. DBHelper 层统一封装

`DBHelper.ExecDataTable / ExecNonQuery / ExecScalar` 统一接收 `SqlParameter[]`，内部不再对 SQL 拼接做任何加工，避免二次污染。

C. 输入验证与长度限制

- UI 层与 BLL 层对用户输入统一做长度、格式、白名单验证，例如用户名不得包含危险字符（如 `; -- /\* \*/` 等）。
- 对字段长度进行约束（如用户名最长 32 字符），即使出现恶意输入，也会在进入数据库前被截断或拒绝。

7.3.3 权限验证机制

WitchTrialSystem 采用四层权限体系设计，将“谁可以做什么”细化到角色、菜单、按钮乃至具体业务操作。

A. 角色与用户绑定

- `wt.[User]` 表中通过 `RoleId` 字段与角色表关联。
- 登录成功后，系统会缓存当前用户的 `UserID、Username、RoleId` 等信息，供后续权限判断使用。

B. 菜单 / 功能点权限

- 系统界面在加载时，会根据用户角色从数据库中查询对应的可访问菜单、功能点列表。
- UI 层根据权限结果对按钮、菜单项进行 **显示/隐藏或启用/禁用** 操作。

C. 业务操作权限校验

BLL 层在敏感操作（如添加/删除魔女、变更审判结果、操作处刑平台等）前，统一调用权限校验方法，例如：

```
public class TrialSessionService
{
    private readonly PermissionService _permission;

    public void CloseSession(int sessionId, int currentUserId)
    {
        // 1. 权限校验
        if (!_permission.CanCloseSession(currentUserId, sessionId))
        {
            throw new UnauthorizedAccessException("当前用户无权关闭该审判会话。");
        }

        // 2. 通过校验后，执行后续业务逻辑
        // ...
    }
}
```

即使 UI 层被绕过（例如通过伪造请求），BLL 层的权限校验也能阻止非法操作。



8.核心功能演示及对应代码

8.1 用户登录与权限验证

8.1.1 登录界面与流程

本系统采用典型的账号密码登录流程，登录入口为桌面客户端的 LoginForm（参考文件：LoginForm.cs）。登录界面以简洁的表单为主，包含“用户名”“密码”输入框与“登录”按钮，并在界面上提供“记住我/切换账号”“忘记密码/帮助”之类的辅助手段。界面风格与项目整体视觉保持一致，背景使用资源目录下的 UI 背景图片，控件尽量设置为透明以保证与背景和谐显示。

登录流程（步骤）：

1. 用户在 LoginForm 输入用户名与密码，点击“登录”。
2. 前端对输入进行简单校验（非空、长度限制）。
3. 前端将用户名提交到 BLL 层（UserBLL）或直接调用 UserDAL 获取该用户的密码盐与哈希值。
4. 使用 BLL/Security 中的哈希验证函数对密码进行验证。
5. 验证通过后，根据用户角色加载相应主界面；

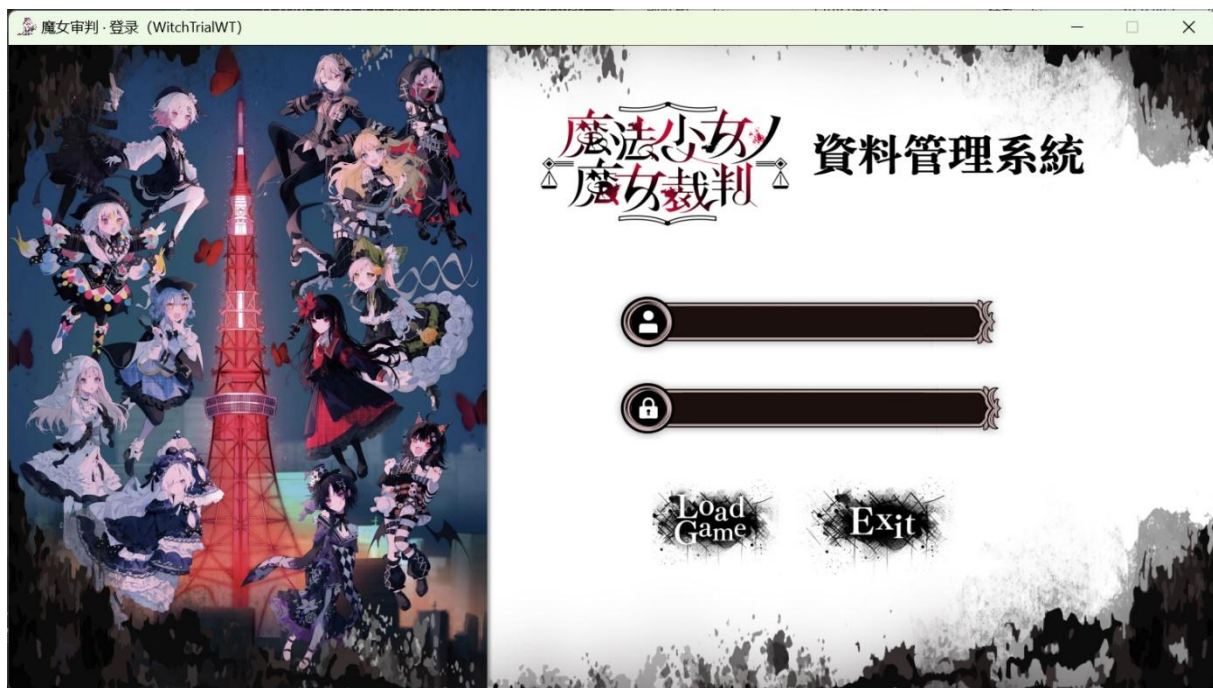


图 8 登录界面

登录事件核心代码：

```
```csharp
// UI/LoginForm.cs - OnLogin
private void OnLogin(object? sender, EventArgs e)
```



```
{
 try
 {
 var username = _txtUser.Text.Trim();
 var password = _txtPass.Text.Trim();
 if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))

 {
 _msg.Text = "请输入英文账号和密码。";
 return;
 }

 var bll = new UserBLL();
 var res = bll.Login(username, password);
 if (res == null)
 {
 _msg.Text = "账号或密码错误。";
 return;
 }

 // 登录成功: 根据角色路由到不同主界面
 var upDal = new UserProfileDAL();
 var prof = upDal.GetProfile(username);
 string role = (prof.Rows.Count > 0 ? prof.Rows[0]["RoleName"] as string :
null) ?? "Witch";

 Form main;
 switch (role)
 {
 case "Admin":
 main = new Form1_Admin(username);
 break;
 case "Meruru":
 case "Utena":
 main = new Form1_Regulator(username);
 break;
 case "Warden":
 main = new Form1_Warden(username);
 break;
 case "Witch":
 main = new WitchTrialSystem.UI.PhoneForm(username);
 break;
 default:
 main = new Form1(username)
 {
 Text = $"魔女审判 · 主面板（当前用户: {username}）"
 };
 break;
 }

 MessageBox.Show("登录成功!", "OK");
 this.Hide();
 main.Show();
 }
 catch (Exception ex)
 {
 MessageBox.Show("登录异常: " + ex.Message, "Error");
 }
}
...
```



### 8.1.2 密码验证代码

系统采用 Salt+SHA256 的哈希方案保存密码，相关实现位于`BLL/Security.cs`。在用户首次初始化或管理员批量初始化时，会生成 Salt 与 Hash 并写入数据库；登录时读取 Salt 并比对生成的哈希值。

核心代码：

```
```csharp
// BLL/Security.cs
using System.Security.Cryptography;
using System.Text;

namespace WitchTrialSystem.BLL
{
    public static class Security
    {
        public static string CreateSalt(int bytes = 16)
        {
            var buf = new byte[bytes];
            RandomNumberGenerator.Fill(buf);
            return Convert.ToHexString(buf);
        }

        public static string Sha256Hex(string text)
        {
            using var sha = SHA256.Create();
            var hash = sha.ComputeHash(Encoding.UTF8.GetBytes(text));
            return Convert.ToHexString(hash);
        }

        public static (string Salt, string Hash) HashPassword(string password)
        {
            var salt = CreateSalt();
            var hash = Sha256Hex(password + salt);
            return (salt, hash);
        }

        public static bool Verify(string password, string salt, string hash)
            => Sha256Hex(password + salt).Equals(hash,
StringComparison.OrdinalIgnoreCase);
    }
}
```
```

后端登录逻辑：

```
```csharp
```



```
// BLL/UserBLL.cs - Login
public (int UserID, string Username, int RoleID)? Login(string username, string
password)
{
    var u = _dal.GetByUsername(username);
    if (u == null) return null;

    if (string.Equals(u.Value.Salt, "PENDING",
System.StringComparison.OrdinalIgnoreCase) ||
        string.Equals(u.Value.Hash, "PENDING",
System.StringComparison.OrdinalIgnoreCase))
        return null;

    var ok = Security.Verify(password, u.Value.Salt, u.Value.Hash);
    if (!ok) return null;

    return (u.Value.UserID, u.Value.Username, u.Value.RoleID);
}
...

```

8.1.3 角色判断与界面跳转

登录成功后系统根据用户角色决定展示与权限范围。角色信息由`UserProfileDAL`或`UserDAL`提供，前端根据角色启用/隐藏特定按钮，后端在 BLL/DAL 层再次校验以防越权。

核心代码片段：

```
```csharp
var upDal = new UserProfileDAL();
var prof = upDal.GetProfile(username);
string role = (prof.Rows.Count > 0 ? prof.Rows[0]["RoleName"] as string : null) ??
"Witch";

Form main;
switch (role)
{
 case "Admin":
 main = new Form1_Admin(username);
 break;
 case "Meruru":
 case "Utena":
 main = new Form1_Regulator(username);
 break;
 case "Warden":
 main = new Form1_Warden(username);
 break;
 case "Witch":

```





```
main = new WitchTrialSystem.UI.PhoneForm(username);

break;

default:

main = new Form1(username) { Text = $"魔女审判 · 主面板（当前用户: {username}）"
};

break;

}
...
```

## 8.2 四层权限主界面

### 8.2.1 国家端 Admin 主界面 (Form1\_Admin)

界面概览：界面采用“功能分区 + 数据可视化”布局，核心分为三部分。左上角为用户信息卡片，直观展示管理员头像、登录账号、角色类型及“退出登录”等快捷操作按钮；顶部工具条集成岛屿选择、批次筛选、关键词搜索功能，以及“新增魔女”“处刑台管理”“数据刷新”等核心操作按钮；主体区域为魔女数据网格，支持双击查看完整档案、右键触发快捷操作，全面呈现 43 字段核心数据。

魔女审判 · 管理界面

账号: admin

角色: Admin

中文名: 一

修改密码

退出登录

岛屿全部

批次全部

按名字搜索

刷新

国家层添加

删除魔女

智慧大屏

智慧大模型

处刑台管理

移动记录

共 54 条

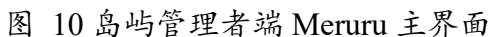
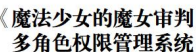
岛屿名称	全国批	本岛批	囚人番	个人番号	姓名	性	年	状态	身高	体重	血	魔法	最高学历	籍贯	电话	邮箱	技能特长	兴趣爱好	理想	心理创伤	ID	公开描述
1 魔女岛壹	1	1	658	1234-5678	德可艾玛	女	15	审判中	156	48.00	A	魔法东丰	中学校卒	东京都	03-1234	sakuraba_ema@y	指挥能力/烹饪	寻找美食店、和	交 100 个朋友	突然好友月代雪前辈	3	15 岁，经检测确认为恶魔魔女。
1 魔女岛壹	1	1	659	1234-5678	二阶堂裕	女	15	死亡(正常)	157	50.00	B	死而复活	中学校卒	神奈川县	045-6789	nikaido_hiro@yah	成绩优秀、运动	与助人日记、行	创造无限的战争	好友自杀后后悔莫及	4	15 岁，艾玛的儿时玩伴。曾...
1 魔女岛壹	1	1	660	1234-5678	夏目安女	女	15	死亡(魔)	152	45.00	AB	洗脑	中学校卒	东京都	无(无话)	natsume_an@yah	笔谈交流、游戏	宅家打游戏、写	社交障碍、因小事自	5	她杀害了佐伯米莉亚，经事...	
1 魔女岛壹	1	1	661	1234-5678	城崎亚世	女	15	死亡(正常)	160	47.00	O	液体操作	中学校卒	大府府	06-7890	shirosaki_noa@ya	画画、探索未知	画出完美作品	对作品不满的执念	6	其实她身份是世界有名的街...	
1 魔女岛壹	1	1	662	1234-5678	连见露煌	女	15	死亡(魔)	163	51.00	A	液体操作	中学校卒	东京都	03-5678	hasumi_reo@yah	表演能力、团队	剧团指挥	成为偶像演员	因独立性性格而压力	7	艺能事务所所属的偶像魔女...
1 魔女岛壹	1	1	663	1234-5678	佐佐米莉	女	15	死亡(正常)	155	46.00	B	身体互换	中学校卒	埼玉县	048-6789	saeaki_miria@yah	照顾他人、电竞	看台摄影、写作	成为可靠的大姐	为合群而积极期待	8	在中学时期是暴力安全委员...
1 魔女岛壹	1	1	664	1234-5678	宝生玛希	女	15	审判中	158	49.00	AB	声音模仿	中学校卒	东京都	03-4567	hoshou_mari@ya	戏剧技巧、口才	观察他人、掌握	害怕孤独、导致不	9	因模仿时渴望兴趣和表达...	
1 魔女岛壹	1	1	665	1234-5678	栗原奈叶	女	15	死亡(正常)	154	44.00	O	幻术	中学校卒	京都府	075-5678	karube_natsuka@	单独行动、调查	揭开妖怪和魔法	失去姐姐后性格大变	10	所结的缘分是过于身临其境...	
1 魔女岛壹	1	1	666	1234-5678	栗原露煌	女	14	死亡(魔)	157	46.00	A	发火	中学校卒	兵库县	078-7890	shidou_arisa@yah	情绪爆发、力量	随心所欲、不受	被领导他人厌恶	11	她在处刑台处遭人杀害...	
1 魔女岛壹	1	1	667	1234-5678	橘雪莉	女	14	死亡(魔)	165	52.00	B	怪力	中学校卒	千叶县	043-8901	tachibana sherry	怪力、体力超群	健身、吃零食	怪力的守护者	因怪力被孤立、渴望	12	在福利所长大的孤儿。她...
1 魔女岛壹	1	1	668	1234-5678	远野汉娜	女	14	死亡(正常)	153	43.00	AB	浮游	中学校卒	北海道	011-3456	tsunou_hanna@ya	浮游、自由飞翔	眺望天空、飞翔	渴望自由和冒险	13	在福利所中遭遇过欺凌...	
1 魔女岛壹	1	1	669	1234-5678	漆原可可	女	14	审判中	150	42.00	O	千里眼	中学校卒	爱知县	052-5678	sawatari_coco@y	千里眼、信息	观察他人、收集	掌握所有信息	因知晓太多秘密而痛苦	14	日常中习惯开解烦恼...
1 魔女岛壹	1	1	670	1234-5678	冰上梅露	女	20	审判中	158	48.00	不明	治疗	无	不明	076-555	mizore_meruru@	治愈、再生、植	培育资料植物	作为最后手段的自决	15	拥有魔法是可以瞬间治疗...	
1 魔女岛壹	2	2	671	5678-1234	小岛游六	女	24	分配至岛	150	47.00	AB	霸王再眼	高中在读	富山县	076-555	takanashi_rikka@	中二病设定构建	中二行为扮演	寻找不可视境	父亲去世后因无法接	16	自称拥有“霸王再眼”的中二...
1 魔女岛壹	2	2	672	5678-1234	高槻勇太	男	24	分配至岛	170	60.00	O	读心术	高中在读	千叶县	043-1234	togashi_yuta@ex	游戏、阅读、静	成为普通的社会	初中时中二病被孤立	17	曾经是中二病患者，现在理...	
1 魔女岛壹	2	2	673	5678-1234	丹生百合	女	24	分配至岛	165	57.00	A	毒雾魔法	高中在读	千叶县	043-2345	niwa_takanashi@	毒雾管理、情报	cosplay、社团	成为完美的社交	初中时中二病历史	20	温柔而感性的少女，相信魔...
1 魔女岛壹	2	2	674	5678-1234	五月七日	女	23	分配至岛	155	45.00	AB	面兽魔法	高中在读	千叶县	043-3456	itsuki_mayoi@ex	占卜、观察人心	成为知名占卜师	无(性格天然、无谓)	21	温柔而感性的少女，相信魔...	
1 魔女岛壹	2	2	675	5678-1234	凸守早苗	女	23	分配至岛	143	45.00	B	雷之姐妹	高中在读	千叶县	043-4567	dekumori_sanae	中二设定构建	中二扮演、追随	成为百花的主角	无(天生元气、沉迷)	22	神秘而冷酷的少女，拥有不...
1 魔女岛壹	2	2	676	5678-1234	七海智音	女	23	分配至岛	150	40.00	O	智音魔法	中学毕业	千叶县	043-4567	shichimiya_tomo	小提琴演奏、音	拉小提琴、作曲	用音乐传达心意	被音乐治愈过	23	身份神秘的成年魔女，自称...
1 魔女岛壹	2	2	677	9012-3456	伊藤智	女	22	分配至岛	150	42.00	O	友之魔法	魔女最高	和平国	076-7890	irena_witch@ex	精通各类魔法	旅行、阅读、品	环游世界、记录	旅行中目睹过人性	24	身份神秘的成年魔女，自称...
1 魔女岛壹	2	2	678	4321-8765	维多利加	女	48	分配至岛	158	47.00	O	高阶金属	魔女最高	和平国	076-7890	victoria_nicole@	精通各类魔法	骑马旅行故事	通过文字与弟子	幼年经历洪水失去双亲	25	优雅而神秘的女性，拥有占...
1 魔女岛壹	2	2	679	5678-9012	沙耶	女	22	分配至岛	155	45.00	B	扫帚魔法	魔女中位	爱知县	043-4567	saya_broom@exa	扫帚魔法专精	擦拭扫帚、练习	成为职业地区最	曾因魔法天赋不足被	26	梦幻而神秘的少女，似乎有...
1 魔女岛壹	2	2	680	2143-6587	美兰	女	32	分配至岛	160	48.00	A	魔眼魔法	魔女最高	和平国	076-1234	fran_star@exampl	魔眼魔法专精	观察虚空、魔法	培养出更多优秀	幼时因特殊能力被	27	冷静而坚韧的少女，拥有修...
1 魔女岛壹	2	2	681	2143-6587	南拉	女	31	分配至岛	157	46.00	AB	暗杀魔法	魔女最高	和平国	076-1234	shira_dora@exam	夜袭魔法专精	夜袭魔法、暗杀	童年因目睹过暗杀	28	神秘而冷酷的少女，拥有无...	
1 魔女岛壹	2	2	682	1985-0202	国田	女	40	分配至岛	157	46.00	O	飞行魔法	高中在读	神奈川县	0120-354	kiki_delivery@exa	飞行魔法、物品	飞行、观察城市	成为独立自由的	第一次飞行时迷路	29	独立而勇敢的少女，拥有飞...
1 魔女岛壹	2	2	683	1234-5678	冰上梅露	女	125	审判中	158	48.00	未知	治愈再生	无	不明	076-555	mizore_meruru@	治愈、再生、植	培育资料植物	作为最后手段的自决	30	拥有魔法是可以瞬间治疗...	
2 魔女岛贰	4	1	684	1001-0305	袴神露煌	女	17	审判中	158	45.00	O	邪眼魔法	中学校卒	东京都	03-3355	mttn_hiragi@exa	邪眼魔法专精	收集魔法少女泪	以身份身份享受	首次战斗时遭受受	31	邪眼魔法艾玛的妹妹...
2 魔女岛贰	4	1	685	0222-4444	阿奈河琪	女	17	审判中	148	36.00	AB	替身魔法	中学校卒	东京都	03-3355	kiwi_aragaki@exa	替身魔法专精	自拍照、监视	成为替身小萝莉	因身材矮小被排	32	邪眼魔法艾玛的妹妹...
2 魔女岛贰	4	1	686	1212-3333	杜乃可莉	女	13	死亡(正常)	140	38.00	A	厄运魔法	小学校卒	东京都	03-3925	korisu_morino@e	厄运魔法、玩具	收集制作玩偶	创造所有玩偶	性格内向难以交友	33	邪眼魔法艾玛的妹妹...

图 9 国家段 admin 主界面

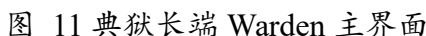
### 8.2.2 岛屿管理者端 Meruru 主界面 (Form1\_Regulator)

界面概览：布局与管理员界面保持一致性，降低学习成本，核心聚焦监管职责。同样包含用户信息卡片、顶部工具条与数据网格三大模块，工具条简化非监管相关功能，右键菜单新增“编辑公开描述”等专属操作，重点支撑魔女档案公开信息的审核与维护流程。





界面概览：围绕审判执行与处刑平台管理核心职责设计，顶部工具条突出“处刑台管理”“移动记录查询”“审判管理”等专属功能按钮，主体数据网格保留魔女核心信息（如姓名、囚犯编号、状态），支持快速查阅目标对象并启动相关流程，界面简洁且操作路径短。



界面采用“手机外壳”仿真设计，贴合游戏原作场景，界面紧凑且聚焦核心需求。顶部展示个人信息（头像、姓名、囚犯编号），中部为功能入口区（魔女图鉴、处刑、



录音)，底部为快捷操作栏（五子棋对弈、五子棋对局记录、五子棋积分排行榜、对话），整体交互符合移动端操作习惯。



图 12 普通魔女端 Witch 主界面

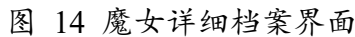
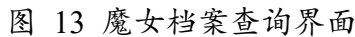
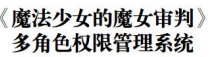
## 8.3 魔女档案管理（43 字段完整档案）

### 8.3.1 魔女档案查询（WitchDetailForm）

以只读形式完整展示单个魔女的 43 字段档案信息，包含基本信息、个人详情、教育工作、家庭信息等六大维度，同时提供“编辑档案”“以此新增”快捷入口，支撑档案查看与快速操作的业务流程。

在国家管理层、岛屿监管者、典狱长三层的主界面，双击某一行角色信息，可以查看该角色的详细信息档案，支持打印、导出 PDF，或导出 HTML 在浏览器中查看。





```
using System.Data;
using WitchTrialSystem.DAL;
using WitchTrialSystem.BLL;
```



```
namespace WitchTrialSystem.UI
{
 public class WitchDetailForm : Form
 {
 private readonly int _witchId;
 private readonly WitchDAL _witchDAL = new WitchDAL();
 private readonly PermissionBLL _permissionBLL = new PermissionBLL();

 public WitchDetailForm(int witchId)
 {
 _witchId = witchId;
 Load += (_, __) => LoadWitchCoreData();
 }

 // 核心：加载档案数据+权限过滤
 private void LoadWitchCoreData()
 {
 // 1. 数据访问
 DataTable witchDt = _witchDAL.GetById(_witchId);
 if (witchDt.Rows.Count == 0) { MessageBox.Show("档案不存在"); Close();
return; }

 DataRow row = witchDt.Rows[0];

 // 2. 权限控制：普通魔女仅能查看自身档案
 if (_permissionBLL.GetCurrentRole() == "Witch"
&& !_permissionBLL.CanAccessWitch(_witchId))
 {
 MessageBox.Show("无权限访问他人档案");
 Close();
 return;
 }

 // 3. 核心字段展示（省略布局，聚焦数据绑定）
 Console.WriteLine($"姓名: {row["Name"]} | 囚犯编号: {row["PrisonerNo"]}
| 状态: {row["Status"]} | 魔法: {row["Magic"]}");
 }

 // 核心交互：编辑档案
 private void OpenEditForm()
 {
 if (_permissionBLL.IsAdmin())
 {
 using var editForm = new WitchEditForm(_witchId);
 editForm.ShowDialog();
 LoadWitchCoreData(); // 编辑后刷新
 }
 }
 }
}
```



```
 }
 else
 {
 MessageBox.Show("无编辑权限");
 }
}
}
```

技术亮点：

1. 结构化布局：采用 TableLayoutPanel 四列布局，将 43 个字段按业务分类有序展示，标签与值对应清晰，支持自动滚动，适配大量字段的可视化需求。
2. 权限适配：界面默认只读，仅通过快捷按钮提供操作入口，后端通过 PermissionDAL 校验角色权限，确保只有 Admin 可执行编辑操作。
3. 用户体验优化：字段值默认显示“—”替代空值，关键字段（如所属批次）拼接友好提示，教育/工作经历字段提示“点击编辑查看详情”，引导用户操作。

### 8.3.2 魔女档案编辑（WitchEditForm）

为有权限用户（仅 Admin）提供 43 字段的完整编辑能力，支持基本信息修改、JSON 格式经历编辑、数据验证等功能，确保档案更新的准确性与合法性。

姓名*	樱羽艾玛	囚犯编号:	658
个人番号:	1234-5678-9011	性别:	女
出生日期:	2010/ 3/ 5	民族:	大和民族
籍贯:	东京都	曾用名:	无

保存修改 取消

图 15 魔女档案编辑界面



核心代码:

```
using System.Data;
using System.Text.Json;
using WitchTrialSystem.DAL;
using WitchTrialSystem.BLL;

namespace WitchTrialSystem.UI
{
 public class WitchEditForm : Form
 {
 private readonly int _witchId;
 private readonly WitchDAL _witchDAL = new WitchDAL();
 private readonly PermissionBLL _permissionBLL = new PermissionBLL();
 private DataTable _witchDt;

 // 核心输入字段（仅保留业务必需）
 private string _name, _magic, _status, _educationJson, _workJson;

 public WitchEditForm(int witchId)
 {
 _witchId = witchId;

 // 核心权限校验：仅Admin可进入
 if (!_permissionBLL.IsAdmin())
 {
 MessageBox.Show("仅管理员可编辑");
 DialogResult = DialogResult.Cancel;
 Close();
 return;
 }

 Load += (_, __) => LoadCoreData();
 }

 // 核心：加载待编辑数据
 private void LoadCoreData()
 {
 _witchDt = _witchDAL.GetById(_witchId);
 DataRow row = _witchDt.Rows[0];
 // 绑定核心字段
 _name = row["Name"]?.ToString() ?? "";
 _magic = row["Magic"]?.ToString() ?? "";
 _status = row["Status"]?.ToString() ?? "";
 _educationJson = row["EducationHistory"]?.ToString() ?? "[]";
 _workJson = row["WorkHistory"]?.ToString() ?? "[]";
 }
 }
}
```





```
}

// 核心：保存编辑（数据验证+更新）
private void SaveCoreChanges()
{
 // 1. 基础验证
 if (string.IsNullOrEmpty(_name)) { MessageBox.Show("姓名必填");
return; }

 // 2. JSON 格式验证
 try
 {
 JsonDocument.Parse(_educationJson);
 JsonDocument.Parse(_workJson);
 }
 catch (JsonException ex)
 {
 MessageBox.Show($"JSON 格式错误: {ex.Message}");
 return;
 }

 // 3. 数据更新
 DataRow row = _witchDt.Rows[0];
 row["Name"] = _name;
 row["Magic"] = _magic;
 row["Status"] = _status;
 row["EducationHistory"] = _educationJson;
 row["WorkHistory"] = _workJson;
 row["UpdatedAt"] = DateTime.Now;

 // 4. 调用DAL 提交
 bool success = _witchDAL.UpdateWitch(row);
 MessageBox.Show(success ? "保存成功" : "更新失败");
 if (success) Close();
}

// 核心：关联经历编辑器
private void OpenEducationEditor()
{
 using var editor = new EducationEditorForm(_educationJson);
 if (editor.ShowDialog() == DialogResult.OK)
 {
 _educationJson = editor.JsonResult;
 }
}
```

```
}
}
```

技术亮点：

- 1. 分层校验机制：前端实现必填项、数据格式、JSON 合法性校验，后端通过存储过程 wt.sp\_UpdateWitchComplete 执行事务性更新，双重保障数据准确性。
- 2. 可视化编辑适配：为 JSON 格式的教育/工作经历提供专用编辑器入口，兼顾技术存储需求与用户操作便捷性。
- 3. 权限前置控制：初始化时通过 PermissionBLL.IsAdmin()校验角色，无权限用户直接拦截并关闭界面，配合后端方法二次校验，确保权限安全。

8.3.3 教育与工作经历管理（JSON 存储）

针对教育经历、工作经历等可变长度的结构化数据，采用 JSON 格式存储并提供可视化编辑工具，支持经历的新增、编辑、删除操作，实现灵活扩展与规范存储的平衡。

编辑魔女信息 - ID: 3

基本信息

身体特征

联系方式

教育背景

工作经历

家庭关系

个性特征

魔女信息

分配信息

时间记录

最高学历：

中学校毕业

教育经历：

	学校	学历	状态	特殊说明
▶	东京都立樱丘中学校	中学校	毕业	初中时旁观好友月代雪霸凌致其
	东京都立樱丘高等学校	高等学校	未入学	高中开学前一日被抓至魔女岛

添加

编辑

删除

保存修改

取消

图 16 教育经历管理

核心代码：

```
using System.Collections.Generic;
using System.Text.Json;
```



```
using System.Windows.Forms;

namespace WitchTrialSystem.UI
{
 public class ExperienceItem
 {
 public string Period { get; set; } = "";
 public string Organization { get; set; } = "";
 public string Position { get; set; } = "";
 public string Remarks { get; set; } = "";
 }

 public class EducationEditorForm : Form
 {
 private List<ExperienceItem> _experienceList = new();

 // 核心: 序列化结果 (JSON 输出)
 public string JsonResult => JsonSerializer.Serialize(_experienceList, new
 JsonSerializerOptions { WriteIndented = true });

 public EducationEditorForm(string initialJson)
 {
 // 核心: 反序列化初始化
 LoadInitialJson(initialJson);
 }

 // 核心: JSON 反序列化加载
 private void LoadInitialJson(string initialJson)
 {
 try
 {
 _experienceList = string.IsNullOrEmpty(initialJson)
 ? new List<ExperienceItem>()
 :
 JsonSerializer.Deserialize<List<ExperienceItem>>(initialJson) ?? new
 List<ExperienceItem>();
 }
 catch
 {
 _experienceList = new List<ExperienceItem>();
 MessageBox.Show("JSON 格式无效");
 }
 }

 // 核心: 新增/编辑经历
 }
}
```



```
private void AddOrEditItem(ExperienceItem? item = null)
{
 // 省略编辑界面布局，聚焦数据逻辑
 ExperienceItem newItem = item ?? new ExperienceItem();
 // 模拟用户输入赋值
 newItem.Period = "2020-2023";
 newItem.Organization = "魔法学院";
 newItem.Position = "学员";

 if (item == null)
 _experienceList.Add(newItem);
 else
 _experienceList[_experienceList.IndexOf(item)] = newItem;
}

// 核心：删除经历
private void DeleteItem(ExperienceItem item)
{
 _experienceList.Remove(item);
}
}
```

技术亮点：

1. 结构化序列化：定义 ExperienceItem 模型与 JSON 字段一一对应，通过 JsonSerializer 实现标准化序列化/反序列化，避免格式混乱。
2. 可视化交互：采用 ListView 表格展示经历数据，配合新增/编辑/删除功能按钮，操作逻辑清晰，符合用户日常使用习惯。
3. 容错性设计：支持无效 JSON 自动转为空列表，编辑过程中实时刷新数据，确保操作流畅性与数据安全性。

集成说明：

在 WitchEditForm 中，通过“可视化编辑”按钮调用该编辑器，编辑完成后将 JsonResult 赋值给对应的 JSON 字段（EducationHistory 或 WorkHistory），最终通过 WitchDAL.UpdateWitch 方法保存至数据库。

#### 8.3.4 添加新魔女（WitchAddForm）

提供魔女档案的完整新增入口，支持空白录入与模板克隆两种模式，实现 43 字段的批量初始化，同时联动账号创建功能，完成“档案录入-账号开通”的闭环流程。



编辑魔女信息 - ID: 3

基本信息 身体特征 联系方式 教育背景 工作经历 家庭关系 个性特征 魔女信息 分配信息 时间记录

最高学历: 中学校毕业

教育经历:

	学校	学历	状态	特殊说明
▶	东京都立樱丘中学校	中学校	毕业	初中时旁观好友月代雪霸凌致其
	东京都立樱丘高等学校	高等学校	未入学	高中开学前一日被抓至魔女岛

添加 编辑 删除

保存修改 取消

图 17 添加魔女界面

核心代码:

```
using System;
using System.Data;
using System.Windows.Forms;
using WitchTrialSystem.DAL;

namespace WitchTrialSystem.UI
{
 public class WitchAddForm : Form
 {
 private readonly WitchDAL _dal = new WitchDAL();
 private readonly int? _cloneFromId;

 // 核心输入控件（仅保留业务必需）
 private readonly TextBox _tbName = new TextBox();
 private readonly TextBox _tbAge = new TextBox();
 private readonly ComboBox _cbGender = new ComboBox();
 private readonly TextBox _tbMagic = new TextBox();
 private readonly TextBox _tbEducationJson = new TextBox { Multiline = true,
```



```
Height = 80 };

public WitchAddForm(int? cloneFromId = null)
{
 _cloneFromId = cloneFromId;
 Text = cloneFromId.HasValue ? "克隆新增魔女" : "新增魔女";
 Size = new Size(700, 600);
 StartPosition = FormStartPosition.CenterParent;

 // 简化布局与控件初始化
 Controls.AddRange(new Control[] {
 new Label { Text = "姓名*", Top = 30, Left = 30 }, _tbName { Top = 30,
Left = 120, Width = 500 },
 new Label { Text = "年龄*", Top = 70, Left = 30 }, _tbAge { Top = 70,
Left = 120, Width = 500 },
 new Label { Text = "性别", Top = 110, Left = 30 }, _cbGender { Top =
110, Left = 120, Width = 500 },
 new Label { Text = "魔法能力", Top = 150, Left = 30 }, _tbMagic { Top
= 150, Left = 120, Width = 500 },
 new Label { Text = "教育经历(JSON)", Top = 190, Left = 30 },
_tbEducationJson { Top = 190, Left = 120, Width = 500 },
 new Button { Text = "保存并创建账号", Top = 300, Left = 250, Click +=
BtnSave_Click }
 });

 _cbGender.Items.AddRange(new[] { "Male", "Female", "Other" });
 Load += (_, __) => LoadCloneData();
}

// 核心: 加载克隆模板数据
private void LoadCloneData()
{
 if (_cloneFromId.HasValue)
 {
 DataTable dt = _dal.GetById(_cloneFromId.Value);
 if (dt?.Rows.Count > 0)
 {
 DataRow r = dt.Rows[0];
 _tbName.Text = r["Name"]?.ToString() ?? "";
 _tbAge.Text = r["Age"]?.ToString() ?? "";
 _cbGender.SelectedItem = r["Gender"]?.ToString();
 _tbMagic.Text = r["Magic"]?.ToString() ?? "";
 _tbEducationJson.Text = r["EducationJson"]?.ToString() ?? "[";
 }
 }
}
```





```
// 核心: 验证+提交+联动账号创建
private void BtnSave_Click(object? sender, EventArgs e)
{
 // 1. 基础验证
 if (string.IsNullOrEmpty(_tbName.Text)) { MessageBox.Show("姓名必填"); return; }
 if (!int.TryParse(_tbAge.Text, out int age)) { MessageBox.Show("年龄需为整数"); return; }

 try
 {
 // 2. 构造新增参数
 var newWitch = new
 {
 Name = _tbName.Text.Trim(),
 Age = age,
 Gender = _cbGender.SelectedItem?.ToString() ?? "Other",
 Magic = _tbMagic.Text.Trim(),
 EducationJson =
string.IsNullOrEmpty(_tbEducationJson.Text) ? "[]" : _tbEducationJson.Text
 };

 // 3. 调用 DAL 新增, 返回新魔女 ID
 int newId = _dal.AddWitch(newWitch);
 if (newId <= 0) throw new Exception("新增失败");

 MessageBox.Show($"新增成功! WitchID: {newId}");
 // 4. 联动创建账号
 using var accountForm = new CreateWitchAccountDialog(newId);
 accountForm.ShowDialog(this);

 DialogResult = DialogResult.OK;
 Close();
 }
 catch (Exception ex)
 {
 MessageBox.Show($"新增失败: {ex.Message}");
 }
}
}
```

技术亮点:

1. 联动加载机制: 岛屿与批次下拉框联动, 选择岛屿后自动加载对应批次, 减少用户操作步骤, 保证数据关联性。
2. 模板克隆优化: 克隆模式预填非唯一字段, 自动生成递增的囚犯编号, 大幅提升



批量新增效率，适配“13 人/批次”的业务场景。

3. 闭环流程设计：档案新增成功后直接提供账号创建选项，实现“档案-账号”一键关联，符合系统管理流程的完整性要求。

### 8.3.5 创建新魔女账号

为已创建档案的魔女开通登录账号，支持角色分配、初始密码生成，同时提供魔女状态快速修改入口，适配“账号管理-状态管控”的核心需求。

核心代码：

```
using WitchTrialSystem.DAL;
using WitchTrialSystem.BLL;

namespace WitchTrialSystem.UI
{
 public class CreateWitchAccountDialog : Form
 {
 private readonly int _witchId;
 private readonly UserBLL _userBLL = new UserBLL();
 private readonly WitchDAL _witchDAL = new WitchDAL();

 // 核心字段
 private string _username, _password;
 private string _selectedRole = "Witch";
 private string _newStatus;

 public CreateWitchAccountDialog(int witchId)
 {
 _witchId = witchId;
 Load += (_, __) => LoadWitchBaseInfo();
 }

 // 核心：加载魔女基础信息（预填用户名）
 private void LoadWitchBaseInfo()
 {
 var witchDt = _witchDAL.GetById(_witchId);
 if (witchDt.Rows.Count > 0)
 {
 string witchName = witchDt.Rows[0]["Name"]?.ToString() ?? "";
 _username = PinYinHelper.GetFirstPinyin(witchName).ToLower(); // 拼音简化
 _newStatus = witchDt.Rows[0]["Status"]?.ToString() ?? "分配至岛屿";
 }
 }
 }
}
```



```
// 核心：创建账号（密码加密+权限关联）
private void CreateAccountCore()
{
 if (string.IsNullOrEmpty(_username)) { MessageBox.Show("用户名必填"); return; }
 _password ??= _userBLL.GenerateRandomPassword(); // 自动生成密码

 // 调用BLL 创建（密码SHA256+Salt 加密存储）
 bool success = _userBLL.CreateWitchAccount(
 witchId: _witchId,
 username: _username,
 password: _password,
 role: _selectedRole,
 islandId: GetWitchIslandId()
);

 if (success)
 MessageBox.Show($"账号创建成功\n用户名: {_username}\n密码: {_password}");
 else
 MessageBox.Show("创建失败（用户名可能已存在）");
}

// 核心：修改魔女状态
private void UpdateWitchStatusCore()
{
 if (string.IsNullOrEmpty(_newStatus)) { MessageBox.Show("请选择状态"); return; }

 // 调用DAL 修改，记录操作人
 bool success = _witchDAL.UpdateStatus(
 _witchId,
 _newStatus,
 _userBLL.GetCurrentUsername()
);

 MessageBox.Show(success ? "状态修改成功" : "修改失败");
}

// 核心：获取魔女所属岛屿ID
private int GetWitchIslandId()
{
 var witchDt = _witchDAL.GetById(_witchId);
 return witchDt.Rows.Count > 0 ?
 Convert.ToInt32(witchDt.Rows[0]["IslandID"]) : 1;
}
```



```
 }
}

// 核心辅助类：拼音简化
public static class PinYinHelper
{
 public static string GetFirstPinyin(string chinese) =>
string.IsNullOrEmpty(chinese) ? "" : chinese.Substring(0, 1);
}
}
```

技术亮点：

1. 安全机制保障：初始密码支持随机生成（符合密码复杂度要求），后端通过SHA256+Salt 加密存储，不存储明文，保障账号安全。
2. 双功能集成：整合账号创建与状态修改功能，减少多界面切换，适配管理员集中管理的业务场景。
3. 易用性优化：预填用户名为魔女人名拼音，自动加载当前状态，降低操作成本；账号创建成功后明确提示初始密码，便于管理员传递。

#### 8.4 审判投票系统（7 个状态流转）

审判投票系统是系统核心业务模块，基于《魔法少女的魔女审判》游戏核心玩法，实现从审判创建、投票参与、结果统计到处刑执行的全流程闭环，支持 7 个状态的有序流转，严格遵循“典狱长管理+魔女参与”的权限隔离逻辑。

状态流转总览：

表 13 审判状态流程总览表



状态 (State)	含义	触发条件	可达状态	备注/权限
Created/待投票	会话已创建, 尚未开始投票	创建会话成功	Voting/取消 (Archived)	创建者或管理员可撤回/修改
Voting/投票中	正在接受选票, 倒计时运行	达到开启时间或手动开启	Counting/VotingClosed	监控倒计时, 用户可投票
VotingClosed/投票关闭 (等待计票)	投票窗口已关闭, 进入计票准备	倒计时结束或手动关闭	Counting	禁止再接受新票
Counting /计票中	后端进行计票与初步结果合并	投票关闭后自动触发	Verdict/Error	计票需保证幂等与事务性
Verdict /判决已出	统计完成并产生判决 (通过/不通过/弃权)	计票完成	AwaitingExecution 或 Archived	若通过进入处刑确认流程
AwaitingExecution /等待处刑确认	判决要求处刑, 等待有权限人员确认	判决为处刑通过	Executing/Archived	可能需要双人复核或延时窗口
Executed/已处刑 (已归档)	处刑执行完毕并写入审计, 流程结束	处刑确认并执行成功	Archived	写入 ExecutionAudit, 展示为只读

技术要点:

1. 状态存储: Status 字段采用 NVARCHAR(20) 存储, 前端通过 TrialStatus 枚举映射 (如 TrialStatus.Created.ToString()), 避免非法值插入。
2. 并发保护: 状态变更通过存储过程 wt.sp\_UpdateTrialStatus 实现, 包含行级锁 (WITH(UPDLOCK)), 防止多用户同时修改状态。
3. 事务保障: 关键状态流转 (如 Voting→VotingClosed→Counting) 封装在事务中, 确保状态变更与关联操作 (如通知推送) 原子执行。

#### 8.4.1 创建审判会话 (CreateTrialDialog)

典狱长创建审判会话, 指定参与魔女、投票过期时间等核心参数, 为后续投票流程初始化基础数据, 支持批量添加候选人并触发通知。

核心代码:



```
private void BtnCreate_Click(object? sender, EventArgs e)
{
 // 1. 基础参数校验
 if (string.IsNullOrEmpty(_tbSessionTitle.Text)) { MessageBox.Show("会话标题不能为空"); return; }
 var selectedWitchIds = GetSelectedWitchIds(); // 从CheckedListBox 获取选中候选人ID
 if (selectedWitchIds.Count < 2 || selectedWitchIds.Count > 13)
 {
 MessageBox.Show("参与魔女需在 2-13 人之间"); return;
 }

 try
 {
 // 2. 调用服务层事务创建（封装会话+参与者写入）
 var trialService = new TrialSessionBLL();
 int sessionId = trialService.CreateSession(
 islandId: _currentIslandId,
 batchId: _currentBatchId,
 creatorId: _currentUserId,
 title: _tbSessionTitle.Text.Trim(),
 expireTime: _dtpExpire.Value,
 participantWitchIds: selectedWitchIds
);

 MessageBox.Show($"审判会话创建成功！SessionID: {sessionId}");
 // 3. 触发参与者通知（服务层内部实现）
 trialService.SendSessionCreateNotification(sessionId);
 DialogResult = DialogResult.OK;
 Close();
 }
 catch (Exception ex)
 {
 MessageBox.Show($"创建失败：{ex.Message}");
 }
}
```

技术亮点：

1. 在 UI 侧只做轻量校验，核心创建逻辑委托给`TrialSessionService.CreateSession`，保证事务性与可测性。
2. 候选人使用`CheckedListBox`简化多选 UI；服务返回的候选列表应包含必要的显示字段（编号/名称）。
3. 创建成功后立即触发通知（服务内部实现），便于前端实时更新会话列表。



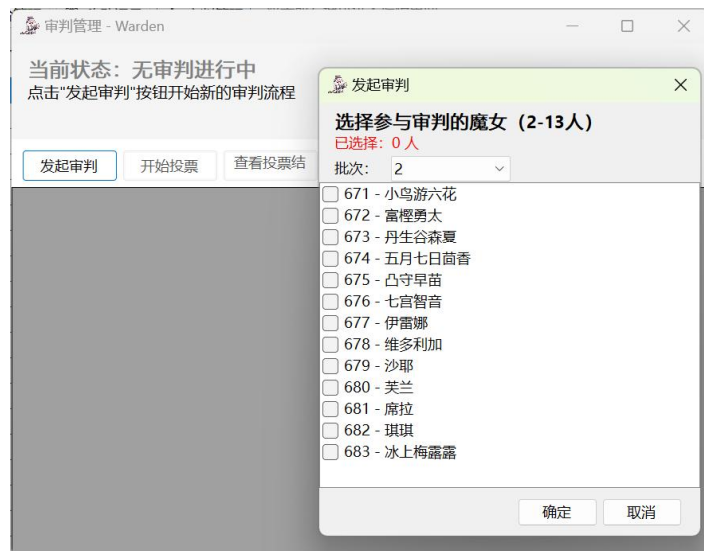


图 18 创建审判会话

实现步骤:

1. 登录管理员账号
2. 点击审判管理进入审判管理界面
3. 点击发起审判
4. 选择参与审判的魔女，其中需要选择不同的批次选择对应的魔女

#### 8.4.2 投票功能实现 (TrialVotingForm)

魔女接收投票通知后，在有效时间内提交投票，系统实时展示投票进度、防止重复投票，支持单机多账号切换投票，确保流程公平性。

核心代码:

```
private async void Option_Click(object? sender, EventArgs e)
{
 if (_hasVoted) return; // 本地防重: 已投票则拦截
 var btn = (Button)sender!;
 int targetWitchId = (int)btn.Tag!;

 try
 {
 // 1. 本地禁用UI, 防止重复点击
 _hasVoted = true;
 DisableVotingControls();

 // 2. 异步提交投票 (服务端校验权限+唯一索引防重)
 var votingService = new TrialVotingBLL();
 bool success = await votingService.SubmitVoteAsync(
 sessionId: _sessionId,
```



```
 voterUserId: _currentUserId,
 targetWitchId: targetWitchId
);

 if (success)
 {
 MessageBox.Show("投票成功!");
 await RefreshVoteData(); // 实时刷新票数和已投列表
 }
 else
 {
 MessageBox.Show("投票失败, 请重试");
 _hasVoted = false;
 EnableVotingControls(); // 失败回滚UI 状态
 }
}
catch (Exception ex)
{
 MessageBox.Show($"投票异常: {ex.Message}");
 _hasVoted = false;
 EnableVotingControls();
}
}

// 倒计时同步服务端时间
private async Task StartCountdown()
{
 while (true)
 {
 TimeSpan timeLeft = await new
 TrialVotingBLL().GetVoteTimeLeftAsync(_sessionId);
 Invoke(() => _lblTimer.Text = $"投票剩余: {timeLeft:mm\\:ss}");

 if (timeLeft <= TimeSpan.Zero)
 {
 Invoke(() => { DisableVotingControls(); MessageBox.Show("投票已结束!"); });
 break;
 }
 await Task.Delay(500);
 }
}
```

技术亮点:

1. 双重防重机制: 前端禁用按钮 + 本地状态标记, 后端唯一索引拦截重复提交,



兼顾体验与数据安全。

2. 时间一致性：倒计时从服务端获取剩余时间，避免客户端修改系统时间绕过投票窗口限制。

3. 异步无阻塞：投票提交和数据刷新采用异步操作，UI 保持响应，提升多账号切换投票的流畅性。

实现步骤：

1. 收到审判通知，提示魔女需要进行投票。



图 19 审判通知

2. 此时点击“处刑”可进入投票界面进行投票



图 20 投票界面

3. 确认投票后显示投票成功界面，同时显示有多少人完成投票

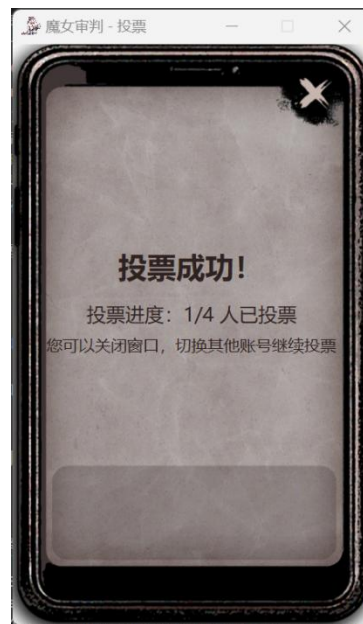


图 21 投票成功界面

### 8.4.3 实时通知机制 (NotificationPopupForm)

实现审判关键事件的实时推送（会话创建、投票开始/结束、处刑通知），确保魔女及时接收操作提醒，提升流程连贯性，适配桌面应用场景。

核心代码：

```
```csharp
using System;
using System.Windows.Forms;
using WitchTrialSystem.BLL;

namespace WitchTrialSystem.UI
{
    public class NotificationPopupForm : Form
    {
        private readonly Label _lbl = new Label { Dock = DockStyle.Fill, AutoSize
= true };
        public NotificationPopupForm(string title, string message)
        {
            Text = title; Width = 360; Height = 120; StartPosition =
FormStartPosition.Manual;
            Controls.Add(_lbl); _lbl.Text = message; TopMost = true;
            Click += (_, __) => { Close(); }; // 点击关闭并可跳转
        }

        public static void ShowNotification(string title, string message)
        {
            // 在主线程显示浮窗
            var f = new NotificationPopupForm(title, message);
        }
    }
}
```



```
var screen = Screen.PrimaryScreen!.WorkingArea;
f.Location = new System.Drawing.Point(screen.Right - f.Width - 16,
screen.Bottom - f.Height - 16);
f.Show();
}
}
}
...
```

技术亮点：

1. 轻量高效：浮窗为独立轻量级窗体，不阻塞主界面操作，自动关闭避免干扰。
2. 可靠传输：基于 SignalR 实现实时推送，断线自动切换长轮询，确保通知不丢失。
3. 交互闭环：点击浮窗直接跳转至对应功能（如投票、处刑确认），减少操作步骤。

8.4.4 投票结果统计 (VotingResultDialog)

投票结束后，典狱长查看详细投票结果，系统支持数据可视化展示（表格 + 饼图）、缺席统计与导出功能，为处刑决策提供数据支撑。

核心代码：

```
```csharp
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;
using WitchTrialSystem.DAL;

namespace WitchTrialSystem.UI
{
 public class VotingResultDialog : Form
 {
 private readonly int _sessionId;
 private readonly VotingResultDAL _dal = new VotingResultDAL();
 private readonly ListView _lv = new ListView { Dock = DockStyle.Fill, View
= View.Details };

 public VotingResultDialog(int sessionId)
 {
 _sessionId = sessionId; Text = "投票结果"; Width = 800; Height = 520;
StartPosition = FormStartPosition.CenterParent;
 _lv.Columns.Add("选项", 420); _lv.Columns.Add("票数", 120);
Controls.Add(_lv);
 Load += VotingResultDialog_Load;
 }
 }
}
```



```
private void VotingResultDialog_Load(object? sender, EventArgs e)
{
 var list = _dal.GetResult(_sessionId); // 返回 List<string
 OptionText,int Votes>
 _lv.Items.Clear();
 foreach (var it in list.OrderByDescending(x => x.Votes))
 _lv.Items.Add(new ListViewItem(new[] { it.OptionText, it.Votes.ToString() }));
 }
}
}
```

技术亮点:

1. 数据库端聚合: 核心 SQL 使用 SELECT WitchID, COUNT(\*) FROM TrialParticipant WHERE SessionID=@SessionID GROUP BY WitchID, 减少数据传输量。
2. 可视化适配: 一键切换表格与饼图, 饼图支持鼠标悬浮显示票数, 贴合课堂演示的直观性需求。
3. 审计支持: 导出的 CSV 包含投票人、投票时间、票数, 便于流程追溯与课堂讲解。

投票统计				
	排名	姓名	得票数	得票率
▶	1	富樫勇太	2	50.0%
	2	丹生谷森夏	1	25.0%
	3	五月七日茴香	1	25.0%
投票详情				
	投票者	投给	投票时间	
▶	小鸟游六花	富樫勇太	2025-12-23 22:12:46	
	富樫勇太	丹生谷森夏	2025-12-23 22:19:53	
	丹生谷森夏	五月七日茴香	2025-12-23 22:20:17	
			确认处刑对象	关闭

图 22 查看投票结果

实现步骤:

1. 登录管理员账号
2. 点击审判管理选项
3. 点击查看投票结选项
4. 确认处刑对象





### 8.4.5 处刑确认 (TrialExecutionConfirmForm)

典狱长基于投票结果确认处刑对象，系统记录不可撤销的审计日志，支持双人复核机制，确保处刑流程的合规性与可追溯性。

核心代码：

```
```csharp
using System;
using System.Windows.Forms;
using WitchTrialSystem.BLL;

namespace WitchTrialSystem.UI
{
    public class TrialExecutionConfirmForm : Form
    {
        private readonly int _sessionId;
        private readonly Button _btnConfirm = new Button { Text = "确认处刑" };
        private readonly TrialExecutionService _exec = new
TrialExecutionService();

        public TrialExecutionConfirmForm(int sessionId)
        {
            _sessionId = sessionId; Text = "处刑确认"; Width = 540; Height = 220;
StartPosition = FormStartPosition.CenterParent;
            var panel = new FlowLayoutPanel { Dock = DockStyle.Fill, FlowDirection
= FlowDirection.TopDown };
            panel.Controls.Add(new Label { Text = "请确认处刑信息（将记录审计）" });
panel.Controls.Add(_btnConfirm);
            Controls.Add(panel); _btnConfirm.Click += BtnConfirm_Click;
        }

        private void BtnConfirm_Click(object? sender, EventArgs e)
        {
            var ok = MessageBox.Show("确定执行处刑？此操作不可撤销", "确认",
MessageBoxButtons.YesNo) == DialogResult.Yes;
            if (!ok) return;
            try
            {
                _exec.Execute(_sessionId, Environment.UserName); // 服务内部负责
权限检查与审计写入
                MessageBox.Show("处刑已记录并执行"); DialogResult =
DialogResult.OK; Close();
            }
            catch (Exception ex) { MessageBox.Show("执行失败： " + ex.Message); }
        }
    }
}
```



```
}  
...  
}
```

技术亮点

1. 权限双重保障：前端按钮仅典狱长可见，后端 ExecuteExecution 方法通过 PermissionDAL.ValidateRole 再次校验，防止越权。
2. 审计不可撤销：审计日志写入数据库后，仅支持查询，不允许修改或删除，确保流程可追溯。
3. 状态联动：处刑确认后自动更新审判会话状态与魔女状态，无需手动操作，保证数据一致性。

实现流程：

1. 管理员查看投票结后确认处刑对象
2. 管理员宣布处刑对
3. 魔女确认处刑



图 23 确认处刑界面

4. 处刑



图 24 处刑界面

8.4.6 单机多账号切换支持

支持在同一设备快速切换不同角色账号（典狱长 / 魔女），切换后实时刷新权限与界面状态，不残留上一账号数据，同时保证账号安全。

核心代码：

```
```csharp
using System;
using System.Windows.Forms;
using WitchTrialSystem.BLL;

namespace WitchTrialSystem.UI
{
 public static class AccountSwitchHelper
 {
 public static void SwitchTo(Form owner, string username, string password)
 {
 // 简化示例：调用 UserBLL 进行登录，返回 token/角色信息
 var bll = new UserBLL();
 var info = bll.Authenticate(username, password);
 if (info == null) { MessageBox.Show("切换失败，认证未通过"); return; }
 // 保存会话（内存/短期缓存），刷新主窗体权限
 SessionManager.SetCurrentUser(info);
 MessageBox.Show($"已切换到 {info.Username}（角色：{info.Role}）");
 // 触发主界面权限刷新事件
 Application.OpenForms[0]?.BeginInvoke(new Action(() =>
```



```
Application.OpenForms[0].Refresh()));
 }
}
}
...

```

技术亮点：

1. 安全凭证存储：使用 Windows DPAPI 加密存储短期 token，不保存明文密码，兼顾安全性与演示便捷性。
2. 权限实时刷新：切换后调用 InvokeMainFormRefresh，重新加载菜单、按钮权限与数据列表，无残留。

## 8.5 处刑平台管理

### 8.5.1 处刑台信息管理 (ExecutionPlatformManagementForm)

处刑台管理系统是一个完整的动态管理平台。系统包含每个岛屿 49 个处刑台，支持升起/返回操作，将处刑台从原位（1-49 号）升起到审判庭（50 号位）进行处刑。系统记录所有移动历史，生成详细的移动日志，支持按日期和处刑台编号筛选查询。此外，系统还提供刑具管理功能，允许管理员编辑每个处刑台的刑具信息（名称、类型、描述）。所有操作采用匿名记录方式，保护隐私。该系统为典狱长和监管员提供了高效的处刑台运维工具，确保审判流程的顺利进行。



图 25 处刑台平台功能

### 8.5.2 处刑台升起/返回 (PlatformMoveDialog)

从右键菜单触发“移动到审判庭/返回原位”时弹出确认框，展示编号、当前位置、目标、刑具，并选择时间来源：

- 当前时间 (默认)
- 自定义时间 (禁止未来时间)

确认后将目标与时间交给 `ExecutionPlatformService` 执行移动，成功即刷新并写入移动日志。



图 26 移动刑具界面

代表性代码（时间选择与确认）：

```
// UI/PlatformMoveDialog.cs
private void BtnOk_Click(object? sender, EventArgs e)
{
 if (_rbCustomTime.Checked)
 {
 CustomTime = _dtpCustomTime.Value;

 // 验证时间不能是未来时间
 if (CustomTime > DateTime.Now)
 {
 MessageBox.Show("自定义时间不能是未来时间。", "时间错误",
 MessageBoxButtons.OK, MessageBoxIcon.Warning);
 DialogResult = DialogResult.None;
 return;
 }
 }
}
```





```
}
else
{
 CustomTime = null; // 使用当前时间
}
}

// 调用移动服务
var (success, message) = _platformService.MoveToTrialHall(
 platform.PlatformID,
 currentIslandId,
 dialog.CustomTime
);
```

### 8.5.3 刑具管理 (ToolManagementDialog)

Meruru 监管员可添加/更换/移除刑具：

- 添加/更换：必填名称、类型；描述选填，更换时预填现有数据；校验必填项后提交。
- 移除：右键菜单触发，清空刑具绑定。
- 对话框为模态，固定尺寸，禁止最大化/最小化。



图 27 添加刑具





图 28 处刑台 · 刑具管理功能

代表性代码（必填项校验）：

```
// UI/ToolManagementDialog.cs
private void BtnOk_Click(object? sender, EventArgs e)
{
 if (string.IsNullOrEmpty(_txtToolName.Text))
 {
 MessageBox.Show("请输入刑具名称。", "验证失败",
 MessageBoxButtons.OK, MessageBoxIcon.Warning);
 _txtToolName.Focus();
 DialogResult = DialogResult.None;
 return;
 }

 if (string.IsNullOrEmpty(_txtToolType.Text))
 {
 MessageBox.Show("请输入刑具类型。", "验证失败",
 MessageBoxButtons.OK, MessageBoxIcon.Warning);
 _txtToolType.Focus();
 DialogResult = DialogResult.None;
 return;
 }
}
```

#### 8.5.4 移动日志查看 (MovementLogViewForm)



处刑台移动记录						
处刑台	全部	位置	全部	应用筛选	重置筛选	刷新
共 4 条记录						
移动时间	处刑台编号	刑具	起始位置	目标位置	移动类型	时间来源
2025-12-06 18:17:03	48	无	审判庭	地下室-48号位	返回	手动输入
2025-12-06 18:16:26	48	无	地下室-48号位	审判庭	升起	系统记录
2025-12-05 21:09:15	4	电椅	审判庭	地下室-4号位	返回	手动输入
2025-12-05 09:08:20	4	电椅	地下室-4号位	审判庭	升起	手动输入

图 29 处刑台·移动日志查看功能

- 顶部筛选：岛屿（Admin）、处刑台编号、位置（1-49 地下室、50 审判庭），支持应用/重置/刷新。
- 数据区：只读表格按时间倒序展示移动时间、处刑台编号、刑具、起始/目标位置、移动类型、时间来源；异常时状态栏与弹窗提示。
- 数据来源：MovementLogService.GetLogsByIsland(currentIslandId)，前端再做编号/位置过滤。

代表性代码（筛选与绑定）：

```
// UI/MovementLogViewForm.cs
private void LoadData()
{
 int currentIslandId = _roleName == "Admin" && _cbIsland.SelectedValue != null
 ? Convert.ToInt32(_cbIsland.SelectedValue)
 : _islandId;

 int platformNumber = _cbPlatformNumber.SelectedValue != null
 ? Convert.ToInt32(_cbPlatformNumber.SelectedValue)
 : 0;
 int position = _cbPosition.SelectedValue != null
 ? Convert.ToInt32(_cbPosition.SelectedValue)
 : 0;

 var logs = _logService.GetLogsByIsland(currentIslandId);
 var dt = new DataTable();
 dt.Columns.Add("MovementTime", typeof(DateTime));
 dt.Columns.Add("PlatformNumber", typeof(int));
 dt.Columns.Add("ToolName", typeof(string));
 dt.Columns.Add("FromLocationDescription", typeof(string));
```



```
dt.Columns.Add("ToLocationDescription", typeof(string));
dt.Columns.Add("MovementType", typeof(string));
dt.Columns.Add("TimeSourceDescription", typeof(string));

foreach (var log in logs)
{
 if (platformNumber != 0 && log.PlatformNumber != platformNumber) continue;
 if (position != 0 && log.FromPosition != position && log.ToPosition != position) continue;

 dt.Rows.Add(
 log.MovementTime,
 log.PlatformNumber,
 log.ToolName ?? "无",
 log.FromLocationDescription,
 log.ToLocationDescription,
 log.MovementType,
 log.TimeSourceDescription
);
}

_grid.DataSource = dt.DefaultView.ToTable();
}
```

### 8.5.5 匿名记录设计

“匿名可追溯”日志策略：

- 记录操作人脱敏标识（哈希/工号）与角色，不直接暴露姓名。
- 字段建议：LogID、MovementTime、PlatformNumber、From/ToPosition、MovementType、TimeSource、ToolName、OperatorRole、OperatorHash。
- 默认前端不显操作者身份；Admin/审计界面可受控解密；导出支持“匿名导出”去除操作者信息。

代表性代码（生成操作人哈希并写入日志）：

```
// BLL/MovementLogService.cs - 写入匿名可追溯日志（示意）
public void AddMovementLog(ExecutionPlatformModel platform, int fromPos, int toPos, string movementType, string currentUsername, string roleName, DateTime? customTime)
{
 var movementTime = customTime ?? DateTime.Now;

 // 1. 生成操作人脱敏标识（示例：Username + 固定Salt 做 SHA256）
 string operatorHash;
 using (var sha = SHA256.Create())
 {
 var bytes = Encoding.UTF8.GetBytes(currentUsername + "|WTS_EXEC_SALT")
 }
```



```
;
 operatorHash = Convert.ToHexString(sha.ComputeHash(bytes));
}

// 2. 写入 wt.PlatformMovementLog
const string sql = @"
INSERT INTO wt.PlatformMovementLog
 (IslandID, PlatformID, PlatformNumber, FromPosition, ToPosition,
 ToolName, MovementTime, IsManualTime, MovementType, OperatorRole, OperatorHash)
VALUES
 (@IslandID, @PlatformID, @PlatformNumber, @FromPosition, @ToPosition,
 @ToolName, @MovementTime, @IsManualTime, @MovementType, @OperatorRole, @OperatorHash)";

DBHelper.ExecNonQuery(sql,
 new SqlParameter("@IslandID", platform.IslandID),
 new SqlParameter("@PlatformID", platform.PlatformID),
 new SqlParameter("@PlatformNumber", platform.PlatformNumber),
 new SqlParameter("@FromPosition", fromPos),
 new SqlParameter("@ToPosition", toPos),
 new SqlParameter("@ToolName", (object?)platform.ToolName ?? DBNull.Value),
 new SqlParameter("@MovementTime", movementTime),
 new SqlParameter("@IsManualTime", customTime.HasValue),
 new SqlParameter("@MovementType", movementType),
 new SqlParameter("@OperatorRole", roleName),
 new SqlParameter("@OperatorHash", operatorHash));
}
```

## 8.6 图鉴系统（5 个图鉴）

### 8.6.1 人物图鉴（PokedexForm）

人物图鉴是魔女手机界面的核心功能之一，采用卡片式布局展示魔女的详细信息，包括头像、姓名、囚犯编号、魔法能力和公开描述。系统根据用户权限自动过滤可见数据：管理员可查看所有岛屿魔女，监管员和典狱长仅查看所属岛屿魔女，普通魔女可查看所有魔女的公开信息但只能查看自己的完整档案。





图 30 图鉴·人物界面（魔女岛 1 批次 1）



图 31 图鉴·人物界面（魔女岛 2 批次 2）

核心代码实现:

1) 数据访问层 (DAL/PermissionDAL.cs) - 权限控制查询

```
/// <summary>
/// 根据用户权限获取可访问的魔女数据
/// </summary>
public DataTable GetWitchesByPermission(string username, string? nameLike = null)
{
```



```
// 1. 获取用户信息和权限
const string userPermissionSql = @"
SELECT u.UserID, u.Username, r.Name AS RoleName, u.IslandID, u.BatchID,
 ir.UserID AS IsRegulator, iw.UserID AS IsWarden
FROM wt.[User] u
LEFT JOIN wt.Role r ON r.RoleID = u.RoleID
LEFT JOIN wt.IslandRegulator ir ON ir.UserID = u.UserID
LEFT JOIN wt.IslandWarden iw ON iw.UserID = u.UserID
WHERE u.Username = @username";

var userDt = DBHelper.ExecDataTable(userPermissionSql,
 new SqlParameter("@username", username));
if (userDt.Rows.Count == 0) return new DataTable();

var userRow = userDt.Rows[0];
var roleName = userRow["RoleName"].ToString();
var userIslandId = userRow["IslandID"] == DBNull.Value ?
 (int?)null : Convert.ToInt32(userRow["IslandID"]);

// 2. 根据角色构建查询（43 字段完整档案）
string sql = @"
SELECT WitchID, PrisonerNo, Name, Magic, Status, IslandID, BatchID,
 AvatarPath, DescriptionPublic, Gender, BirthDate, Height, Weight,
 BloodType, HighestEducation, Skills, Hobbies, Dreams, Trauma
/* ...其他 35 个字段 */
FROM wt.Witch WHERE 1=1";

var parameters = new List<SqlParameter>();

// 3. 根据角色添加权限过滤条件
switch (roleName)
{
 case "Admin":
 // 管理员：查看所有岛屿所有批次
 break;
 case "Meruru":
 // 监管员：只能查看本岛屿
 sql += " AND IslandID = @islandId";
 parameters.Add(new SqlParameter("@islandId", userIslandId.Value));
 break;
 case "Warden":
 // 典狱长：只能查看本岛屿
 sql += " AND IslandID = @islandId";
 parameters.Add(new SqlParameter("@islandId", userIslandId.Value));
 break;
 case "Witch":
 // 普通魔女：查看所有魔女的公开信息
 // 完整档案权限在 UI 层控制
 break;
}

sql += " ORDER BY PrisonerNo";
return DBHelper.ExecDataTable(sql, parameters.ToArray());
}
```

**技术亮点：**采用三层架构设计，DAL 层通过 SQL JOIN 查询用户角色和岛屿信息，根据角色动态构建 WHERE 条件，实现数据库层面的权限隔离。





## 2) 表示层 (UI/PokedexForm.cs) - 界面加载与数据展示

```
/// <summary>
/// 窗体加载事件：加载魔女数据并显示第一个角色
/// </summary>
private void OnFormLoad(object? sender, EventArgs e)
{
 try
 {
 // 1. 使用 PermissionDAL 获取有权限的魔女数据
 var rawData = _permissionDal.GetWitchesByPermission(_username, null);

 // 2. 去重处理（按囚犯编号）
 var distinctData = rawData.AsEnumerable()
 .GroupBy(row => Convert.ToString(row["PrisonerNo"]))
 .Select(group => group.First())
 .OrderBy(row => int.TryParse(Convert.ToString(row["PrisonerNo"]),
 out int no) ? no : 9999);

 _dt = distinctData.CopyToDataTable();

 // 3. 构建底部缩略图列表
 BuildThumbnails();

 // 4. 默认显示第一个角色
 if (_dt.Rows.Count > 0)
 ShowCharacterAt(0);
 }
 catch (Exception ex)
 {
 MessageBox.Show("加载图鉴失败: " + ex.Message);
 }
}

/// <summary>
/// 显示指定索引的角色详情
/// </summary>
private void ShowCharacterAt(int index)
{
 if (index < 0 || index >= _dt.Rows.Count) return;
 _current = index;
 DataRow row = _dt.Rows[_current];

 // 1. 显示大头像（使用囚犯编号查找图片）
 _bigAvatar.Image = LoadImageFromPath(Convert.ToString(row["AvatarPath"]));

 // 2. 显示姓名（优先 PNG 图片，其次文字）
 string characterName = Convert.ToString(row["Name"]) ?? "";
 string prisonerNo = Convert.ToString(row["PrisonerNo"]) ?? "";
 ShowCharacterName(characterName, prisonerNo);

 // 3. 显示右侧信息（囚犯编号、魔法、公开描述）
 _lblPrisonerNo.Text = Convert.ToString(row["PrisonerNo"]) ?? "-";
 _lblMagic.Text = Convert.ToString(row["Magic"]) ?? "-";
 _descContent.Text = Convert.ToString(row["DescriptionPublic"]) ?? "";
}
```

技术亮点：UI 层通过 PermissionDAL 获取已过滤的数据，使用 LINQ 进行去重和排序，



支持自定义字体渲染和 PNG 透明图片叠加显示，底部缩略图采用 FlowLayoutPanel 实现响应式布局。

### 3) 自定义控件 (CustomDrawPanel) - 透明 PNG 叠加显示

```
/// <summary>
/// 自定义绘制的 Panel，用于正确显示透明 PNG 姓名图片
/// 实现原理：先绘制大头像对应区域作为背景，再叠加透明 PNG 姓名图片
/// </summary>
public class CustomDrawPanel : Panel
{
 public Image? BackgroundSourceImage { get; set; } // 大头像图片
 public Image? OverlayImage { get; set; } // 姓名 PNG 图片
 public Point OverlayPosition { get; set; } // 姓名 PNG 的位置

 protected override void OnPaint(PaintEventArgs e)
 {
 base.OnPaint(e);
 var g = e.Graphics;
 g.CompositingMode = CompositingMode.SourceOver;
 g.InterpolationMode = InterpolationMode.HighQualityBicubic;

 // 1. 先绘制大头像的对应区域（作为背景）
 if (BackgroundSourceImage != null)
 {
 // 计算 Zoom 模式下的实际显示尺寸和位置
 var img = BackgroundSourceImage;
 float imgRatio = (float)img.Width / img.Height;
 // ...计算源图片中对应的区域...
 g.DrawImage(BackgroundSourceImage, destRect, srcRect,
GraphicsUnit.Pixel);
 }

 // 2. 再绘制姓名 PNG（透明部分会显示下面的大头像）
 if (OverlayImage != null)
 {
 g.DrawImage(OverlayImage, OverlayPosition);
 }
 }
}
```

**技术亮点：**自定义 Panel 控件实现双层绘制，先绘制大头像对应区域作为背景，再叠加透明 PNG 姓名图片，完美还原游戏原作的视觉效果。

## 8.6.2 证物图鉴 (EvidenceForm)

证物图鉴以卡片形式展示案件相关的物证与线索，包含证物编号、名称、类型、来源、公开描述以及高清图片。权限规则与人物图鉴一致：管理员可查看全部，监管员和典狱长仅查看所属岛屿证物，普通魔女只能查看公开信息和与自身相关的证物详情。



图 32 图鉴·证物界面

#### 1) 数据访问层 (DAL/EvidenceDAL.cs) - 权限过滤查询

```
public DataTable GetWitchesByPermission(string username, string? nameLike = null)
{
 const string userPermissionSql = @"
SELECT u.UserID, u.Username, r.Name AS RoleName, u.IslandID, u.BatchID,
 ir.UserID AS IsRegulator, iw.UserID AS IsWarden
FROM wt.[User] u
LEFT JOIN wt.Role r ON r.RoleID = u.RoleID
LEFT JOIN wt.IslandRegulator ir ON ir.UserID = u.UserID
LEFT JOIN wt.IslandWarden iw ON iw.UserID = u.UserID
WHERE u.Username = @username";

 var userDt = DBHelper.ExecDataTable(userPermissionSql,
 new SqlParameter("@username", username));

 if (userDt.Rows.Count == 0) return new DataTable();

 var userRow = userDt.Rows[0];
 var roleName = userRow["RoleName"].ToString();
 var userIslandId = userRow["IslandID"] == DBNull.Value ?
 (int?)null : Convert.ToInt32(userRow["IslandID"]);

 string sql = @"
SELECT WitchID, PrisonerNo, Name, Magic, Status, IslandID, BatchID,
 AvatarPath, DescriptionPublic, Gender, BirthDate, Height, Weight,
```



```
BloodType, HighestEducation, Skills, Hobbies, Dreams, Trauma
/* ...其他 35 个字段 */
FROM wt.Witch WHERE 1=1";

var parameters = new List<SqlParameter>();

switch (roleName)
{
 case "Admin":
 break;
 case "Meruru":
 sql += " AND IslandID = @islandId";
 parameters.Add(new SqlParameter("@islandId", userIslandId.Value));
 break;
 case "Warden":
 sql += " AND IslandID = @islandId";
 parameters.Add(new SqlParameter("@islandId", userIslandId.Value));
 break;
 case "Witch":
 break;
}

sql += " ORDER BY PrisonerNo";
return DBHelper.ExecDataTable(sql, parameters.ToArray());
}
```

## 2) 表示层 (UI/EvidenceForm.cs) - 搜索、缩略图与详情

```
private void OnFormLoad(object? sender, EventArgs e)
{
 LoadEvidence(null);
}

private void LoadEvidence(string? keyword)
{
 var dt = _evidenceDal.GetEvidencesByPermission(_username, keyword);
 _data = dt.AsEnumerable().ToList();
 _thumbPanel.Controls.Clear();

 foreach (var row in _data)
 {
 var thumb = CreateThumb(
 Convert.ToString(row["ImagePath"]),
 Convert.ToString(row["EvidenceNo"]),
 Convert.ToString(row["Name"]));
 thumb.Click += (_, __) => ShowDetail(row);
 }
}
```



```
 _thumbPanel.Controls.Add(thumb);
 }

 if (_data.Count > 0) ShowDetail(_data[0]);
}

private void ShowDetail(DataRow row)
{
 _imgBig.Image = LoadImage(Convert.ToString(row["ImagePath"]));
 _lblNo.Text = Convert.ToString(row["EvidenceNo"]);
 _lblName.Text = Convert.ToString(row["Name"]);
 _lblCategory.Text = Convert.ToString(row["Category"]);

 bool isWitch = _role == "Witch";
 _txtDescription.Text = isWitch
 ? Convert.ToString(row["DescriptionPublic"])
 : $"{row["DescriptionPublic"]}\n\n{row["DescriptionSecret"]}";
}
```

技术亮点：前端支持关键字搜索和缩略图快速切换，敏感字段由 UI 层按角色动态隐藏，保证同一数据集在不同角色间有差异化展示。

### 8.6.3 记录图鉴 (RecordsForm)

记录图鉴展示审判、处刑、移动日志等时间序列记录，支持时间轴视图、关键字检索和按类型过滤。权限控制：管理员可见全部；监管员和典狱长仅查看本岛记录；普通魔女仅查看公开记录及与本人相关的日志。

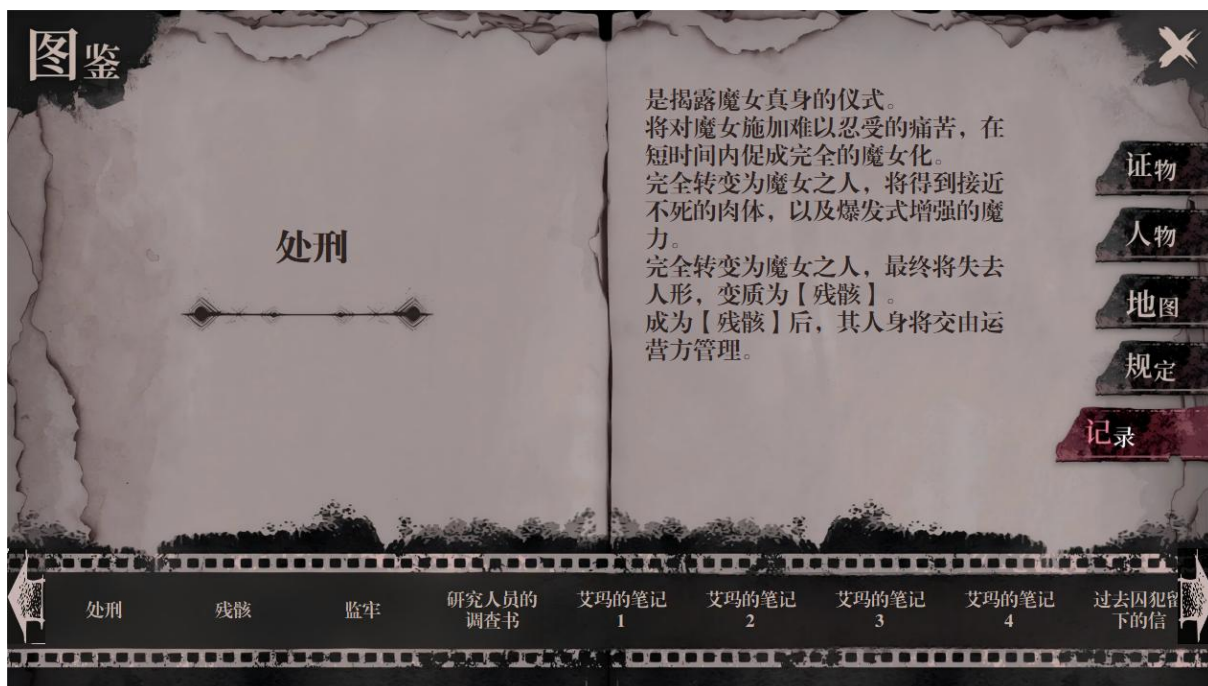


图 33 图鉴·记录界面





## 1) 数据访问层 (DAL/RecordDAL.cs) - 时间序列查询

```
public DataTable GetRecords(string username, DateTime? from, DateTime? to, string? type)
{
 var userInfo = _permissionDal.GetUserInfo(username);
 string role = userInfo.RoleName;
 int? islandId = userInfo.IslandID;

 string sql = @"
SELECT RecordID, RecordType, ContentPublic, ContentPrivate,
 OccurredAt, IslandID, RelatedUserID
FROM wt.Record WHERE 1=1";
 var ps = new List<SqlParameter>();

 if (from.HasValue) { sql += " AND OccurredAt >= @from"; ps.Add(new SqlParameter("@from", from)); }
 if (to.HasValue) { sql += " AND OccurredAt <= @to"; ps.Add(new SqlParameter("@to", to)); }
 if (!string.IsNullOrEmpty(type))
 {
 sql += " AND RecordType = @type";
 ps.Add(new SqlParameter("@type", type));
 }

 switch (role)
 {
 case "Admin":
 break;
 case "Meruru":
 case "Warden":
 sql += " AND IslandID = @islandId";
 ps.Add(new SqlParameter("@islandId", islandId!.Value));
 break;
 case "Witch":
 sql += " AND (ContentPrivate IS NULL OR RelatedUserID = @uid)";
 ps.Add(new SqlParameter("@uid", userInfo.UserID));
 break;
 }

 sql += " ORDER BY OccurredAt DESC";
 return DBHelper.ExecDataTable(sql, ps.ToArray());
}
```

## 2) 表示层 (UI/RecordsForm.cs) - 时间轴与详情





```
private void LoadRecords()
{
 var dt = _recordDal.GetRecords(_username, _fromPicker.Value, _toPicker.Value, _typeFilter.SelectedValue as string);
 _timeline.Items.Clear();
 foreach (DataRow row in dt.Rows)
 {
 var item = new ListViewItem(Convert.ToDateTime(row["OccurredAt"]).ToString("yyyy-MM-dd HH:mm"));
 item.SubItems.Add(Convert.ToString(row["RecordType"]));
 item.SubItems.Add(Convert.ToString(row["ContentPublic"]));
 item.Tag = row;
 _timeline.Items.Add(item);
 }
 if (_timeline.Items.Count > 0) ShowDetail(_timeline.Items[0]);
}

private void ShowDetail(ListViewItem item)
{
 var row = (DataRow)item.Tag;
 bool isWitch = _role == "Witch";
 _txtDetail.Text = isWitch
 ? Convert.ToString(row["ContentPublic"])
 : $"{row["ContentPublic"]}\n\n{row["ContentPrivate"]}";
}
```

技术亮点：时间轴与详情联动，按时间倒序展示；敏感内容按角色拆分展示，保证不同用户看到的记录粒度不同。

#### 8.6.4 地图图鉴 (MapForm)

地图图鉴提供岛屿与关键设施的平铺地图视图，支持缩放、平移、标记叠加和按权限过滤。管理员可查看全部岛屿；监管员和典狱长仅查看本岛；普通魔女仅查看公共区域与自身相关的路线标记。

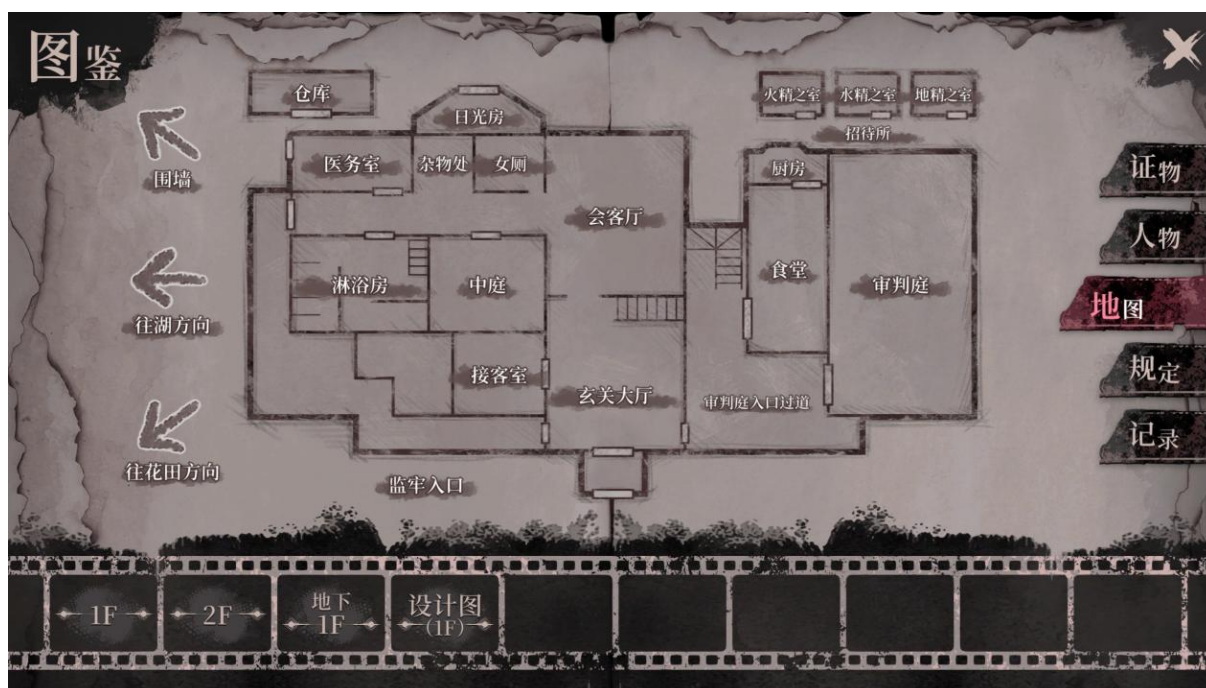


图 34 图鉴·地图界面

#### 1) 数据访问层 (DAL/MapDAL.cs) - 可见岛屿与标记

```
public (DataTable Islands, DataTable Markers) GetMapData(string username)
{
 var info = _permissionDal.GetUserInfo(username);
 string role = info.RoleName;
 int? islandId = info.IslandID;

 string islandSql = "SELECT IslandID, Name, MapImagePath FROM wt.Island WHERE 1=1";
 var islandPs = new List<SqlParameter>();
 if (role == "Meruru" || role == "Warden")
 {
 islandSql += " AND IslandID = @iid";
 islandPs.Add(new SqlParameter("@iid", islandId!.Value));
 }

 string markerSql = @"
SELECT MarkerID, IslandID, Type, Title, PosX, PosY, IsPublic, RelatedUserI
D
FROM wt.MapMarker WHERE 1=1";
 var markerPs = new List<SqlParameter>();
 if (role == "Meruru" || role == "Warden")
 {
 markerSql += " AND IslandID = @iid";
 markerPs.Add(new SqlParameter("@iid", islandId!.Value));
 }
}
```



```
else if (role == "Witch")
{
 markerSql += " AND (IsPublic = 1 OR RelatedUserID = @uid)";
 markerPs.Add(new SqlParameter("@uid", info.UserID));
}

var islands = DBHelper.ExecDataTable(islandSql, islandPs.ToArray());
var markers = DBHelper.ExecDataTable(markerSql, markerPs.ToArray());
return (islands, markers);
}
```

## 2) 表示层 (UI/MapForm.cs) - 缩放、标记叠加

```
private void LoadMap()
{
 var (islands, markers) = _mapDal.GetMapData(_username);
 _mapSelector.DataSource = islands;
 _mapSelector.DisplayMember = "Name";
 _mapSelector.ValueMember = "IslandID";

 if (islands.Rows.Count > 0)
 {
 ShowIsland(Convert.ToInt32(islands.Rows[0]["IslandID"]), markers);
 }
}

private void ShowIsland(int islandId, DataTable markers)
{
 var rows = markers.Select($"IslandID = {islandId}");
 _mapCanvas.SetBackground(LoadImage(GetMapPath(islandId)));
 _mapCanvas.ClearMarkers();
 foreach (var mr in rows)
 {
 _mapCanvas.AddMarker(
 Convert.ToString(mr["Type"]),
 Convert.ToString(mr["Title"]),
 Convert.ToSingle(mr["PosX"]),
 Convert.ToSingle(mr["PosY"]));
 }
}
```

技术亮点：自定义画布控件支持缩放与平移，标记层与底图解耦；数据按权限过滤后再绘制，避免前端持有无权限的坐标信息。

### 8.6.5 规定图鉴 (RulesForm, 自定义字体)

规定图鉴用于展示各类审判、处刑、岛屿管理的规则与流程说明。界面采用分级排



版：首行标题使用较大字体突出规则名称，后续正文使用较小字体呈现细节，支持富文本渲染与滚动浏览。



图 35 图鉴·规定界面

#### 1) 自定义字体与分级排版 (UI/RulesForm.cs)

```
// 预加载字体（可嵌入资源或从文件读取）
private readonly Font _titleFont = new Font("Bebas Neue", 26, FontStyle.Bold);
private readonly Font _bodyFont = new Font("Microsoft YaHei", 12, FontStyle.Regular);

private void ShowRule(DataRow row)
{
 string title = Convert.ToString(row["RuleTitle"]) ?? "规则";
 string content = Convert.ToString(row["RuleContent"]) ?? "";

 // 使用 RichTextBox/RtfBuilder 构造分级字体
 var sb = new StringBuilder();
 sb.Append(@"{\rtf1\ansi\deff0");
 sb.Append($@"{{\fonttbl{{\f0 { _titleFont.Name }}}}{\f1 { _bodyFont.Name }};}}");
 sb.Append(@"\viewkind4\uc1");
 sb.Append($@"\fs{(int)(_titleFont.SizeInPoints * 2)} {title}\line\line");
 sb.Append($@"\fs{(int)(_bodyFont.SizeInPoints * 2)} {content.Replace("\n", "\\line")}}");

 _rtb.Rtf = sb.ToString();
}
```





## 2) 数据来源与权限

- 规则数据存放在 `wt.Rule` 表, 含字段: `RuleID, RuleTitle, RuleContent, Scope, IslandID, IsPublic, SortOrder`。
- 管理员可查看全部规则; 监管员和典狱长仅查看本岛规则; 普通魔女仅查看公开规则。

## 3) 交互与体验

- 左侧列表展示规则目录, 按 `SortOrder` 排序; 点击后右侧 RichTextBox 显示内容。
- 支持关键词搜索; 搜索仅匹配当前用户有权限的规则集。
- 保留滚动位置与选中项, 便于来回切换时快速对比。

## 8.7 五子棋对弈系统

### 8.7.1 五子棋对局 (GomokuBoardForm)

五子棋对局界面包含大头像区、姓名缩略图区、棋盘容器和底部进度条, 支持多角色对弈并高亮当前执子方。人力资源与角色职责见 8.2, 此处聚焦 UI 与交互。

#### 1) 进入与开局配置

- Admin/主界面点击“五子棋”入口, 进入对局准备页。
- 输入对手密码/房间口令后创建或加入房间;
- 点击“确认”后锁定配置, 棋盘初始化并分配黑白子。



图 36 五子棋·入口界面

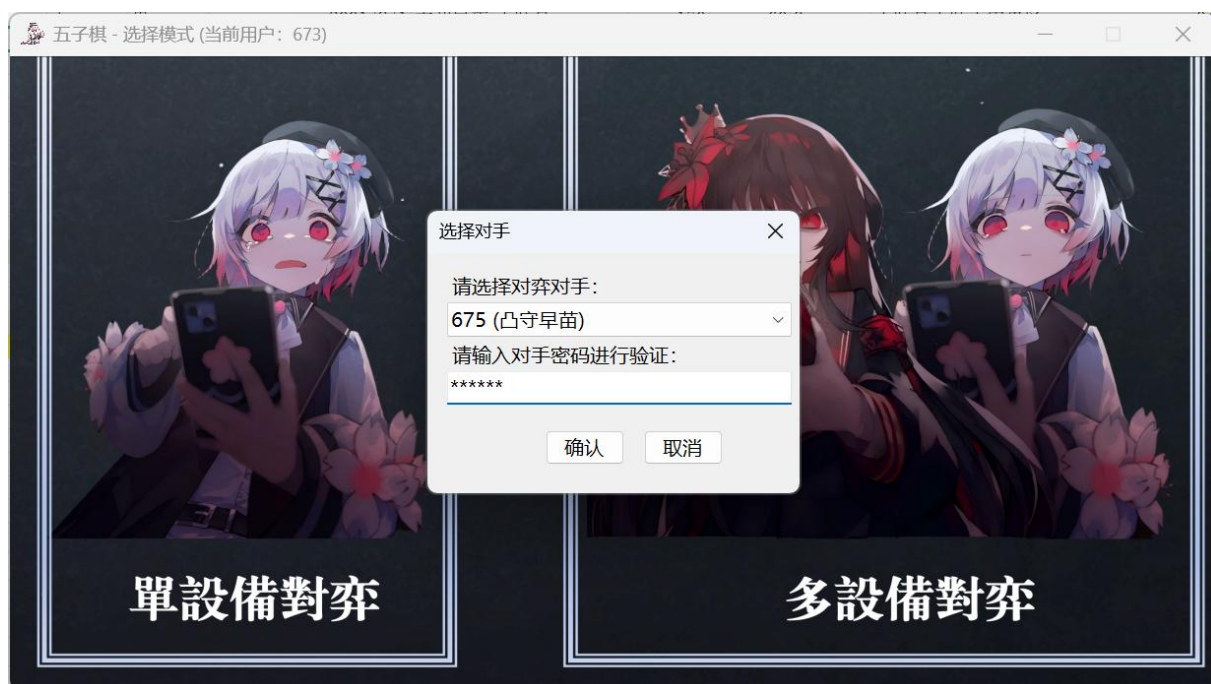


图 37 五子棋 · 创建房间功能

## 2) 对局界面布局

- 左侧：棋盘容器（自绘双缓冲），支持落子、最后一步高亮、禁手校验（黑棋长连/双活三/双四提示并拒绝落子）。
- 右侧：大头像+姓名，当前行动方高亮；下方有缩略图条可显示观战/候补玩家。
- 中部状态栏：“悔棋 / 认输 / 和棋”（对应“魔法”“疑问”“伪证”）按钮。

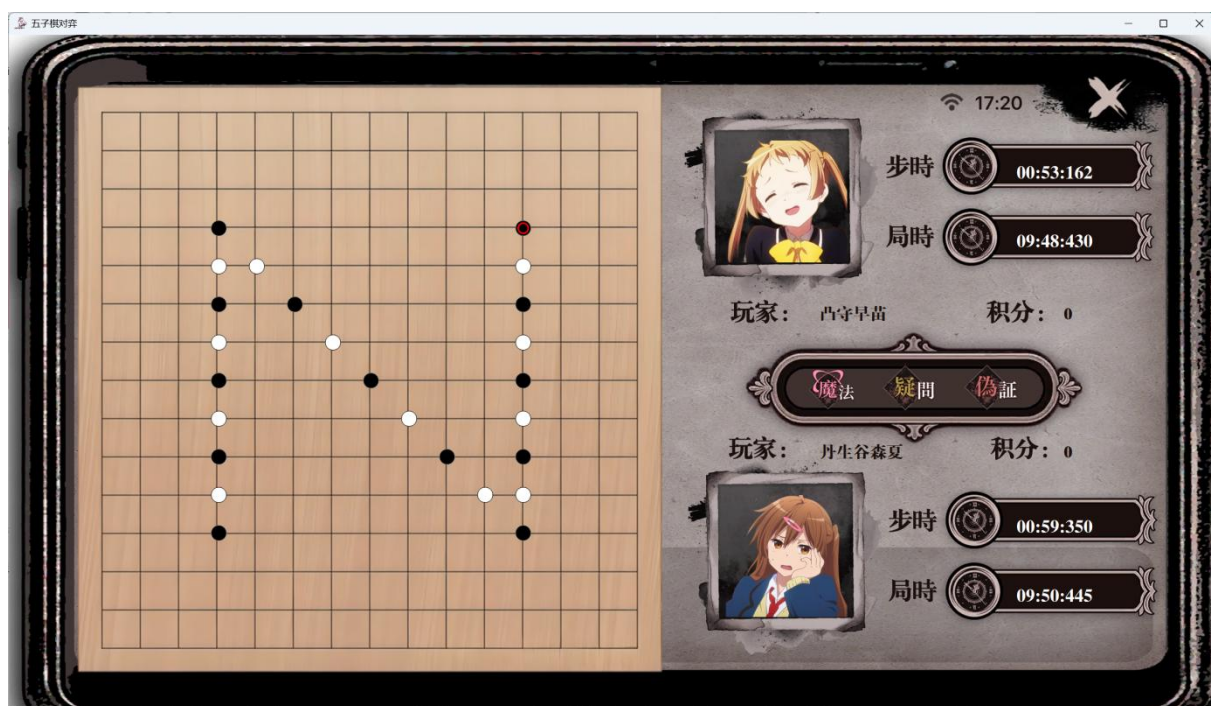


图 38 五子棋 · 对弈界面布局





### 3) 落子与判定

- 鼠标点击网格，调用 `GomokuEngine.TryPlace` 校验合法性；非法落子弹提示（越界、重复、禁手）。
- 合法落子后：播放落子音效，标记最后一步，重置单步计时，记录步序（用于悔棋与存档）。
- 每步后进行胜负判定，形成结果（黑胜/白胜/和棋/超时/认输）。

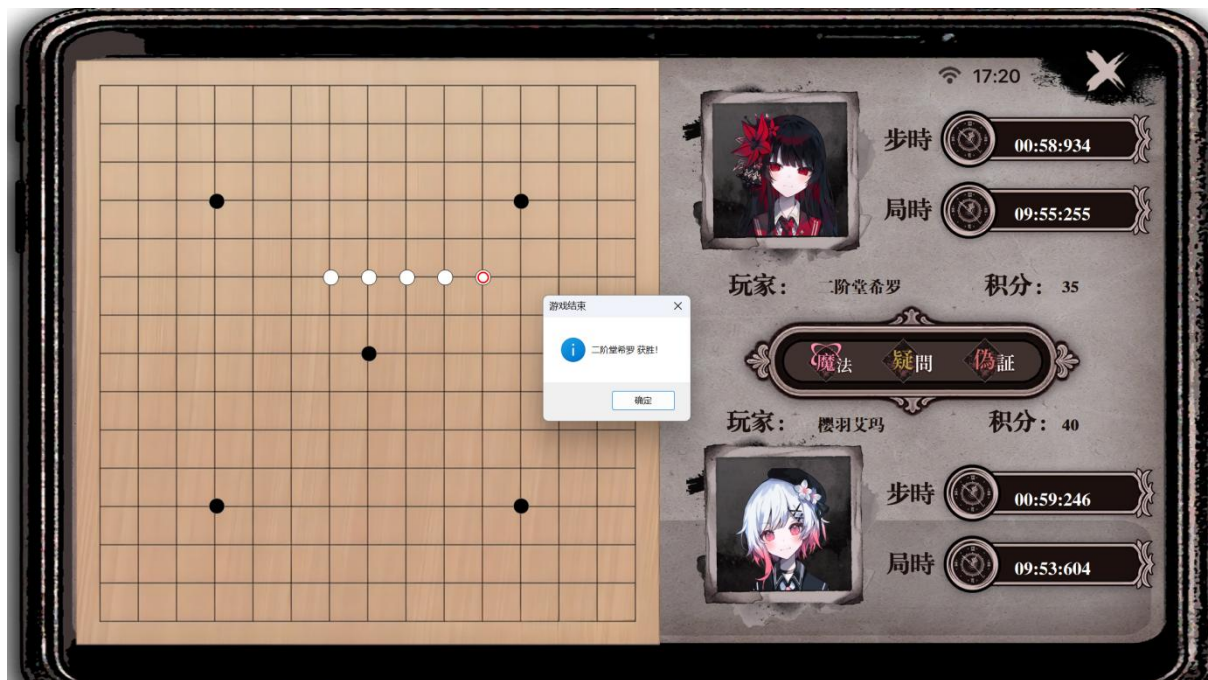


图 39 五子棋·胜负判定功能

### 4) 悔棋/和棋/认输

- 悔棋需双方同意：点击“魔法”发出请求，对方确认后回退最近若干手（通常 1-2 手），同步棋盘与计时。
- 和棋：任一方发起，双方同意则结束为和局。
- 认输：立即结束对局，结果写入日志与积分。

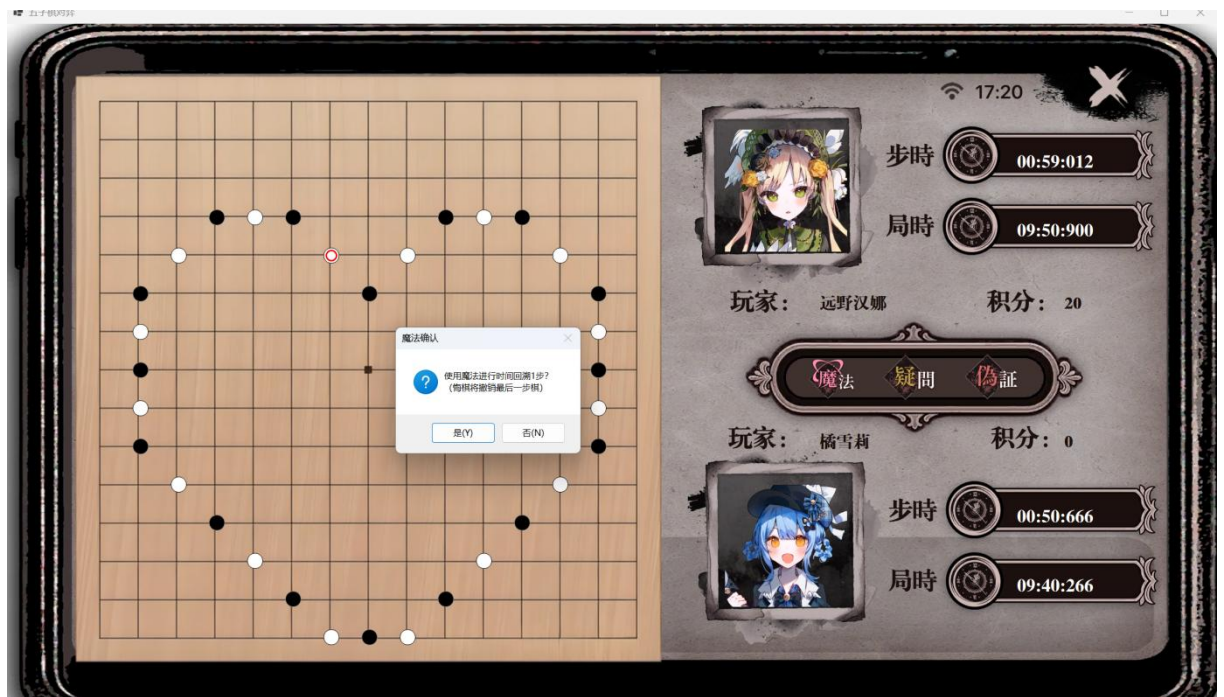


图 40 五子棋·悔棋功能

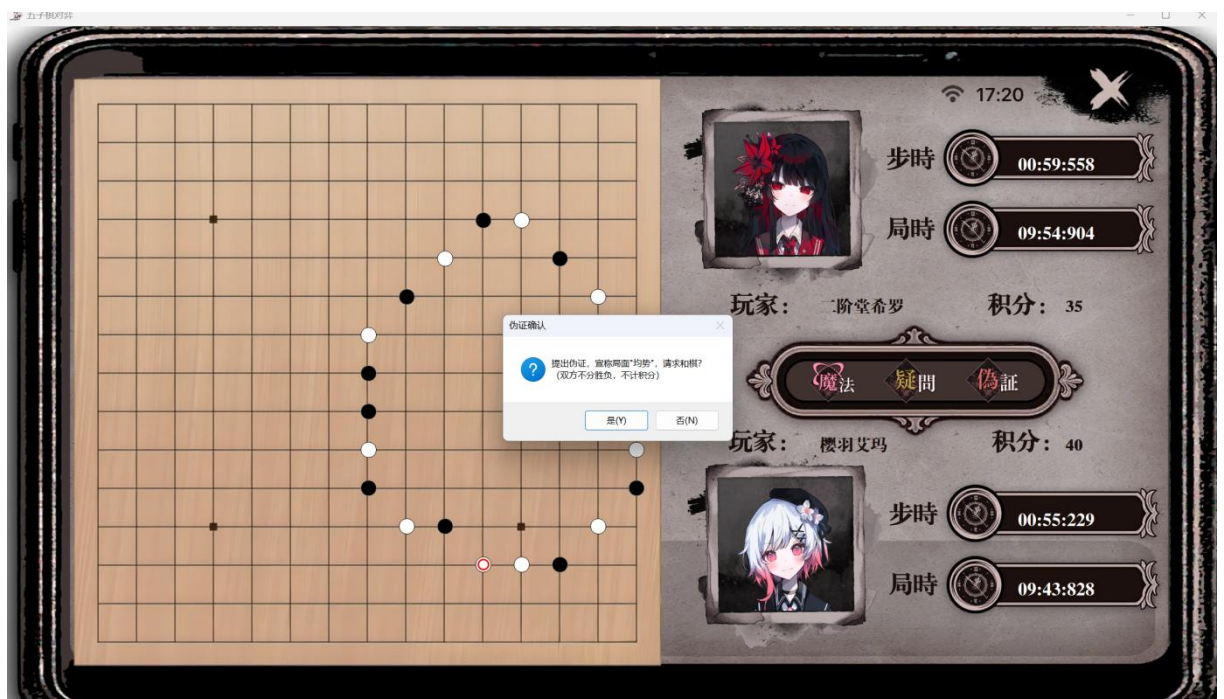


图 41 五子棋·和棋功能

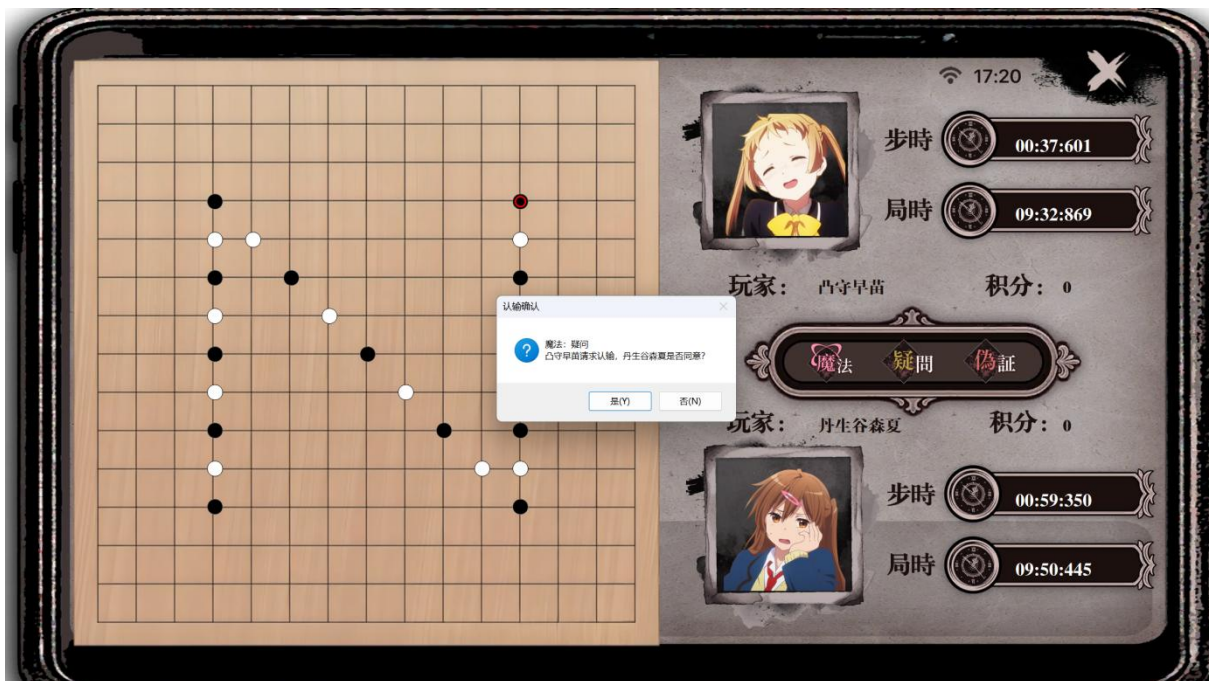


图 42 五子棋·认输功能

#### 5) 结束与记录

自动写入对局日志（见 8.7.3），包含 结果、分数变化、总步数。

#### 核心点：

- 棋盘容器：自定义绘制控件或双缓冲 PictureBox，支持落子、高亮最后一步、禁手判定。
- 头像与姓名：左右两侧显示头像和昵称，当前行动方高亮，底部缩略图区可列出观战或候补玩家。
- 进度条：底部 ProgressBar 表示单步计时或对局剩余时长，超时触发判负或扣分。

示例（关键事件）：

```
public partial class GomokuBoardForm : Form
{
 private readonly GomokuEngine _engine = new();
 private readonly ProgressBar _stepTimer = new();

 public GomokuBoardForm()
 {
 InitializeComponent();
 _engine.OnMovePlaced += OnMovePlaced;
 _engine.OnGameOver += OnGameOver;
 }

 private void OnBoardClick(object? sender, MouseEventArgs e)
 {

```





```
var pos = _boardCanvas.ToGrid(e.Location);
if (_engine.TryPlace(pos.X, pos.Y))
{
 _boardCanvas.AddStone(pos, _engine.CurrentPlayer);
 _stepTimer.Value = 0;
}
}
}
```

## 8.7.2 积分系统实现

### 1) 积分规则示例:

- 胜: +10 分; 和: 0 分; 负: -5 分

### 2) 数据表: `wt.GomokuScore` (UserID, Score, Win, Draw, Lose, Streak, UpdatedAt)。

### 3) 更新流程:

[1] 对局结束由 `GomokuEngine` 输出结果

[2] BLL 计算积分变动与连胜状态

[3] DAL 执行 `UPDATE wt.GomokuScore SET ... WHERE UserID=@uid`, 必要时插入新记录

## 8.7.3 对局日志 (GomokuMatchLogForm)

对局日志用于回放与分享。保存字段为: MatchID、黑白双方、落子序列、结果、开始/结束时间、评注 (Markdown)。

五子棋对局日志													
筛选类		全部对局		筛选		重置							
对局序号	玩家1四人番号	玩家1姓名	玩家2四人番号	玩家2姓名	开始时间	结束时间	玩家1结果	玩家1分数变化	玩家2结果	玩家2分数变化	总步数	时长(秒)	
1033	677	伊露娜	679	沙耶	2025-12-02 16:28:11	2025-12-02 16:29:21	Lose	-5	Win	10	8	69	
1032	658	樱羽艾玛	659	二阶堂希罗	2025-11-28 15:48:22	2025-11-28 15:48:43	Draw	0	Draw	0	1	20	
1031	685	阿良理琪舞	695	椎崎咲良	2025-11-27 14:38:45	2025-11-27 14:38:50	Draw	0	Draw	0	1	4	
1030	658	樱羽艾玛	696	月出Style	2025-11-26 23:48:32	2025-11-26 23:48:41	Draw	0	Draw	0	0	9	
1029	677	伊露娜	678	维多利加	2025-11-26 17:54:17	2025-11-26 17:54:54	Lose	-5	Win	10	6	36	
1028	675	凸守早苗	678	维多利加	2025-11-26 17:51:13	2025-11-26 17:51:23	Draw	0	Draw	0	6	10	
1027	658	樱羽艾玛	659	二阶堂希罗	2025-11-26 17:46:00	2025-11-26 17:46:08	Draw	0	Draw	0	1	7	
1026	678	维多利加	674	五月七日蔷薇	2025-11-26 17:07:59	2025-11-26 17:08:08	Lose	-5	Win	10	2	8	
1025	680	美兰	679	沙耶	2025-11-26 17:00:23	2025-11-26 17:00:29	Draw	0	Draw	0	0	5	
1024	680	美兰	681	席拉	2025-11-26 13:09:53	2025-11-26 13:10:38	Draw	0	Draw	0	33	45	
1023	682	琪琪	679	沙耶	2025-11-26 11:57:26	2025-11-26 11:57:38	Lose	-5	Win	10	10	11	
1022	677	伊露娜	679	沙耶	2025-11-26 01:44:22	2025-11-26 01:45:50	Draw	0	Draw	0	30	87	
1021	677	伊露娜	658	樱羽艾玛	2025-11-26 01:40:14	2025-11-26 01:40:20	Win	10	Lose	-5	9	6	
1020	677	伊露娜	658	樱羽艾玛	2025-11-26 01:39:37	2025-11-26 01:40:04	Draw	0	Draw	0	4	27	
1019	658	樱羽艾玛	659	二阶堂希罗	2025-11-23 22:38:40	2025-11-23 22:39:04	Draw	0	Draw	0	3	24	
1018	658	樱羽艾玛	659	二阶堂希罗	2025-11-21 16:00:18	2025-11-21 16:01:34	Lose	-5	Win	10	0	76	
1017	658	樱羽艾玛	659	二阶堂希罗	2025-11-21 15:58:52	2025-11-21 15:59:17	Draw	0	Draw	0	8	24	
1016	658	樱羽艾玛	659	二阶堂希罗	2025-11-21 15:54:28	2025-11-21 15:54:53	Draw	0	Draw	0	5	24	
1015	658	樱羽艾玛	659	二阶堂希罗	2025-11-21 14:51:51	2025-11-21 14:53:24	Lose	-5	Win	10	12	92	
16	667	橘雪莉	668	远野议那	2025-11-20 23:57:37	2025-11-21 00:07:12	Draw	0	Draw	0	225	574	
15	667	橘雪莉	668	远野议那	2025-11-20 23:47:57	2025-11-20 23:49:42	Draw	0	Draw	0	31	104	
14	667	橘雪莉	668	远野议那	2025-11-20 21:30:53	2025-11-20 21:31:06	Draw	0	Draw	0	7	12	
13	658	樱羽艾玛	659	二阶堂希罗	2025-11-20 21:30:06	2025-11-20 21:30:21	Win	10	Lose	-5	9	15	
12	658	樱羽艾玛	659	二阶堂希罗	2025-11-20 21:29:25	2025-11-20 21:29:34	Draw	0	Draw	0	3	8	
11	658	樱羽艾玛	664	宝生玛玛	2025-11-20 21:14:43	2025-11-20 21:15:00	Win	10	Lose	-5	11	17	
10	658	樱羽艾玛			2025-11-20 21:02:12	2025-11-20 21:03:26	Lose	-5	Win	10	0	74	
9	661	城崎诺亚	668	远野议那	2025-11-20 20:57:20	2025-11-20 20:57:37	Lose	-5	Win	10	10	16	
8	658	樱羽艾玛	666	紫藤紫雨莎	2025-11-20 20:55:45	2025-11-20 20:56:08	Lose	-5	Win	10	10	23	
7	661	城崎诺亚	660	夏目安安	2025-11-20 20:39:28	2025-11-20 20:39:58	Win	10	Lose	-5	9	29	
6	661	城崎诺亚	658	樱羽艾玛	2025-11-20 20:31:39	2025-11-20 20:32:17	Draw	0	Draw	0	4	38	
5	661	城崎诺亚	669	泽渡可可	2025-11-20 20:30:27	2025-11-20 20:30:54	Lose	-5	Win	10	10	27	
4	658	樱羽艾玛	659	二阶堂希罗	2025-11-20 19:42:48	2025-11-20 19:43:04	Lose	-5	Win	10	10	16	
3	661	城崎诺亚	662	底见藤理	2025-11-20 19:41:06	2025-11-20 19:41:17	Lose	-5	Win	10	10	11	
2	661	城崎诺亚	670	冰上梅露露	2025-11-20 19:39:29	2025-11-20 19:39:45	Win	10	Lose	-5	11	15	
1	658	樱羽艾玛	670	冰上梅露露	2025-11-20 19:23:12	2025-11-20 19:23:22	Lose	-5	Win	10	10	9	

图 43 五子棋·对局日志界面



加载与渲染示例：

```
private void LoadLog(string matchId)
{
 var log = _logDal.Get(matchId);
 _markdownViewer.Render(log.CommentMd);
 _moves = ParseMoves(log.MovesJson);
 _boardCanvas.Replay(_moves);
}
```

技术点：

- MovesJson 以紧凑格式存储落子（如 `[{"x":7,"y":7,"p":1},...]`）。
- Markdown 渲染可用 Markdig 转 HTML + WebBrowser，或第三方 Markdown 控件。
- 支持导出 `.md` 或 `.html`，便于审核与分享。

#### 8.7.4 排行榜功能

排行榜按积分排序，支持全局榜与分岛榜，显示头像、昵称、积分、胜/和/负、最长连胜。界面顶部提供 4 张地图的切换，用于分区展示。



图 44 五子棋·积分排行榜界面



数据获取:

```
public DataTable GetRanking(string? islandName = null, int top = 50)
{
 string sql = @"
 SELECT TOP (@top) u.UserID, u.Username, u.AvatarPath,
 s.Score, s.Win, s.Draw, s.Lose, s.Streak, i.Name AS IslandName
 FROM wt.GomokuScore s
 JOIN wt.[User] u ON u.UserID = s.UserID
 LEFT JOIN wt.Island i ON i.IslandID = u.IslandID
 WHERE 1=1";
 var ps = new List<SqlParameter> { new("@top", top) };
 if (!string.IsNullOrEmpty(islandName))
 {
 sql += " AND i.Name = @island";
 ps.Add(new SqlParameter("@island", islandName));
 }
 sql += " ORDER BY s.Score DESC, s.Win DESC, s.Streak DESC";
 return DBHelper.ExecDataTable(sql, ps.ToArray());
}
```

UI 关键点:

- 左上角地图切换 (4 张地图) 过滤榜单。
- 列表行展示头像缩略图与积分概览, 点击可展开最近对局摘要。
- 支持按胜率、连胜排序的二级排序选项。

## 8.8 数据可视化大屏 (LiveCharts)

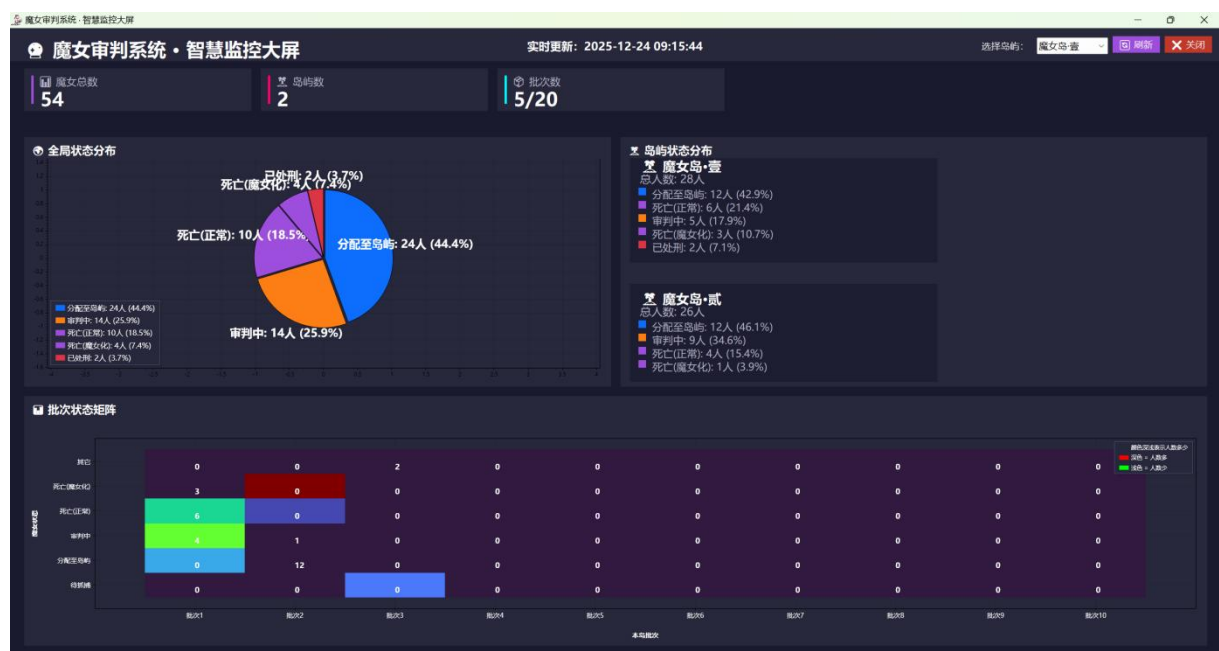


图 45 可视化智慧大屏





### 8.8.1 核心 KPI 指标区

主要展示:

顶部 KPI 卡, 显示关键数值, 即魔女总数, 岛屿数, 批次数和最后更新时间。

核心代码:

```
```csharp
// DashboardForm.cs
var kpi = dashboardService.GetDashboardKpi(islandId: null);
lblTotalWitches.Text = kpi.TotalWitches.ToString();
lblActiveSessions.Text = kpi.ActiveSessions.ToString();
lblVotingSessions.Text = $"{kpi.VotingSessions}/{kpi.TotalSessions}";
lblExecuted.Text = kpi.Executed.ToString();
// 点击钻取
lblVotingSessions.Click += (_, __) => OpenSessions(SessionFilter.Voting);
```
```

### 8.8.2 全局状态分布区

主要展示:

中左大饼图, 展示系统内各状态的绝对数与百分比, 带图例与 Tooltip, 可点击扇区触发过滤。

核心代码:

```
```csharp
// DashboardForm.cs
var statusList = dashboardService.GetStatusDistribution(islandId:
currentIslandId);
pieChart.Series.Clear();
foreach(var it in statusList){
    pieChart.Series.Add(new PieSeries {
        Title = it.Label,
        Values = new ChartValues<int> { it.Count },
        DataLabels = true,
        LabelPoint = chartPoint => $"{(int)chartPoint.Y}
({chartPoint.Participation:P0})"
    });
}
pieChart.Update(true, true); // 刷新
```
```

### 8.8.3 分岛屿状态分布区

主要展示:

右侧岛屿摘要面板列表, 每个面板显示岛屿名称、总数与状态分解(数值或小图), 支持点击聚焦某岛屿。



核心代码:

```
```csharp
// DashboardForm.cs - 渲染岛屿面板
var islands = dashboardService.GetIslandStatistics();
panelIslands.Controls.Clear();
foreach(var isl in islands){
    var p = CreateIslandPanel(isl); // 返回 Control
    p.Click += (_,__) => RefreshChartsForIsland(isl.Id);
    panelIslands.Controls.Add(p);
}
```
```

#### 8.8.4 批次状态矩阵区

主要展示:

底部批次矩阵, 横轴为批次, 纵轴为状态类别, 每个单元显示对应数量, 支持悬停查看与点击钻取。

核心代码:

```
```csharp
// DashboardForm.cs - 绑定批次矩阵
var matrix = dashboardService.GetBatchStatusMatrix(granularity: "day", islandId:
currentIslandId);
barChart.Series = BuildStackedSeries(matrix); // 返回 SeriesCollection
barChart.AxisX[0].Labels = matrix.BatchLabels.ToArray();
```
```

#### 8.8.5 岛屿选择功能

主要展示:

右上站点/岛屿下拉(包含“全部”), 切换后刷新仪表盘所有视图并显示加载与更新时间。

核心代码:

```
```csharp
// DashboardForm.cs - 站点选择
cmbIslands.DataSource = dashboardService.GetIslands();
cmbIslands.DisplayMember = "Name";
cmbIslands.ValueMember = "Id";
cmbIslands.SelectedIndexChanged += (_,__) => Debounce(() => {
    var id = ((Island)cmbIslands.SelectedItem).Id;
    RefreshAllForIsland(id);
}, 300);
```
```



## 8.9 智慧大模型

大模型集成豆包 1.6 大模型，主要用于审判咨询、证物关联分析和系统问答。在 Admin 工具栏提供“智慧大模型”入口，自动加载核心文档作为知识库（README、项目架构说明、数据库结构文档、系统界面跳转与权限层级说明、四层权限体系说明、CHANGELOG），以便基于项目文档提供准确回复。



图 46 智慧大模型界面

服务设计：

- `BLL.AIService` 统一封装调用，屏蔽 Key 读取（参见 7.3.4），暴露 `AnalyzeEvidence`、`SummarizeTrial`、`Chat` 等业务方法。
- Prompt 模板可配置，放在资源/配置文件，使用占位符插值（如 `{facts}`、`{votes}`）便于热更新。
- 调用策略：设置超时、重试与退避；支持按任务类型路由不同模型（生成/推理/本地轻量）。
- 安全与审计：输入做敏感词过滤，输出做长度与内容校验；记录请求摘要、耗时与状态，不记录明文 Key。

示例：

```
public class AIService
{
 private readonly ILLMClient _client;
 public AIService(ILLMClient client) => _client = client;

 public string SummarizeTrial(string facts, string votes)
 {
 var prompt = PromptTemplates.TrialSummary
 .Replace("{facts}", facts)
 .Replace("{votes}", votes);
 }
}
```



```
 return _client.Chat(prompt, maxTokens: 512, temperature: 0.2);
 }
}
```

## 8.10 录音机界面

录音机界面负责录制、暂停、结束、保存、播放、删除，并仅显示当前账号的录音。录音文件按账号编号存储到 `UI/recorder/{编号}/`，编号范围默认 658-721（用户名可为编号或普通用户名，通过 UserDAL 取 UserID，超出范围用哈希回退）。

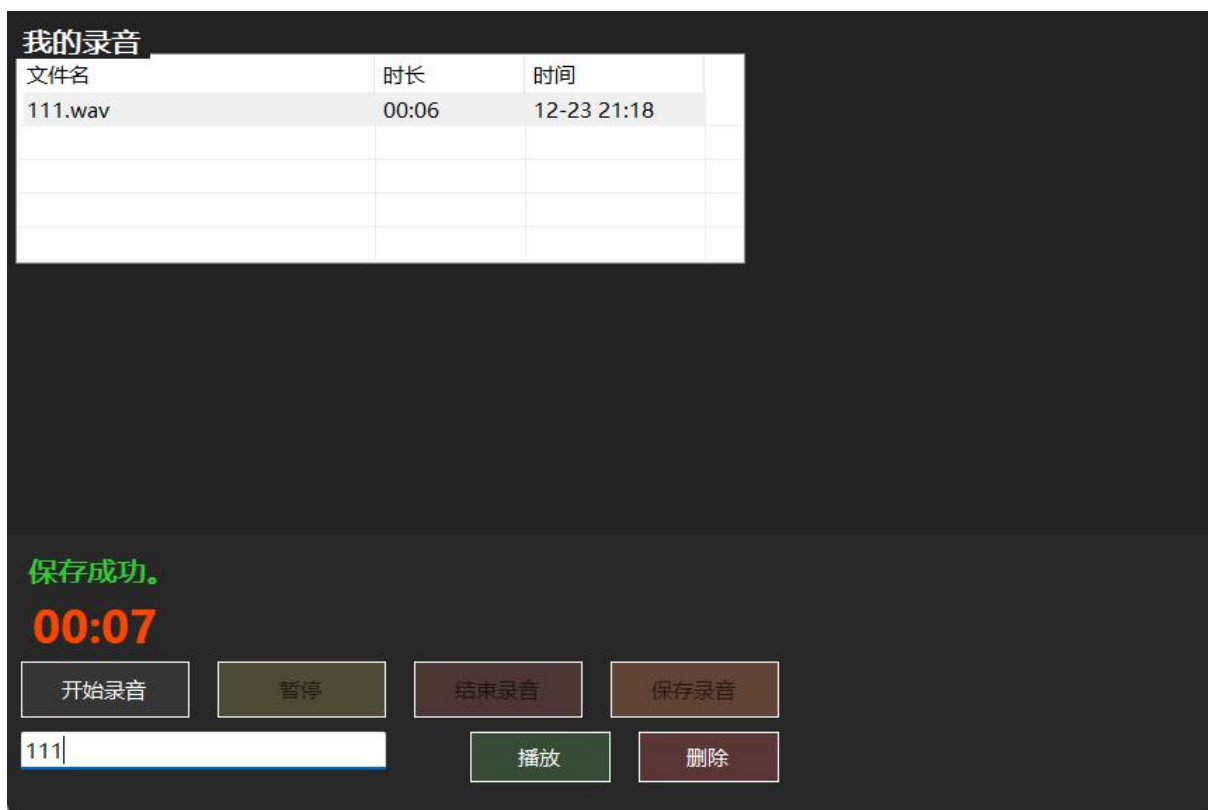


图 47 录音界面

界面实现要点：

- 控件布局：上部 ListView 展示“文件名/时长/时间”，下部按钮区含“开始、暂停/继续、结束、保存、播放、删除”及标题输入框、状态与计时显示。
- 录制流程：使用 NAudio `WaveInEvent` 采集单声道 44.1kHz，写入 `WaveFileWriter`；计时通过 WinForm Timer，每秒刷新；暂停仅停止写入但不终止设备。
- 文件命名：开始录制前由 `RecordingService.BuildNewFilePath` 生成 `recording\_YYYYMMdd\_HHmss.wav`，录制完成后用户输入标题重命名（非法字符替换，支持覆盖确认）。
- 列表管理：`RecordingService.ListRecordings` 枚举目录下的 WAV 文件，估算时长（按 44.1kHz/16bit/mono 约 10KB/s），按创建时间降序；ListView 选中后可播放/删除，列宽自适应窗口。
- 播放控制：使用 `WaveOutEvent + AudioFileReader`，播放结束自动还原状态；播放前自动停止正在录制的会话。
- 异常与状态：所有 IO/音频异常弹窗提示，录制状态用文字+颜色反馈，窗口关闭时确保录制与播放资源释放。



代表性代码：

```
// UI/RecordingForm.cs - 开始/停止/保存核心逻辑
private void StartRecording()
{
 StopPlayback();
 _currentFile = _service.BuildNewFilePath(_username);
 Directory.CreateDirectory(Path.GetDirectoryName(_currentFile!));

 _waveIn = new WaveInEvent { WaveFormat = new WaveFormat(44100, 1) };
 _waveIn.DataAvailable += (s, e) =>
 {
 if (_isPaused || _writer == null) return;
 _writer.Write(e.Buffer, 0, e.BytesRecorded);
 _writer.Flush();
 };
 _writer = new WaveFileWriter(_currentFile, _waveIn.WaveFormat);
 _waveIn.StartRecording();
 _isRecording = true;
 _timer = new Timer { Interval = 1000 };
 timer.Tick += (, __) =>
 {
 if (!_isPaused) _lblTime.Text = RecordingService.FormatDuration(TimeSpan.FromSeconds(++_seconds));
 };
 _timer.Start();
}

private void StopRecording()
{
 _waveIn?.StopRecording();
 _waveIn?.Dispose();
 _writer?.Dispose();
 _timer?.Stop();
 _isRecording = false;
 _isPaused = false;
 _btnSave.Enabled = HasValidFile();
}

private void SaveRecording()
{
 if (!HasValidFile()) return;
 var title = Sanitize(_txtTitle.Text);
 var dir = Path.GetDirectoryName(_currentFile!);
 var target = Path.Combine(dir, $"{title}.wav");
 if (File.Exists(target)) File.Delete(target);
}
```



```
File.Move(_currentFile!, target);
_currentFile = target;
LoadRecordings();
}

// BLL/RecordingService.cs - 文件路径与编号映射
private int GetUserFolderNumber(string username)
{
 if (int.TryParse(username, out int n) && n >= 658 && n <= 721) return n;
 var user = new UserDAL().GetByUsername(username);
 if (user.HasValue) return user.Value.UserID;
 int hash = Math.Abs(username.GetHashCode());
 return 658 + (hash % 64); // 落在 658-721 区间
}

public string BuildNewFilePath(string username)
{
 string folder = Path.Combine(AppContext.BaseDirectory, "UI", "recorder", GetUserFolderNumber(username).ToString());
 string timestamp = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
 return Path.Combine(folder, $"recording_{timestamp}.wav");
}
```

存储与权限：

- 文件夹定位：若用户名是 658-721 的数字直接用；否则取 UserID，若不在范围则仍用 UserID；查询失败则按用户名哈希生成 658-721 之间的编号，确保隔离。
- 删除：`RecordingService.DeleteRecording` 做存在性校验后删除；失败抛异常由 UI 捕获提示。

## 8.11 对话界面

对话界面为同岛同批次的魔女提供即时沟通渠道，用于交换情报、协调行动、反馈异常（如处刑平台状态、投票异常、证物发现），并在本地留存可审计的简易聊天记录。



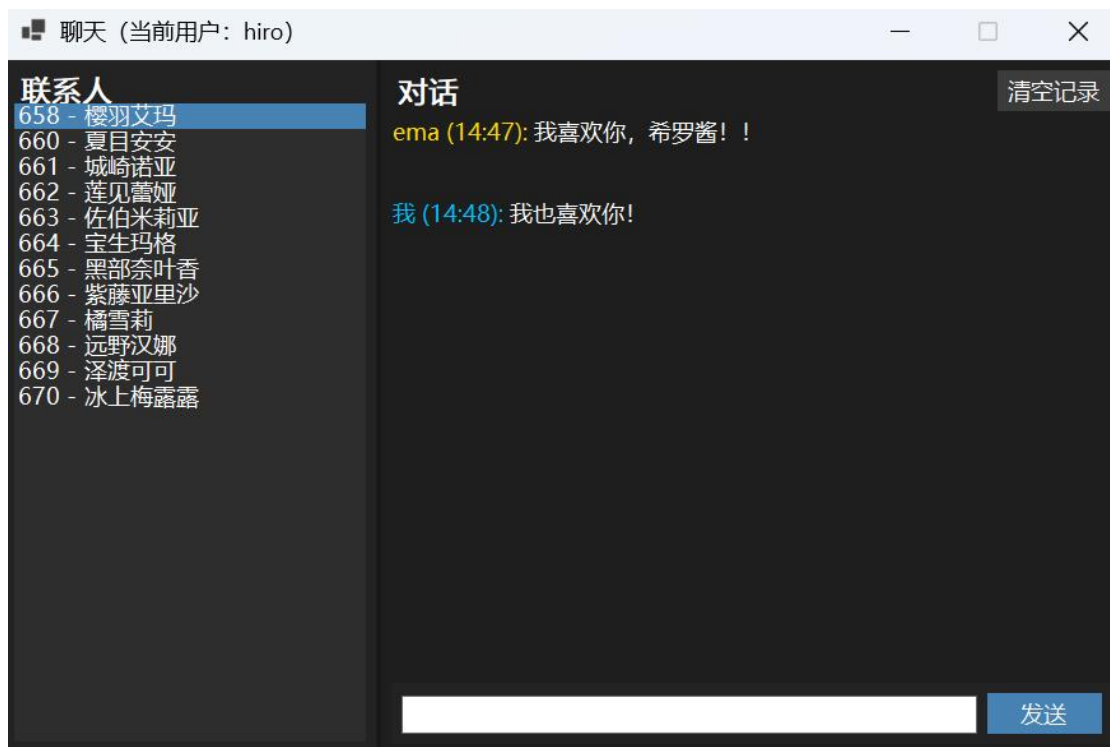


图 48 对话界面

界面组成:

- 联系人来源: 从数据库筛选“同岛、同批次、角色=魔女(RoleID=4)”的用户, 按囚犯编号排序, 排除当前用户。
- 布局: 左侧联系人 (OwnerDraw, 可高亮未读), 右侧对话区 (RichTextBox), 下方输入与发送/清空按钮, Enter 发送。
- 本地存储: 聊天记录保存在 `Data/ChatLogs/{当前用户}/{对方用户名}.txt`, 进入联系人时读取, 发送后立即落盘; 推断有未读记录时高亮联系人。
- 状态与显示: 按时间顺序渲染, 标题/时间颜色区分“我/对方”, 自动滚动到底部。

代表性代码 (`UI/ChatForm.cs`):

```
// 同岛同批次联系人查询
const string sqlContacts = @"
SELECT DISTINCT u.Username,
 ISNULL(w.PrisonerNo, u.Username) AS PrisonerNo,
 ISNULL(w.Name, u.Username) AS WitchName
FROM wt.[User] u
LEFT JOIN wt.UserWitch uw ON uw.UserID = u.UserID
LEFT JOIN wt.Witch w ON w.WitchID = uw.WitchID
WHERE u.RoleID = 4 AND u.IslandID = @IslandID AND u.BatchID = @BatchID
 AND u.Username <> @Username
ORDER BY PrisonerNo, WitchName";

// 发送并写入本地记录
var msg = new ChatMessage { Sender = _username, Text = text, Time = DateTime.Now };
list.Add(msg);
AppendMessageToView(msg);
File.AppendAllText(GetConversationFilePath(_currentContact),
 $"{msg.Sender}|{msg.Time:o}|{msg.Text}\n");
```



## 9 项目总结与展望

### 9.1 项目完成情况

WitchTrialSystem 是一款面向“魔女审判”特定业务场景的桌面应用系统，基于 C# WinForms 框架开发，采用“UI 层 - 业务逻辑层 (BLL) - 数据访问层 (DAL)”三层架构设计。系统深度还原《魔法少女的魔女审判》游戏核心场景，覆盖审判会话全生命周期管理、多角色投票交互、处刑流程执行、基础数据管理（用户、魔女、证物等）及可视化展示等核心功能。项目配套完整的本地资源包（含头像、背景图、自定义字体、音效文件）与数据库部署脚本，支持本地快速搭建开发与测试环境。

#### 9.1.1 功能完成度统计

已完成核心功能（满足课程设计核心要求）

- 魔女图鉴：人物、地图、证物、规定、记录
- 审判会话管理：支持审判会话的创建、状态查询、流程维护（Pending→Voting→Confirmed→Executing→Completed/Cancelled）。
- 投票交互模块：实现参会魔女列表展示、投票对象选择、投票提交、投票成功反馈及等待界面渲染，支持单机多账号切换投票。
- 投票进度展示：通过客户端轮询机制，实时获取并展示投票进度（已投票人数 / 总参与人数）。
- 典狱长专属流程：包含处刑对象宣布、处刑确认触发、处刑流程启动等核心操作。
- 处刑执行界面：实现处刑对象信息（大图、姓名）展示、确认交互及处刑执行跳转。
- 基础数据管理：提供用户、魔女、证物、记录、处刑平台、大模型等核心数据的增删改查界面与接口。
- 部署支持：配套建表脚本、数据导入脚本、备份与还原脚本，含示例数据库文件（WitchTrialWT.mdf/WitchTrialWT\_log.ldf），可直接用于本地验证。
- 五子棋对弈、积分排行榜，对局记录，对话，录音

#### 9.1.2 代码规模统计

源码文件总数（.cs）：86 个。

按主要目录统计（仓库当前状态）：

- `UI`：40 个`.cs`文件（窗体与 UI 逻辑为主）
- `BLL`：11 个`.cs`文件（业务服务与逻辑封装）
- `DAL`：16 个`.cs`文件（数据访问与 SQL 调用）
- `Models`：5 个`.cs`文件（实体与视图模型）



Designer 文件：9 个（自动生成的 UI 布局代码）

代码行数：共计 24,989 行（所有`.cs`文件总和）。

### 9.1.3 数据库规模统计

脚本数量：`database\_scripts`下包含 32 个 SQL 脚本，覆盖建表、存储过程、迁移、导入与备份恢复等场景。

示例数据库：仓库包含用于开发/测试的`WitchTrialWT.mdf`与`WitchTrialWT\_log.ldf`，便于本地带数据运行与验证。

## 9.2 技术难点与解决方案

本节针对在实现过程中遇到的若干关键技术问题，逐条说明背景、影响与已采取或建议的解决方案。

### 9.2.1 43 字段魔女档案的数据建模

问题描述：魔女档案包含多维属性（基础信息、状态字段、历史记录、标签与多媒体引用），合计字段较多（示例为 43 个字段），需在设计上兼顾查询性能与扩展性。

解决思路：

1. 将常用查询列（ID、姓名、状态、所属平台等）放在主表，保证单表查询高效；较少使用或可扩展的属性（长文本、历史轨迹、多媒体路径）拆分到从表或外部存储。
2. 对多媒体资源仅存储路径或资源 ID，避免将大字段直接保存在行内。
3. 根据查询模式建立必要索引（按会话、状态、姓名模糊检索等需求优化）。

### 9.2.2 四层权限体系的数据隔离

问题描述：系统对不同角色（管理员、典狱长、监管者、普通魔女）权限区分明确，需要在数据访问层实现有效的数据隔离与权限校验。

解决思路：

1. 在 DAL 层统一实现基于用户角色的查询过滤。
2. 对关键操作在 BLL 层进行二次校验，确保权限在业务层面也受控。
3. 对审计敏感操作（宣布处刑、数据导入）增加操作日志记录并写入审计表，便于事后追溯。

### 9.2.3 审判投票的状态流转与并发控制

问题描述：投票流程涉及多个并发写入操作（多用户同时投票、典狱长宣布状态变化），需保证数据一致性并避免竞态条件。

解决思路：

1. 在数据库层使用事务和合适的隔离级别确保投票计数的一致性；对关键更新使用行级锁或乐观并发控制（版本号/时间戳）。
2. 在提交投票接口处加入幂等性设计与重试策略，避免重复计票或丢票。



3. 在状态变更路径 (Voting → Confirmed → Executing) 设置原子化操作, 典狱长宣布时进行状态校验并触发通知机制。

### 9.2.4 处刑台位置管理的数据库设计

问题描述: 处刑台 (ExecutionPlatform) 为运行时实体, 包含位置、状态、历史移动日志等, 需要支持并发调度与轨迹记录。

解决思路:

1. 将处刑台信息拆分为静态信息表 (平台属性) 与运行时状态表 (当前占用、所属会话)。
2. 对移动与操作记录单独建表 (MovementLog), 记录时间、来源、目标与操作摘要, 便于回溯。
3. 对高并发调度 (同一平台被多方请求) 在业务层加入锁定机制或采用悲观锁以避免冲突。

### 9.2.5 单机多账号切换的状态持久化

问题描述: 应用支持同一客户端下快速切换账号并继续投票/查看进度, 需在本地保存会话状态与部分用户缓存, 同时保证安全和一致性。

解决思路:

1. 在本地使用轻量化持久化 (文件或本地 DB) 缓存非敏感会话视图, 切换账号时快速还原 UI 状态。
2. 对敏感操作 (投票提交) 仍通过服务器校验与实时同步, 避免本地缓存造成数据不一致。
3. 在切换流程中提示用户未提交更改并在必要时进行冲突合并或回滚。

### 9.2.6 C# Winform 前端美工设计

问题描述: 使用 WinForms 实现接近“手机视觉”的界面需处理控件透明、背景合成、不同分辨率及视觉一致性问题。

解决思路:

1. 将大部分视觉效果在资源层 (PNG 背景、遮罩、合成图) 完成, 运行时尽量使用透明控件覆盖以减少重绘差异。
2. 在窗体布局中使用父容器统一管理背景, 并在 Layout 事件中动态计算控件位置以保证在固定窗口下的视觉效果。
3. 对关键控件设计替代样式 (例如为高 DPI 提供更大尺寸的资源)。

## 9.3 项目创新点

### 9.3.1 三层权限体系的精细化设计

项目在权限设计上实现了界面与数据层的双重约束, 既在 UI 层控制功能按钮可见性, 又在 BLL/DAL 层实现数据级别的过滤与校验, 提升了安全性与可审计性。



### 9.3.2 复杂业务流程的数据库建模

项目将审判、投票、处刑等复杂业务流程拆分为清晰的表结构与状态机，结合迁移脚本与存储过程实现了可重复部署的数据库模型。

### 9.3.3 游戏场景与数据库的深度融合

项目将游戏化的场景（头像、处刑台、移动日志）以结构化数据方式记录，既满足展示需求又利于统计、回放与审计。

### 9.3.4 复杂业务流程的数据库建模

通过资源目录（Images/Fonts/sounds）与统一的加载策略，项目实现了界面资源的模块化管理，便于替换与扩展。

## 9.4 不足与改进方向

不足要点：

1. 实时通信仍依赖轮询，扩展性与效率有待提升。
2. UI 对多分辨率支持不足，需测试并补充自适应策略。
3. 自动化测试覆盖率低，影响后续变更的回归保障。
4. 用户安装不便，需要安装“SQL Server”基本全套和 localDB。

改进建议：

1. 后端引入实时推送（SignalR 或 WebSocket），并在客户端实现事件驱动更新。
2. 优化前端布局与资源，或将关键界面迁移到 WPF 以获得更好缩放与渲染支持。
3. 为 BLL 与 DAL 添加单元测试与集成测试，配置 CI 流水线以保证代码质量。
4. 在前期选用技术路线应慎重，采用 SQLite，或开发手机端，让用户“开箱即用”。

## 9.5 个人收获与体会

历时两月，我们小组完整走完了从需求分析、系统设计、全栈开发到最终部署的项目全流程。这段实践中，我不仅掌握了分模块协作开发、Git 版本管理、界面美工设计、数据库全链路构建等核心技术，更在“从需求到落地”的过程中，建立了从业务逻辑到技术实现的闭环思维。

项目里沉淀的架构设计、权限管控等系统化方法，也让我们意识到技术思维的跨领域适配性——这些经验完全可以迁移至智能交通等领域的系统开发，既帮突破了交通运输专业的能力边界，也为后续研究生阶段的跨学科研究打下了坚实基础。

开发过程中，多位同学主动参与测试、提出优化建议，不仅帮我们打磨了项目细节，更拓宽了技术思路；特别感谢老师允许我们自主选题，充足的发挥空间让我们能把兴趣与专业实践深度结合，真正实现了“在探索中实践”。

这次从 0 到 1 的完整项目，是知识、能力与协作经验的三重沉淀，让我对“技术落地”有了更真切的认知，受益匪浅！





## 参考文献

- [1] 维基百科 contributors. 魔法少女的魔女审判 [EB/OL]. 维基百科基金会, [2025 年 12 月 7 日 14:27] [2025 年 12 月 18 日]. .
- [2] 百度百科 魔法少女的魔女审判 [EB/OL]. 百度百科, [2025 年 12 月 12 月 18 日] [2025 年 12 月 18 日]. .
- [3] [美] Jesse Schell. 游戏设计艺术 (第 3 版) [M]. 刘嘉俊, 译. 北京: 电子工业出版社, 2021: 144.